Machine Learning

From Intuition to Implementation



A textbook for BCA students
Course code NBCA-601

Unit I Introduction · Unit II Supervised Learning · Unit III Ensembles & SVM
Unit IV Unsupervised Learning · Unit V Neural Networks

How to Use This Book

Learning Objectives

Every chapter opens with a box like this one, telling you exactly what you will be able to do by the end.

Key Idea

The one idea to remember. If you remember nothing else, remember this box.

Definition: Term

Formal definitions you may be asked to write in exams.

Example

Worked examples with real numbers, solved step by step.

Common Mistake

Mistakes students commonly make, and how to avoid them.

Try It Yourself

Short Python experiments. Run them, change numbers, and watch what happens.

Exam Tip

Frequently asked exam questions and how to answer them.

Contents

1 Introduction to Machine Learning **1**

1.1 Why Do We Need Machine Learning? . . . 1

1.2 Artificial Intelligence, Machine Learning and Deep Learning . . . 3

1.3 Where Is Machine Learning Used? . . . 5

1.4 Basic Terminology . . . 6

1.5 Types of Machine Learning Systems . . . 7

1.5.1 Supervised Learning . . . 8

1.5.2 Unsupervised Learning . . . 9

1.5.3 Semi-supervised Learning . . . 11

1.5.4 Reinforcement Learning . . . 11

1.5.5 Batch vs. Online Learning . . . 13

1.5.6 Instance-based vs. Model-based Learning . . . 15

1.6 The Machine Learning Workflow . . . 17

1.7 Main Challenges of Machine Learning . . . 18

1.7.1 Problems with the Data . . . 18

1.7.2 Problems with the Algorithm . . . 19

1.7.3 Testing and Validating . . . 22

1.7.4 No Free Lunch Theorem . . . 22

Exam Corner . . . 23

2 Statistical Learning **27**

2.1 Introduction to Statistical Learning . . . 27

2.1.1 The Statistical Model . . . 28

2.1.2 Why Estimate f? Prediction and Inference . . . 28

2.1.3 Parametric and Non-parametric Methods . . . 29

2.2 Supervised and Unsupervised Learning: The Statistical View . . . 30
2.2.1 Supervised Learning . . . 30
2.2.2 Unsupervised Learning . . . 30
2.3 Loss Functions, Training Loss and Test Loss . . . 31
2.3.1 Loss Functions . . . 31
2.3.2 Risk, Training Loss and Test Loss . . . 32
2.3.3 Splitting the Data . . . 34
2.4 Trade-offs in Statistical Learning . . . 34
2.4.1 The Bias–Variance Trade-off . . . 35
2.4.2 Other Trade-offs . . . 37
2.5 Estimating Risk Statistics . . . 38
2.5.1 Hold-out (Validation-set) Estimate . . . 38
2.5.2 k-Fold Cross-Validation . . . 39
2.5.3 Risk Statistics for Classification . . . 41
2.6 Sampling Distribution of an Estimator . . . 42
2.6.1 Properties of Good Estimators . . . 43
2.6.2 The Sample Mean . . . 43
2.6.3 Central Limit Theorem . . . 45
2.6.4 A Biased Estimator: the Variance . . . 45
Exam Corner . . . 47

3 Distance-Based Methods, Decision Trees and Naive Bayes 51

3.1 Distance-Based Methods . . . 52
3.1.1 Common Distance Measures . . . 52
3.1.2 Feature Scaling . . . 54
3.2 Nearest Neighbours (k-NN) . . . 55
3.2.1 The Algorithm . . . 55
3.2.2 Choosing k . . . 57
3.2.3 Weighted k-NN and k-NN Regression . . . 57
3.2.4 Strengths and Weaknesses . . . 58
3.3 Decision Trees . . . 58
3.3.1 Idea and Structure . . . 58
3.3.2 Measures of Impurity . . . 59

3.3.3 Worked Example: Building a Tree with ID3 . . . 60
3.3.4 Algorithm Summary . . . 62
3.3.5 Overfitting and Pruning . . . 63
3.3.6 Strengths and Weaknesses . . . 64
3.4 Naive Bayes . . . 64
3.4.1 Bayes' Theorem . . . 64
3.4.2 The Naive Bayes Classifier . . . 65
3.4.3 Worked Example: Play-Tennis Again . . . 66
3.4.4 Laplace Smoothing . . . 67
3.4.5 Variants of Naive Bayes . . . 68
3.5 Putting It Together: The Three Methods in Python . . . 69
3.6 Comparison of the Three Methods . . . 70
Exam Corner . . . 71

4 Linear Models and Support Vector Machines 76

4.1 Linear Regression . . . 77
4.1.1 Simple Linear Regression . . . 78
4.1.2 Normal Equation (Matrix Form) . . . 80
4.1.3 Gradient Descent . . . 81
4.1.4 Multiple Linear Regression . . . 82
4.1.5 Evaluating a Regression Model . . . 83
4.1.6 Regularised Linear Regression . . . 84
4.2 Logistic Regression . . . 84
4.2.1 Why Not Use Linear Regression for Classes? . . . 86
4.2.2 Odds and Log-odds . . . 86
4.2.3 Loss Function: Cross-entropy from Maximum Likelihood . . . 87
4.2.4 Real-life Example: Will the Student Pass? . . . 87
4.2.5 Training by Gradient Descent: One Step by Hand . . . 89
4.2.6 Multiclass Logistic Regression . . . 89
4.2.7 Choosing the Threshold and Regularisation . . . 90
4.3 Generalized Linear Models (GLMs) . . . 91
4.3.1 Real-life Example: Poisson Regression for Emergency Calls . . . 92
4.4 Binary Classification and Support Vector Machines . . . 93

4.4.1 Binary Classification . . . 93
4.4.2 The Support Vector Machine (Linear, Hard Margin) . . . 93
4.4.3 Soft-Margin SVM . . . 96
4.5 Choosing Among the Methods of Unit II . . . 98
Exam Corner . . . 99

5 Ensemble Learning 104
5.1 Introduction to Ensemble Learning . . . 104
5.1.1 The Idea: Wisdom of the Crowd . . . 105
5.1.2 Why Does It Work? A Simple Calculation . . . 105
5.1.3 Types of Ensemble Methods . . . 106
5.2 Voting Classifiers . . . 108
5.2.1 Hard vs. Soft Voting: Worked Example . . . 109
5.3 Bagging and Pasting . . . 110
5.3.1 The Bootstrap . . . 110
5.3.2 Algorithm and Aggregation . . . 111
5.3.3 Out-of-Bag (OOB) Evaluation . . . 112
5.4 Random Forests . . . 113
5.4.1 Feature Importance . . . 114
5.5 Boosting . . . 116
5.5.1 AdaBoost (Adaptive Boosting) . . . 117
5.5.2 Gradient Boosting . . . 120
5.6 Stacking . . . 122
5.7 Comparison and Practical Advice . . . 124
Exam Corner . . . 126

6 Support Vector Machines and Naive Bayes Classifiers 132
6.1 Linear SVM Classification . . . 133
6.1.1 The Idea: Build the Widest Road . . . 133
6.1.2 Definitions . . . 134
6.1.3 Soft-Margin SVM: Strict and Relaxed Teachers . . . 134
6.1.4 The Dual Form and Support Vectors . . . 136
6.1.5 Practical Points . . . 136

6.2 Nonlinear SVM Classification . . . 137
6.2.1 When a Straight Line Is Not Enough . . . 137
6.2.2 The Kernel Trick . . . 139
6.2.3 How an RBF SVM Decides: Hills of Influence . . . 140
6.2.4 Choosing C and γ: Real Results . . . 141
6.3 SVM Regression . . . 143
6.3.1 The Idea: Points Inside the Tunnel Are Free . . . 143
6.3.2 Real Results . . . 145
6.4 Naive Bayes Classifiers . . . 146
6.4.1 The Detective Who Multiplies Clues . . . 147
6.4.2 Gaussian Naive Bayes (continuous features) . . . 147
6.4.3 Multinomial Naive Bayes (word counts) . . . 149
6.4.4 Bernoulli Naive Bayes (word present or absent) . . . 150
6.4.5 Generative vs. Discriminative Classifiers . . . 152
6.5 Comparing the Methods of Unit III . . . 153
Exam Corner . . . 153

Chapter 1

Introduction to Machine Learning

Learning Objectives

After studying this chapter you will be able to:

- explain the relationship between **AI, Machine Learning and Deep Learning**;
- state and apply Samuel's and Mitchell's definitions of Machine Learning;
- use the vocabulary of ML correctly: *instance, feature, label, training set, model, parameter, hyperparameter*;
- classify ML systems as **supervised, unsupervised, semi-supervised** or **reinforcement**;
- distinguish **batch vs. online** and **instance-based vs. model-based** learning;
- fit a straight-line model by hand using the least-squares formula;
- describe the main **challenges** of ML (data problems, *overfitting*, *underfitting*) and the remedies for each;
- answer 2-, 5- and 10-mark university questions on this unit.

> *Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.*
>
> — Arthur Samuel, 1959

1.1 Why Do We Need Machine Learning?

Suppose you must write a program that detects *spam e-mail*. The traditional approach is to write rules by hand:

- If the subject contains "FREE" ⇒ spam.
- If the sender is unknown and the body contains "winner" ⇒ spam.

- . . . and hundreds more rules.

This approach has three serious weaknesses.

1. **Long, hard-to-maintain rule lists.** Every new spam trick needs a new rule, and rules start to contradict each other.
2. **No adaptation.** Spammers change "FREE" to "FR33" or "fr.ee" and the rules stop working.
3. **Problems too complex for rules.** No one can write explicit rules to recognise a spoken word or a face in a photo.

A machine-learning program instead studies thousands of e-mails already labelled *spam* or *not spam* and discovers the pattern by itself. When spammers change tactics, we simply retrain it on fresh data.

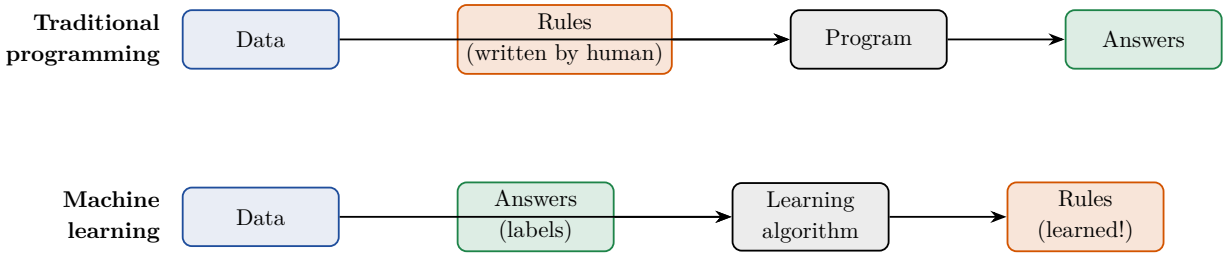


Figure 1.1. Traditional programming versus machine learning.

Reading the Figure

Top row (traditional programming). A programmer writes the *rules*. The program applies those rules to the *data* and produces *answers*.
Bottom row (machine learning). The inputs are swapped. We give the algorithm the *data* together with the correct *answers*, and what comes out is the *rules*. These learned rules can then be applied to new data.
Note the colours: orange marks the *rules*. In the top row a human produces them; in the bottom row the algorithm does. This swap of inputs and outputs is the whole idea of ML.

Key Idea

Traditional programming: *data* + *rules* → *answers*. Machine learning: *data* + *answers* → *rules*.

1.2 Artificial Intelligence, Machine Learning and Deep Learning

Definition: Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science concerned with building machines that perform tasks that normally require human intelligence, such as reasoning, understanding language, recognising images and making decisions.

Early AI (1950s–1980s) was dominated by **symbolic AI** or *expert systems*, in which experts typed in thousands of "if–then" rules. These systems were useful in narrow areas such as medical diagnosis but broke down on messy real-world data. Since the 1990s the focus has shifted to systems that *learn* from data.

Definition: Machine Learning (Samuel, 1959)

Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Definition: Machine Learning (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Samuel's definition is informal and historical (he wrote a checkers program that improved by playing against itself). Mitchell's definition is more engineering-oriented because it tells us *what to measure*: to claim that a program is learning we must name the task, the metric and the experience.

Definition: Deep Learning

Deep Learning is a subfield of ML that uses *artificial neural networks with many layers* to learn increasingly abstract representations of data (e.g. pixels → edges → shapes → faces). It is the subject of Unit V.

Reading the Figure

The figure is a **set diagram**: each ellipse lies completely inside the one before it.

- *Outer ellipse (AI):* the biggest set. It includes ML but also non-learning AI such as hand-coded chess engines and rule-based expert systems.
- *Middle ellipse (ML):* every ML system is AI, but not every AI system learns.

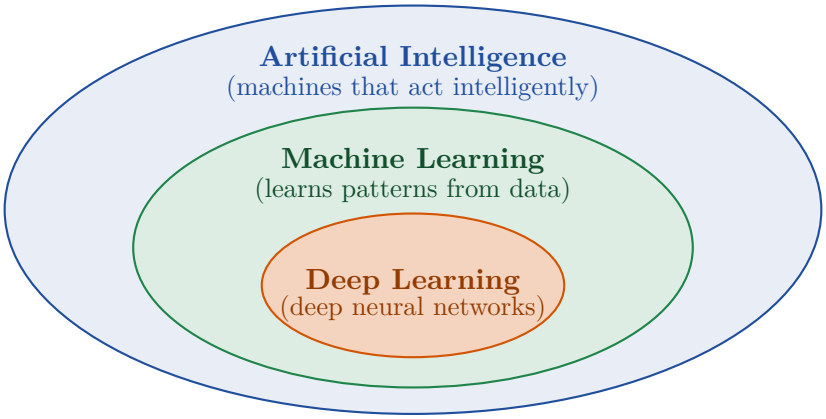


Figure 1.2. AI, ML and Deep Learning as nested sets.

- *Inner ellipse (DL):* deep learning is one family of ML methods. Decision trees, k-NN and SVMs are ML but *not* deep learning.

Remember the order DL $\subset$ ML $\subset$ AI.

Example: Spam filter in (T, P, E) form

Task T	Flag new e-mails as spam or not spam
Performance P	Percentage of e-mails classified correctly (accuracy)
Experience E	A collection of e-mails labelled by users

Check: if accuracy rises from 90% to 96% as we add more labelled mail, the program is learning. If accuracy does not change however much data we add, it is *not* learning, whatever its marketing says.

Example: Which of these is ML?

(a) A program that fixes a date's format using a fixed rule. **Not ML:** no experience is used.

(b) A music app that suggests songs based on what you skipped. **ML:** behaviour (experience) improves recommendations.

(c) A calculator. **Not ML:** its performance never improves with experience.

Exam Tip

"Define Machine Learning" appears almost every year. For full marks write *both* Samuel's and Mitchell's definitions, then give the spam-filter example in (T, P, E) form.

Write-up for the Exam: Machine Learning

Definition

Machine Learning (ML) is a branch of Artificial Intelligence in which a computer program *learns patterns from data* and improves its performance on a task with experience, without being explicitly programmed with rules. (Samuel, 1959; Mitchell, 1997: performance P at task T improves with experience E.)

Working

1. Collect data relevant to the task.
2. Clean and prepare the data (missing values, encoding, scaling).
3. Choose a model and a loss function.
4. Train: fit the parameters by minimising the loss on the training set.
5. Evaluate on unseen test data using a performance measure.
6. Deploy the model and monitor it; retrain when performance drops.

Advantages

- Solves problems too complex for hand-written rules
- Adapts when data changes
- Discovers hidden patterns in large datasets
- Improves as more data becomes available

Disadvantages

- Needs large amounts of good-quality data
- May overfit or inherit bias from the data
- Many models are hard to interpret (black boxes)
- Training can be computationally expensive

Applications

- Spam filtering and recommendation systems
- Medical diagnosis and fraud detection
- Speech, image and language recognition
- Self-driving vehicles

1.3 Where Is Machine Learning Used?

ML is the right tool when (i) rules are too many or too complex, (ii) the environment keeps changing, (iii) the problem has no known algorithmic solution, or (iv) large amounts of data are available to learn from.

Area	Application	ML task
E-mail	Spam filtering	Classification
E-commerce	"Customers also bought"	Recommendation
Health	Detecting disease from X-rays	Image classification
Transport	Self-driving cars	Perception and control
Language	Translation, chatbots	Sequence modelling
Finance	Credit-card fraud detection	Anomaly detection
Real estate	Predicting house prices	Regression
Marketing	Grouping similar customers	Clustering

1.4 Basic Terminology

Before going further, learn this vocabulary. It is used in every chapter.

Term	Meaning
Instance (sample)	One data point, e.g. one e-mail or one house.
Feature (attribute)	An input measurement describing an instance, e.g. area, number of rooms.
Label (target)	The correct output for an instance, e.g. price or "spam".
Training set	Data used to *fit* the model.
Test set	Data held back to *evaluate* the final model on unseen instances.
Model	The function f learned from data, $\hat{y} = f(\boldsymbol{x})$.
Parameters	Numbers learned *by the algorithm* during training (e.g. θ_0, θ_1).
Hyperparameters	Settings chosen *by the human* before training (e.g. k in k-NN, learning rate).

Example: A small dataset

	Area (m^2)	Rooms	Age (yr)	Price (lakh Rs.)
Instance 1	80	2	10	45
Instance 2	120	3	5	78
Instance 3	60	1	20	28

Here *Area, Rooms, Age* are the three **features**; the feature vector of instance 1 is $\boldsymbol{x}_1 = (80, 2, 10)$, and its **label** is $y_1 = 45$. The first three columns together form the input X, and

the last column is the target y.

Common Mistake

Do not mix up **parameters** and **hyperparameters**. Parameters are *learned from data*; hyperparameters are *set by you* and are tuned by trying several values and comparing performance on held-out data.

1.5 Types of Machine Learning Systems

ML systems are classified in three independent ways. The same system gets one label from each way, e.g. "a supervised, batch, model-based system".

Q1. How much supervision does it get during training? (supervised, unsupervised, semi-supervised, reinforcement)

Q2. Can it learn incrementally on the fly? (batch vs. online)

Q3. Does it compare new data to stored examples or build a model? (instance-based vs. model-based)

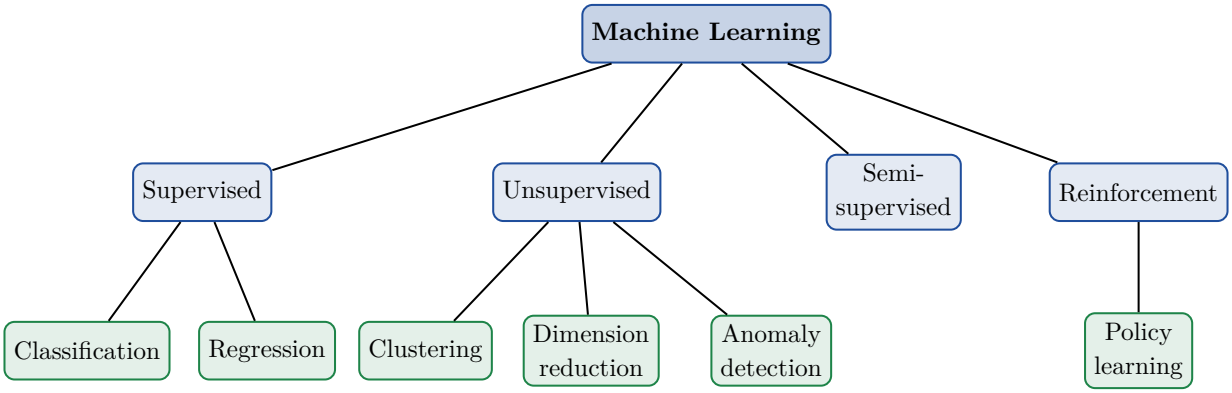


Figure 1.3. Taxonomy of ML systems by the type of supervision. Blue = learning paradigm, green = typical task.

Reading the Figure

Read this **tree from top to bottom**. The root is Machine Learning. The second level (blue) shows the four paradigms; the third level (green) shows the typical *tasks* that each paradigm solves. For example, *classification* and *regression* both need labelled data and therefore sit under *supervised*; *clustering* needs no labels and so sits under *unsupervised*. Semi-supervised learning is a mix of the first two and is usually applied to the same tasks.

1.5.1 Supervised Learning

In supervised learning the training data includes the **correct answer** (the label). The learner is like a student who checks each practice answer against an answer key.

Definition: Supervised Learning

Given a training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ where $\boldsymbol{x}_i \in \mathbb{R}^d$ is the feature vector and y_i is the label, learn a function f such that $f(\boldsymbol{x}) \approx y$ *for new, unseen inputs.*

Types of supervised problems.

- **Classification:** y is a *category.*
  - *Binary:* two classes (spam / not spam).
  - *Multiclass:* one of many classes (digit 0–9).
  - *Multilabel:* several labels at once (a photo tagged "beach" and "sunset").
- **Regression:** y is a *real number* (house price, temperature). If there is one feature it is *simple* regression; if more, *multiple* regression.



Figure 1.4. (a) Classification separates two classes with a decision boundary. (b) Regression fits a line that predicts a number.

Reading the Figure

Panel (a): each point is one instance, plotted by two features x_1, x_2. Blue circles belong to class A and orange triangles to class B. The dashed green line is the *decision boundary* the classifier has learned: points on one side are predicted A, points on the other side B. The *output is a category.*
Panel (b): each dot is a house. The horizontal axis is the feature (size) and the vertical axis is the target (price). The green line is the fitted model; to predict the price of a new house, find its size on the horizontal axis and read the height of the line. The *output is a number.*

Write-up for the Exam: Supervised Learning

Definition

Supervised learning learns a function f from *labelled* training examples $(\boldsymbol{x}_i, y_i)$ so that $f(\boldsymbol{x}) \approx y$ for new, unseen inputs. The labels act as a teacher.

Working

1. Collect a labelled dataset and split it into training and test sets.
2. Choose a model (e.g. linear regression, decision tree) and a loss function.
3. Fit the parameters by minimising the training loss.
4. Tune hyperparameters on a validation set.
5. Report final performance on the test set.

Types

- **Classification**: the label is a category (spam / not spam)
- **Regression**: the label is a number (house price)

Advantages

- Clear goal; accuracy can be measured directly
- Many mature and well-understood algorithms

Disadvantages

- Labelled data is expensive and slow to obtain
- Poor labels (noise) lead to poor models
- Does not generalise beyond the training distribution

Applications

- Spam detection and credit scoring
- House-price and sales prediction
- Disease diagnosis from tests or images

1.5.2 Unsupervised Learning

The training data is **unlabelled**. The system must discover structure on its own, like sorting a heap of unfamiliar coins into groups by size and colour without anyone naming them.

Definition: Unsupervised Learning

Given only inputs $\{\boldsymbol{x}_i\}_{i=1}^{n}$ with no labels, discover hidden structure, such as groups, directions of variation or unusual points, in the data.

- **Clustering:** group similar instances, e.g. segment customers by buying behaviour (K-Means,

DBSCAN; Chapter 7).

- **Dimensionality reduction:** compress many features into a few while keeping the information (PCA; Chapter 8). Useful for visualisation and for speeding up learning.
- **Anomaly (outlier) detection:** flag unusual instances, e.g. a suspicious credit-card transaction.
- **Association rule learning:** find relations such as "people who buy bread also buy butter".

Write-up for the Exam: Unsupervised Learning

Definition

Unsupervised learning discovers hidden structure (groups, patterns, directions of variation) in *unlabelled* data $\{\boldsymbol{x}_i\}$; there is no target variable.

Working

1. Collect unlabelled data and scale the features.
2. Choose a task and algorithm (clustering, dimensionality reduction, anomaly detection).
3. Optimise a criterion such as within-cluster distance or retained variance.
4. Interpret the discovered structure using domain knowledge.

Types

- Clustering (K-Means, DBSCAN)
- Dimensionality reduction (PCA)
- Anomaly detection and association rules

Advantages

- No labels needed; works on cheap raw data
- Reveals patterns humans did not expect
- Useful as preprocessing for supervised learning

Disadvantages

- No ground truth, so results are hard to evaluate
- Results can be subjective and depend on parameters
- Sensitive to feature scaling and noise

Applications

- Customer segmentation
- Fraud and fault detection
- Topic discovery in documents
- Data compression and visualisation

1.5.3 Semi-supervised Learning

Labelling is slow and costly (a doctor must label every X-ray), but unlabelled data is cheap. **Semi-supervised** algorithms use a *small labelled* set and a *large unlabelled* set. A typical method is **self-training**: train on the labelled data, predict labels for the unlabelled data, keep the most confident predictions as new labels, and retrain.

Example. Photo apps first cluster faces automatically (unsupervised), then ask you to name each person once (supervised); the name then spreads to every photo in the cluster.

Write-up for the Exam: Semi-supervised Learning

Definition

Semi-supervised learning trains a model using a *small amount of labelled data together with a large amount of unlabelled data.*

Working (self-training)

1. Train an initial model on the labelled examples.
2. Predict labels for the unlabelled examples.
3. Keep the most confident predictions as new (pseudo) labels.
4. Retrain on the enlarged labelled set and repeat.

Advantages

- Greatly reduces the cost of labelling
- Usually more accurate than using the few labels alone

Disadvantages

- Early wrong pseudo-labels can reinforce themselves
- Assumes labelled and unlabelled data come from the same distribution

Applications

- Photo tagging and face grouping
- Speech recognition
- Medical image analysis

1.5.4 Reinforcement Learning

There are no labelled examples. An **agent** observes the **state** of an **environment**, chooses an **action**, and receives a **reward** (positive) or **penalty** (negative). It learns a **policy**, a rule that says which action to take in each state, that maximises the total reward over time.

Term	Example: robot learning to walk
Agent	The robot's control program
Environment	The floor, gravity, the robot's body
State s	Joint angles, speed, whether upright
Action a	Move a joint motor
Reward r	$+1$ for moving forward, -10 for falling
Policy π	The learned rule: state $\mapsto$ action

Because rewards may arrive late (a chess move is only rewarded when the game ends), the agent maximises the **discounted return**

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k\, r_{t+k+1}, \qquad 0 \le \gamma \le 1,$$

where the *discount factor* γ says how much we value future rewards. With $\gamma = 0$ the agent is short-sighted (only the next reward matters); with γ close to 1 it is far-sighted. Famous examples are DeepMind's AlphaGo, which beat the world Go champion, and robots that learn to walk.

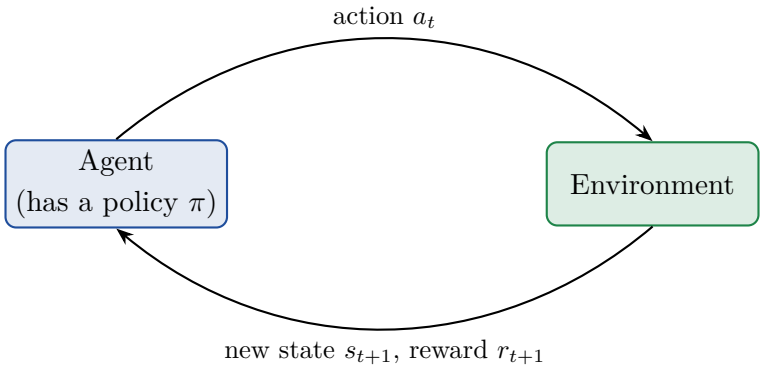


Figure 1.5. The reinforcement-learning loop.

Reading the Figure

The picture shows a **closed loop** that repeats at every time step t. (1) The agent looks at the current state and uses its policy to pick an action a_t (top arrow). (2) The environment reacts and sends back the next state s_{t+1} and a reward r_{t+1} (bottom arrow). (3) The agent uses that feedback to improve its policy, and the loop repeats. Learning happens by *trial and error*: actions that lead to high reward become more likely.

Type	Labels?	Feedback	Typical task	Example
Supervised	Yes	Correct answer	Classification, regression	Spam, prices
Unsupervised	No	None	Clustering, PCA	Customer groups
Semi-supervised	Few	Partial	Mixed	Photo tagging
Reinforcement	No	Reward/penalty	Sequential decisions	Games, robots

Write-up for the Exam: Reinforcement Learning

Definition

In **reinforcement learning (RL)** an *agent* interacts with an *environment*: it observes a state, takes an action, and receives a reward or penalty. It learns a *policy* that maximises the total discounted reward $G_t = \sum_k \gamma^k r_{t+k+1}$.

Working

1. The agent observes the current state s_t.
2. It chooses an action a_t using its policy (balancing *exploration* and *exploitation*).
3. The environment returns the next state s_{t+1} and a reward r_{t+1}.
4. The agent updates its policy to increase future reward; the loop repeats.

Advantages

- Needs no labelled data
- Can learn long-term strategies for sequential decisions
- Adapts to a changing environment

Disadvantages

- Needs very many trials (sample inefficient)
- Designing a good reward function is difficult
- Exploration can be unsafe or costly

Applications

- Game playing (AlphaGo, chess)
- Robotics and self-driving control
- Resource scheduling and recommendation

1.5.5 Batch vs. Online Learning

Batch (offline) learning. The system is trained once on *all* available data, then deployed and no longer learns. To include new data it must be retrained from scratch on old + new data. This is simple but can take hours and a lot of memory.

Online (incremental) learning. The system is trained continuously on single instances or small *mini-batches* as they arrive. Each step is cheap, so it suits streaming data (stock prices) and limited memory; it can even handle datasets too large for memory (*out-of-core learning*).

The key hyperparameter of online learning is the **learning rate**: a *high* rate adapts quickly but forgets old data fast and is sensitive to noise; a *low* rate is stable but slow to adapt. A risk is that *bad data* (a faulty sensor) can quickly degrade a live system, so its performance must be monitored.

Write-up for the Exam: Batch (Offline) Learning

Definition

Batch learning trains the model once on *all* the available data, then deploys it unchanged; it does not learn after deployment.

Working

1. Gather the complete dataset and train the model offline.
2. Deploy the trained model.
3. To include new data, retrain from scratch on old + new data.

Advantages

- Simple, stable and reproducible
- Any algorithm can be used

Disadvantages

- Cannot learn incrementally; retraining is slow and memory-hungry
- The model becomes outdated as data changes (concept drift)
- Unsuitable for huge or streaming datasets

Applications

- Credit-scoring models retrained monthly
- Offline image classifiers

Write-up for the Exam: Online (Incremental) Learning

Definition

Online learning updates the model continuously, one instance or small *mini-batch* at a time, as new data arrives.

Working

1. Receive a new instance or mini-batch.
2. Update the parameters with a small step controlled by the **learning rate**.
3. Discard the data (saving memory) and repeat.

Advantages

- Fast, cheap updates; low memory use
- Adapts quickly to changing data
- Handles very large datasets (out-of-core learning) and streams

Disadvantages

- Sensitive to the choice of learning rate

- Bad data can quickly degrade the live model
- Needs continuous monitoring

Applications

- Stock-price prediction
- Daily-updated spam filters
- Sensor and news-feed streams

1.5.6 Instance-based vs. Model-based Learning

Instance-based learning. The system *memorises* the training examples and, for a new point, finds the most *similar* stored ones and copies their answer. *Example:* k-Nearest Neighbours (Chapter 3). No explicit model is built.

Model-based learning. The system builds a *mathematical model* from the examples, then uses only the model (the training data may be thrown away) to predict. *Example:* linear regression.

	Instance-based	**Model-based**
Stores	All training data	Only parameters
Prediction by	Similarity to stored data	Evaluating the learned formula
Training cost	Low (just store)	Higher (fit parameters)
Prediction cost	High (compare to many points)	Low
Example	k-NN	Linear regression

Write-up for the Exam: Instance-based Learning

Definition

Instance-based (memory-based) learning stores the training examples and predicts for a new point by *comparing it with the stored examples* using a similarity measure. It is a *lazy learner*: it builds no explicit model.

Working

1. Store all training instances.
2. For a new point, measure its distance to every stored instance.
3. Predict from the most similar instances (e.g. majority class of the k nearest).

Advantages

- No training phase
- Flexible; can fit complex local patterns
- Immediately uses new data

Disadvantages

- Slow prediction and large memory use
- Sensitive to feature scaling and irrelevant features

Applications

- k-Nearest Neighbours
- Recommender systems and pattern recognition

Write-up for the Exam: Model-based Learning

Definition

Model-based learning builds a mathematical model with parameters learned from the training data and then predicts using *only the model*.

Working

1. Choose a model form, e.g. $\hat{y} = \theta_0 + \theta_1 x$.
2. Define a loss and fit the parameters to minimise it.
3. Predict by evaluating the fitted model; the training data may be discarded.

Advantages

- Fast prediction; compact model
- Often interpretable

Disadvantages

- A wrong model form causes high bias
- Training can be costly

Applications

- Linear and logistic regression
- SVMs and neural networks

Worked example: a model-based learner by hand

Suppose we believe that house price y depends linearly on size x:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Learning means choosing θ_0, θ_1 so that predictions are close to the data. For simple linear regression the least-squares solution is

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}.$$

Example: Fit a line by hand

Data: size x (100 m^2) and price y (lakh Rs.).

x	1	2	3	4	5	$\bar{x} = 3$
y	15	26	29	44	46	$\bar{y} = 32$
$x - \bar{x}$	-2	-1	0	1	2	
$y - \bar{y}$	-17	-6	-3	12	14	
$(x-\bar{x})(y-\bar{y})$	34	6	0	12	28	$\Sigma = 80$
$(x-\bar{x})^2$	4	1	0	1	4	$\Sigma = 10$

Step 1: $\theta_1 = 80/10 = 8$. **Step 2:** $\theta_0 = 32 - 8 \times 3 = 8$.
Model: $\hat{y} = 8 + 8x$. For a new house of size $x = 6$: $\hat{y} = 8 + 8 \times 6 = 56$ lakh Rs..

Try It Yourself

Check the hand calculation with `scikit-learn`. The output should be intercept 8, coefficient 8 and prediction 56.

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4], [5]]       # feature: size (100 m^2)
y = [15, 26, 29, 44, 46]            # label: price (lakh Rs)

model = LinearRegression()          # 1. choose the model
model.fit(X, y)                     # 2. learn theta_0 and theta_1
print(model.intercept_, model.coef_[0])   # -> 8.0  8.0
print(model.predict([[6]]))                # -> [56.]
```

Challenge: add the point (6, 58) to the data. Do the parameters change? Why?

1.6 The Machine Learning Workflow

A real ML project is more than choosing an algorithm. It follows a cycle of steps.

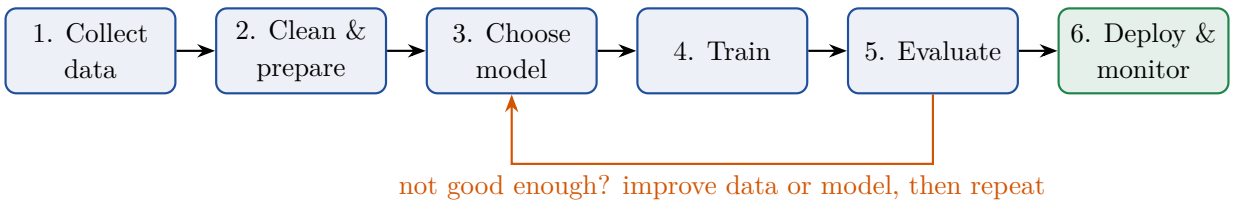


Figure 1.6. The machine-learning workflow.

Reading the Figure

Follow the **blue boxes left to right**; they are the steps in order. The **orange arrow** is the most important part: it shows that evaluation usually sends you *back* to try a different model or better data. ML is therefore iterative.

1. **Collect data:** from databases, sensors, web, surveys.
2. **Clean and prepare:** handle missing values, remove duplicates and outliers, encode text categories as numbers, and *scale* features to comparable ranges. This step often takes most of the project time.
3. **Choose a model** that suits the task (classifier, regressor, clusterer).
4. **Train:** the algorithm finds the parameters that fit the training set.
5. **Evaluate** on data *not* used for training, using the performance measure P.
6. **Deploy and monitor:** use the model on real inputs and watch for performance decay.

1.7 Main Challenges of Machine Learning

Two things can go wrong: *bad data* and *bad algorithm*.

1.7.1 Problems with the Data

1. **Insufficient training data.** A toddler needs to be shown a ball once or twice; ML systems need thousands or millions of examples. Banko and Brill (2001) showed that, given enough data, even simple algorithms perform almost as well as sophisticated ones: *data matters more than the algorithm.*
2. **Non-representative training data.** The training data must represent the cases the model will meet. If you build a "happiness vs. income" model using only rich countries, it will predict poorly for poor countries. This is **sampling bias**. A famous case is the 1936 *Literary Digest* poll: it surveyed 2.4 million people drawn from telephone and car owners, who were mostly wealthy, and wrongly predicted that Landon would beat Roosevelt. A small *sampling noise* (by chance) arises when the sample is too small; *sampling bias* arises when the method of sampling is flawed, even if the sample is large.
3. **Poor-quality data.** Errors, outliers, noise and missing values hide the patterns. Cleaning (discard outliers, fill or drop missing values) is essential.
4. **Irrelevant features.** "Garbage in, garbage out." **Feature engineering** has three parts: *feature selection* (choose the most useful features), *feature extraction* (combine features, e.g. dimensionality reduction) and *creating new features* by collecting new data.

1.7.2 Problems with the Algorithm

Overfitting. The model performs very well on the training data but badly on new data, because it has *memorised the noise* instead of learning the pattern. It is like a student who memorises last year's paper word for word and then cannot answer a new question. Overfitting happens when the model is *too complex* relative to the amount and noisiness of the data.

Underfitting. The model is too simple to capture the structure of the data, so it does badly even on the training set (like a student who read only the chapter titles).

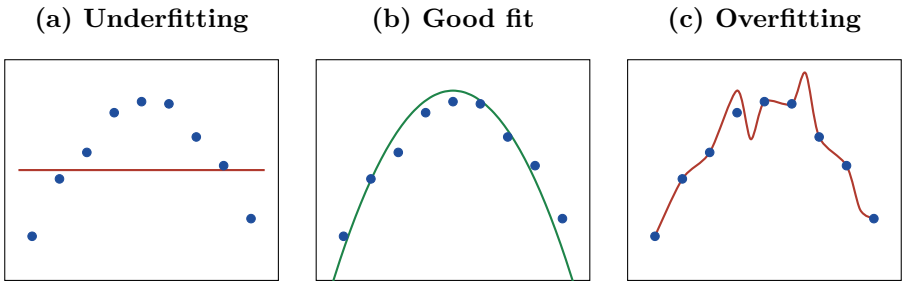


Figure 1.7. The same nine data points fitted by three models of different complexity.

Reading the Figure

All three panels show the *same* blue data points; only the fitted curve (red or green) changes. **(a)** The model is a flat horizontal line. It ignores the rise-and-fall pattern, so errors are large for *every* point: **high training error** ⇒ underfitting. **(b)** A smooth curve passes close to all points without chasing each one: this is the **right complexity** and will generalise well. **(c)** The wiggly curve passes *exactly* through every point, including the random noise. Training error is almost zero, but the zig-zags will mispredict new points: **low training error, high test error** ⇒ overfitting.

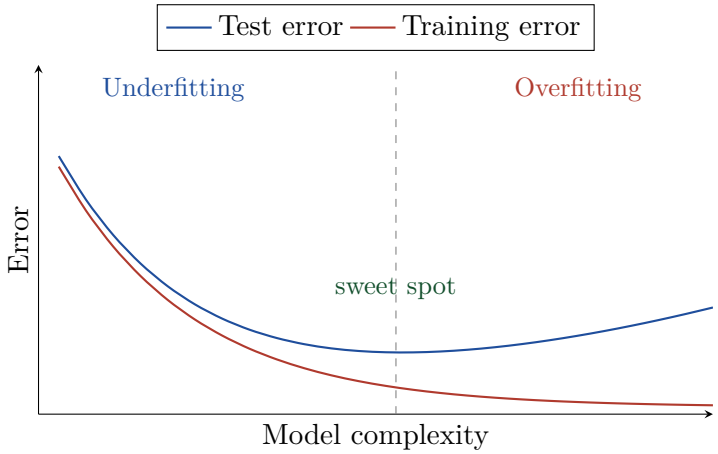


Figure 1.8. Training and test error as model complexity increases.

Reading the Figure

The horizontal axis is how *flexible* the model is (e.g. the degree of a polynomial). The vertical axis is the error.

- **Red curve (training error)** always *falls*: a more flexible model can always fit the training data better.
- **Blue curve (test error)** falls at first and then *rises again*. Beyond the dashed line the model starts fitting noise, so it gets worse on unseen data.
- **Left of the dashed line:** both errors are high ⇒ underfitting. **Right of it:** a growing gap between training and test error ⇒ overfitting.
- **The dashed line** marks the minimum of the test error, the best complexity.

Rule of thumb: judge a model by the *test* curve, never by the training curve.

Common Mistake

Very high accuracy on training data does **not** mean the model is good. Always evaluate on data the model has never seen (Chapter 2).

	Overfitting	**Underfitting**
Model is	too complex	too simple
Training error	very low	high
Test error	high	high
Also called	high variance	high bias
Remedies	more training data; simpler model or fewer features; **regularisation**; remove noise/outliers; early stopping	more complex model; better or more features; reduce regularisation; train longer

Definition: Regularisation

Regularisation means deliberately *constraining* a model to make it simpler and so reduce overfitting, for example by adding a penalty on large parameter values to the loss (Chapter 4). The amount of regularisation is a hyperparameter.

Write-up for the Exam: Overfitting

Definition

Overfitting occurs when a model fits the training data, *including its noise*, so closely that

it performs poorly on new data (low bias, high variance).

Causes

- Model too complex for the amount of data
- Too little or noisy training data
- Too many irrelevant features
- Training for too long

How to detect

Very low training error but much higher validation or test error.

Remedies

- Collect more data
- Use a simpler model or fewer features
- Apply regularisation
- Use cross-validation, early stopping or pruning

Example

A degree-9 polynomial through 10 noisy points has zero training error but swings wildly between the points.

Write-up for the Exam: Underfitting

Definition

Underfitting occurs when a model is too simple to capture the pattern in the data, so it performs poorly on *both* training and test data (high bias).

Causes

- Model too simple (e.g. a straight line for curved data)
- Too few or poorly chosen features
- Too much regularisation
- Insufficient training

How to detect

High training error and high test error, close to each other.

Remedies

- Use a more complex model
- Add or engineer better features
- Reduce regularisation
- Train longer

Example

Fitting a horizontal line to data that clearly rises and then falls.

1.7.3 Testing and Validating

The only reliable way to know how well a model generalises is to try it on new cases. So we split the data:

- **Training set** (e.g. 60–80%): used to fit the parameters.
- **Validation set** (e.g. 10–20%): used to compare models and tune *hyperparameters*.
- **Test set** (e.g. 10–20%): used *once*, at the very end, to estimate the **generalisation error**.

Common Mistake

If you tune hyperparameters on the *test* set, the test set is no longer "unseen": your reported error will be optimistic. Use the validation set (or cross-validation) for tuning and keep the test set locked away.

1.7.4 No Free Lunch Theorem

Wolpert (1996) proved that, averaged over *all possible problems*, no learning algorithm is better than any other. If we make *no assumption* about the data, there is no reason to prefer a linear model over a neural network. In practice we must therefore (i) make reasonable **assumptions** about the data and (ii) *try several models* and compare them on a validation set.

Key Idea

The goal of ML is **generalisation**: performing well on *new* data, not on the data used for training.

Chapter Summary

- DL $\subset$ ML $\subset$ AI. ML learns *rules from data*; Mitchell's definition uses (T, P, E).
- Vocabulary: instance, feature, label, training/test set, model, parameters (learned) vs. hyperparameters (set by us).
- By supervision: supervised (labels), unsupervised (no labels), semi-supervised (few labels), reinforcement (rewards).
- Supervised tasks: classification (category) and regression (number). Unsupervised tasks: clustering, dimensionality reduction, anomaly detection.

- Batch learns offline from all data; online learns incrementally (learning rate matters). Instance-based memorises; model-based fits parameters.
- Challenges: insufficient data, non-representative data (sampling bias), poor quality, irrelevant features, overfitting, underfitting.
- Fix overfitting by more data, simpler models and regularisation; fix underfitting by richer models and better features.
- Split data into training, validation and test sets; never tune on the test set.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define machine learning. **[2 marks]**
2. What is meant by (T, P, E) in Mitchell's definition? **[2 marks]**
3. Differentiate between a feature and a label. **[2 marks]**
4. Differentiate a parameter from a hyperparameter. **[2 marks]**
5. Name two applications each of classification and regression. **[2 marks]**
6. What is semi-supervised learning? Give one example. **[2 marks]**
7. Define overfitting. **[2 marks]**
8. What is the purpose of the test set? **[2 marks]**

Model Answer: Q2 and Q4

Q2. T is the *task* the program must perform, P is the *performance measure* used to judge it, and E is the *experience* (data) it learns from. A program learns if its P on T improves with E.

Q4. A *parameter* (e.g. θ_1 in linear regression) is learned automatically from the training data. A *hyperparameter* (e.g. k in k-NN or the learning rate) is set by the user before training and tuned using a validation set.

B. Short Answer Questions (5 marks each)

1. Differentiate between AI, machine learning and deep learning with a suitable diagram. **[5 marks]**
2. Compare supervised and unsupervised learning with examples. **[5 marks]**
3. Explain batch and online learning. What is the role of the learning rate? **[5 marks]**

4. Differentiate between instance-based and model-based learning with examples. **[5 marks]**
5. Explain reinforcement learning with a diagram. Define agent, state, action, reward and policy. **[5 marks]**
6. Explain the bias in training data (sampling bias) with an example. **[5 marks]**

Model Answer: B2 (outline for 5 marks)

Supervised learning uses *labelled* data $(\boldsymbol{x}, y)$ and learns a mapping $f(\boldsymbol{x}) \approx y$; tasks are classification and regression (e.g. spam detection, house-price prediction); feedback is the correct answer. **Unsupervised learning** uses *unlabelled* data $\boldsymbol{x}$ and finds hidden structure; tasks are clustering, dimensionality reduction, anomaly detection (e.g. customer segmentation); there is no feedback.

Basis	Supervised	Unsupervised
Data	Labelled	Unlabelled
Goal	Predict output	Discover structure
Algorithms	k-NN, SVM, linear regression	K-Means, PCA, DBSCAN
Evaluation	Direct (compare with labels)	Indirect

C. Long Answer Questions (10 marks each)

1. Explain the different types of machine-learning systems based on the amount of supervision, with diagrams and examples. **[10 marks]**
2. Describe the main challenges of machine learning. **[10 marks]**
3. What are overfitting and underfitting? Explain with diagrams, state the causes and the remedies for each. **[10 marks]**
4. Using the data $(1, 15), (2, 26), (3, 29), (4, 44), (5, 46)$, fit a straight line $\hat{y} = \theta_0 + \theta_1 x$ by least squares and predict y for $x = 7$. **[10 marks]**

Model Answer: C4 (numerical)

$\bar{x} = 3,\ \bar{y} = 32.\ \sum(x-\bar{x})(y-\bar{y}) = 34+6+0+12+28 = 80;\ \sum(x-\bar{x})^2 = 4+1+0+1+4 = 10.$
$\theta_1 = 80/10 = 8,\ \ \theta_0 = 32 - 8(3) = 8.$ Model: $\hat{y} = 8 + 8x.$
For $x = 7$: $\hat{y} = 8 + 8(7) = \mathbf{64}$.

D. Multiple-Choice Questions

1. Which of the following is *not* a type of ML system by supervision?
   (a) Supervised (b) Unsupervised (c) Reinforcement (d) **Compiled**

2. Grouping customers by buying behaviour without predefined labels is:
   (a) classification (b) regression (c) clustering (d) reinforcement

3. A model with very low training error but high test error is:
   (a) underfitting (b) overfitting (c) well fitted (d) unbiased

4. In Mitchell's definition, accuracy of a spam filter is an example of:
   (a) task (b) experience (c) performance measure (d) model

5. k-Nearest Neighbours is an example of:
   (a) model-based learning (b) instance-based learning (c) online learning (d) RL

6. The discount factor γ is used in:
   (a) clustering (b) regression (c) reinforcement learning (d) PCA

7. Hyperparameters are tuned using the:
   (a) training set (b) validation set (c) test set (d) none

Answers: 1-d, 2-c, 3-b, 4-c, 5-b, 6-c, 7-b.

E. Practice Problems

1. Label each as classification, regression or clustering: (a) predicting tomorrow's temperature, (b) deciding whether a tumour is benign, (c) grouping news articles by topic.

2. Write the (T, P, E) description for a handwriting-recognition program.

3. A robot has $\gamma = 0.9$ and receives rewards 1, 0, 5 at the next three steps (and none afterwards). Compute G_t. *(Answer:* $1 + 0.9 \cdot 0 + 0.81 \cdot 5 = 5.05$*.)*

4. **Coding:** extend the `LinearRegression` program to print predictions for sizes 6, 7 and 8. Plot the data and the fitted line with `matplotlib`.

Quick Revision

Topic	One-line recall
Samuel (1959)	Learning without being explicitly programmed.
Mitchell (1997)	Performance P at task T improves with experience E.
Supervised	Labelled data; classification (category) / regression (number).
Unsupervised	No labels; clustering, PCA, anomaly detection.
Semi-supervised	Few labels + many unlabelled (self-training).
Reinforcement	Agent–environment loop; maximise $G_t = \sum \gamma^k r_{t+k+1}$.
Batch / Online	All data once / incremental; learning rate.
Instance / Model-based	Memorise and compare / fit parameters.
Overfitting	Low train error, high test error $\rightarrow$ more data, regularise, simplify.
Underfitting	High train and test error $\rightarrow$ richer model, better features.
Data splits	Train: fit. Validation: tune. Test: final estimate.
No Free Lunch	No best algorithm for all problems; make assumptions and compare.

Chapter 2

Statistical Learning

Learning Objectives

After studying this chapter you will be able to:

- describe the statistical model $Y = f(X) + \varepsilon$ and explain *prediction* versus *inference*;
- state the statistical view of **supervised** and **unsupervised** learning;
- define and compute common **loss functions** (squared, absolute, 0–1);
- distinguish **training loss**, **test loss** and **risk**, and explain why training loss is optimistic;
- derive and interpret the **bias–variance decomposition** and describe the main **trade-offs** of statistical learning;
- estimate risk using the **hold-out**, **k-fold** and **leave-one-out** methods and compute classification statistics (accuracy, precision, recall, F1);
- explain the **sampling distribution of an estimator**, its bias, variance and standard error, and use the Central Limit Theorem;
- answer 2-, 5- and 10-mark university questions on Unit I.

> *All models are wrong, but some are useful.*
>
> — George Box

2.1 Introduction to Statistical Learning

Chapter 1 told us *what* machine learning is. **Statistical learning** is the mathematical framework that tells us *why it works*, *how well* it works, and *how to measure that*. It treats data as random samples drawn from some unknown real-world process and asks: how can we learn that process from a finite sample?

2.1.1 The Statistical Model

Suppose we observe p input variables $X = (X_1, \ldots, X_p)$ (the *features*) and one output variable Y (the *response*). We assume that they are related by

$$Y = f(X) + \varepsilon. \tag{2.1}$$

- f is an **unknown fixed function**: the systematic, predictable part of the relationship.
- ε is a **random error** (noise) with mean zero, $\mathbb{E}[\varepsilon] = 0$, independent of X and with variance σ^2. It stands for everything we did not measure or that is genuinely random.

Statistical learning is therefore *the set of methods for estimating* f from data. Our estimate is written $\hat{f}$.

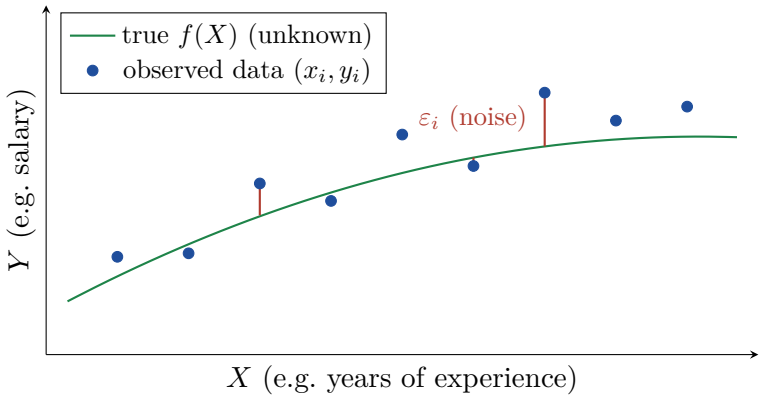


Figure 2.1. Data are generated as $Y = f(X) + \varepsilon$. The curve is the true but unknown f; dots are noisy observations.

Reading the Figure

The **green curve** is the true relationship $f(X)$, which in real life we never see. The **blue dots** are what we actually collect: each dot lies slightly above or below the curve because of random noise. The **short red vertical segments** are examples of the noise term $\varepsilon_i = y_i - f(x_i)$. Our job is to look only at the dots and recover a curve $\hat{f}$ as close to the green one as possible. Because of the noise, we can *never* recover f perfectly.

2.1.2 Why Estimate f? Prediction and Inference

Prediction. We only care about getting accurate predictions $\hat{Y} = \hat{f}(X)$ for new X. $\hat{f}$ can be a "black box". *Example:* forecasting tomorrow's sales.

Inference. We want to *understand* how Y changes with each X_j: which features matter, whether the effect is positive or negative, and how strong it is. $\hat{f}$ must be interpretable. *Example:* how much does one extra year of education raise salary?

Reducible and irreducible error. For a fixed $\hat{f}$ and X,

$$\mathbb{E}\big[(Y-\hat{Y})^2\big] = \underbrace{\big[f(X)-\hat{f}(X)\big]^2}_{\text{reducible}} + \underbrace{\operatorname{Var}(\varepsilon)}_{\text{irreducible}=\sigma^2} . \tag{2.2}$$

Better methods shrink the reducible part by making $\hat{f}$ closer to f, but *no* method can remove the irreducible part σ^2. This is the lower limit on any model's error.

2.1.3 Parametric and Non-parametric Methods

	Parametric	**Non-parametric**
Idea	Assume a form for f, e.g. $f(X) = \theta_0 + \theta_1 X$, then estimate a few parameters	Make no strong assumption about f; let the data decide its shape
Advantage	Simple, needs little data, easy to interpret	Flexible; can fit any shape
Disadvantage	Wrong assumption $\Rightarrow$ poor fit	Needs a lot of data; risk of over-fitting
Examples	Linear / logistic regression, Naive Bayes	k-NN, decision trees

Write-up for the Exam: Statistical Learning

Definition

Statistical learning is a set of tools for *estimating the unknown relationship* $Y = f(X) + \varepsilon$ between inputs X and a response Y from data, either to **predict** Y for new X or to **infer** how Y depends on each X_j.

Working

1. Assume a model family for f (parametric or non-parametric).
2. Choose a loss function that measures prediction error.
3. Estimate $\hat{f}$ by minimising the training loss.
4. Estimate the risk on unseen data (test set or cross-validation).

Assumptions

- Observations are independent and identically distributed (i.i.d.)
- The error ε has mean 0, variance σ^2 and is independent of X

Advantages

- Gives a principled way to measure error and uncertainty
- Supports both prediction and inference

Disadvantages

- Conclusions depend on the assumptions made
- The irreducible error σ^2 can never be removed
- Needs enough data to estimate f reliably

Applications

- Forecasting and risk modelling
- Scientific data analysis
- Design and evaluation of ML algorithms

2.2 Supervised and Unsupervised Learning: The Statistical View

2.2.1 Supervised Learning

We observe n pairs $(x_1, y_1), \ldots, (x_n, y_n)$ drawn *independently* from an unknown joint distribution $P(X, Y)$ (this is the **i.i.d. assumption**: independent and identically distributed). We use them to learn a function that predicts Y from X.

Key Idea

What is the best possible predictor?

- For **regression** with squared error, the best predictor is the *conditional mean* $f^*(x) = \mathbb{E}[Y \mid X = x]$ (the **regression function**).
- For **classification**, the best predictor is the **Bayes classifier**: assign x to the class with the highest posterior probability, $\hat{y} = \arg\max_k P(Y = k \mid X = x)$.

Every supervised method is an attempt to approximate one of these using only a finite sample.

The error of the Bayes classifier is the **Bayes error rate**. It is the irreducible error for classification: even the perfect classifier is wrong sometimes when classes overlap.

2.2.2 Unsupervised Learning

We observe only $x_1, \ldots, x_n$ and no y. Statistically, the goal is to learn the structure of the distribution $P(X)$ itself:

- **density estimation:** estimate $p(x)$;
- **clustering:** find groups (modes) in $p(x)$;

- **dimensionality reduction:** find a low-dimensional description of X that retains most of its variation.

Since there is no "correct answer" to compare against, evaluating unsupervised methods is harder and often relies on domain judgement.

	Supervised	**Unsupervised**
Data	(x_i, y_i) pairs	x_i only
Learns	$P(Y \mid X)$ or f	$P(X)$ / structure
Goal	Predict Y for new X	Discover patterns
Loss	Prediction error vs. true y	No direct loss (e.g. within-cluster variance)

2.3 Loss Functions, Training Loss and Test Loss

2.3.1 Loss Functions

To say how good $\hat{f}$ is, we need a number measuring the cost of a wrong prediction.

Definition: Loss function

A **loss function** $L(y, \hat{y}) \geq 0$ measures how bad the prediction $\hat{y}$ is when the true value is y. Lower is better, and $L(y, y) = 0$.

Loss	**Formula**	**Used for / property**
Squared error (L2)	$(y - \hat{y})^2$	Regression; punishes large errors heavily; smooth
Absolute error (L1)	$\lvert y - \hat{y} \rvert$	Regression; robust to outliers
0–1 loss	$\mathbb{1}[y \neq \hat{y}]$	Classification; counts mistakes
Cross-entropy (log loss)	$-\sum_k \mathbb{1}[y = k] \ln \hat{p}_k$	Probabilistic classifiers (Chapter 4, 9)

Example: Computing losses

True values $y = (3, 5, 7, 9)$; predictions $\hat{y} = (2, 6, 7, 11)$.

i	1	2	3	4	**Sum**
$y_i - \hat{y}_i$	1	-1	0	-2	
$\lvert y_i - \hat{y}_i \rvert$	1	1	0	2	4
$(y_i - \hat{y}_i)^2$	1	1	0	4	6

$$\text{MAE} = \tfrac{4}{4} = 1, \qquad \text{MSE} = \tfrac{6}{4} = 1.5, \qquad \text{RMSE} = \sqrt{1.5} \approx 1.22.$$

Observation: the single error of 2 contributes $2/4 = 50\%$ of the absolute error but $4/6 \approx 67\%$ of the squared error. Squared loss emphasises large errors.

2.3.2 Risk, Training Loss and Test Loss

Definition: Risk (expected loss, generalisation error)

The **risk** of a function f is its *expected* loss on a new observation drawn from the true distribution:

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y)}\big[L(Y, f(X))\big].$$

The risk is exactly what we want to minimise, but it needs the unknown $P(X, Y)$, so we can never compute it. We estimate it from samples.

Definition: Training loss (empirical risk)

For the training sample $(x_i, y_i)_{i=1}^n$,

$$\hat{\mathcal{R}}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, f(x_i)).$$

Empirical risk minimisation (ERM): choose $\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{\mathcal{R}}_{\text{train}}(f)$ over a chosen family $\mathcal{F}$ of models.

Definition: Test loss

For a *separate* sample $(x'_j, y'_j)_{j=1}^m$ that was **not** used for training,

$$\hat{\mathcal{R}}_{\text{test}}(\hat{f}) = \frac{1}{m}\sum_{j=1}^{m} L(y'_j, \hat{f}(x'_j)).$$

It is an *honest* (unbiased) estimate of the risk $\mathcal{R}(\hat{f})$.

Common Mistake

Training loss is an optimistic estimate of risk. The model was *chosen to minimise it* on exactly those points, so it fits them (and their noise) better than it will fit new data. Test loss $\geq$ training loss on average. Never report training loss as the performance of a model.

Try It Yourself

See the gap between training and test loss for polynomial fits of degree 1, 3 and 9 to noisy data from $y = \sin(2\pi x)$:

```
import numpy as np
rng = np.random.default_rng(0)
x = np.sort(rng.uniform(0, 1, 30))
y = np.sin(2*np.pi*x) + rng.normal(0, 0.25, 30)    # f + noise
idx = rng.permutation(30)
tr, te = idx[:20], idx[20:]                          # 20 train / 10 test

def mse(coef, i):
    return np.mean((np.polyval(coef, x[i]) - y[i])**2)

for d in [1, 3, 9]:
    coef = np.polyfit(x[tr], y[tr], d)               # ERM for degree d
    print(d, round(mse(coef, tr), 3), round(mse(coef, te), 3))
```

Output (seed 0):

Degree	Train MSE	Test MSE	Verdict
1	0.222	0.255	underfits (both high)
3	0.041	0.050	good fit
9	0.026	0.102	overfits (test $\approx 4\times$ train)

Notice that the degree-9 model has the *lowest* training loss but a test loss twice that of degree 3. Training loss alone would have picked the wrong model.

Write-up for the Exam: Empirical Risk Minimisation (ERM)

Definition

Empirical risk minimisation chooses the model $\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_i L(y_i, f(x_i))$, i.e. the function in a chosen family $\mathcal{F}$ with the *lowest average loss on the training data*, as a stand-in for the true (unknown) risk.

Working

1. Choose a model family $\mathcal{F}$ and a loss L.
2. Compute the training loss (empirical risk) of candidate functions.
3. Optimise to find the minimiser $\hat{f}$.
4. Estimate the true risk of $\hat{f}$ using the test loss.

Assumptions

- Training and test data come from the same distribution
- $\mathcal{F}$ is rich enough to contain a good function but not so rich that it memorises noise

Advantages

- Simple and general; underlies most ML algorithms

Disadvantages

- Training loss is an *optimistic* estimate of risk
- Overfits if $\mathcal{F}$ is too flexible (needs regularisation or validation)

Applications

- Least-squares regression
- Logistic regression and SVMs
- Neural network training

2.3.3 Splitting the Data

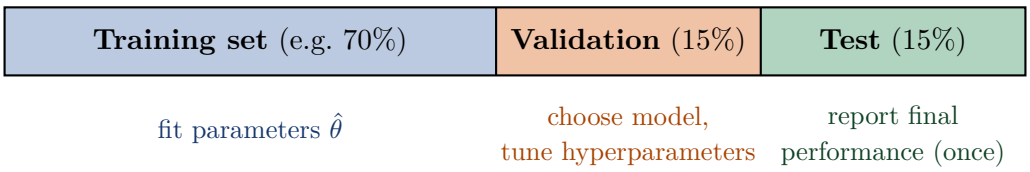


Figure 2.2. Splitting a dataset into training, validation and test sets.

Reading the Figure

The long bar is the *whole* dataset, cut into three **non-overlapping** parts. The label under each part says its single job. The **blue** part is the only one the learning algorithm is allowed to fit. The **orange** part is used to *compare* models and choose hyperparameters. The **green** part is kept locked away until the very end and used *once* to report the final honest performance. Mixing these jobs (for example, tuning on the test part) makes the final number over-optimistic.

2.4 Trade-offs in Statistical Learning

2.4.1 The Bias–Variance Trade-off

Take a *fixed* test point x_0. Imagine collecting many different training sets, each of size n, and fitting $\hat{f}$ on each one. The predictions $\hat{f}(x_0)$ vary from set to set. The expected squared prediction error at x_0 then splits into three parts:

$$\mathbb{E}\Big[\big(y_0 - \hat{f}(x_0)\big)^2\Big] = \underbrace{\Big(\mathbb{E}[\hat{f}(x_0)] - f(x_0)\Big)^2}_{\mathrm{Bias}(\hat{f}(x_0))^2} + \underbrace{\mathbb{E}\Big[\big(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)]\big)^2\Big]}_{\mathrm{Var}(\hat{f}(x_0))} + \underbrace{\sigma^2}_{\text{irreducible}}. \tag{2.3}$$

Definition: Bias and variance of a method

Bias is the error caused by approximating a complicated real problem by a simpler model: how far the *average* prediction is from the truth. **Variance** is how much the prediction would *change* if we trained on a different sample.

Proof sketch of Equation (2.3). Write $y_0 = f + \varepsilon$ and let $\bar{f} = \mathbb{E}[\hat{f}]$. Add and subtract $\bar{f}$:

$$y_0 - \hat{f} = (f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon.$$

Square and take expectations. The cross-terms vanish because $\mathbb{E}[\bar{f} - \hat{f}] = 0$ and $\mathbb{E}[\varepsilon] = 0$ with ε independent of $\hat{f}$. What remains is $(f - \bar{f})^2 + \mathbb{E}[(\hat{f} - \bar{f})^2] + \mathbb{E}[\varepsilon^2]$, i.e. $\mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$. ■

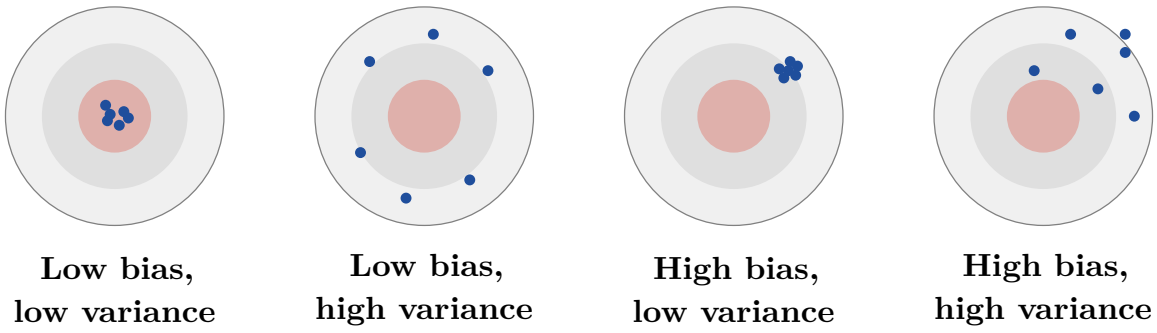


Figure 2.3. Bias and variance explained with a dartboard. Red centre = true value $f(x_0)$; blue dots = predictions from models trained on different samples.

Reading the Figure

Think of each **blue dot** as the prediction made by a model trained on a different random sample, and the **red bull's-eye** as the true value.

- *Low bias, low variance:* dots are tightly clustered *on* the centre. This is the ideal.
- *Low bias, high variance:* dots are centred correctly on average but scattered. Predictions depend heavily on the particular sample (**overfitting**).
- *High bias, low variance:* dots are tightly clustered but *away* from the centre. The model is consistently wrong in the same way (**underfitting**).
- *High bias, high variance:* dots are scattered and off-centre: the worst case.

Bias is the distance from the *centre of the dots* to the bull's-eye; *variance* is the *spread* of the dots.

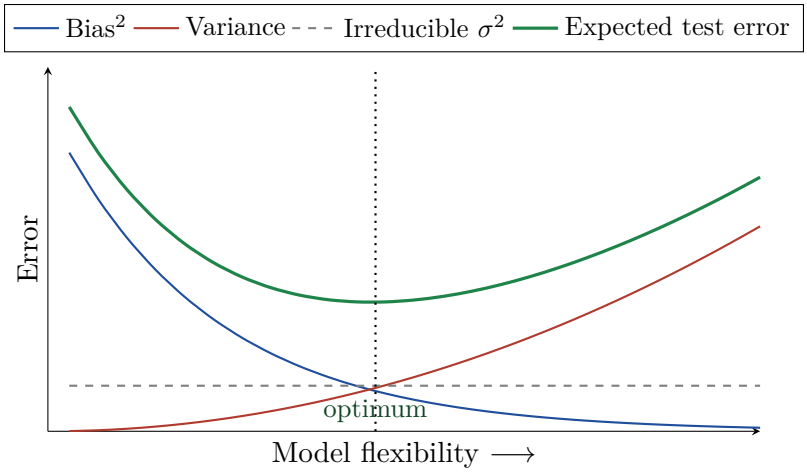


Figure 2.4. Bias^2 falls and variance rises as flexibility increases; expected test error is their sum plus noise.

Reading the Figure

The horizontal axis is how flexible the model is (e.g. polynomial degree, or $1/k$ in k-NN). Read the four curves:

- **Blue (bias^2)** starts high and falls: a flexible model can follow the true f more closely.
- **Red (variance)** starts near zero and rises: a flexible model chases the noise in each training sample.
- **Grey dashed (irreducible error)** is a flat floor that no model can beat.
- **Green (expected test error)** is the *sum* of the three. It is U-shaped: its lowest point (dotted line) is the best flexibility.

Left of the optimum, bias dominates (underfitting); right of it, variance dominates (overfitting). Compare this with Figure 1.8 in Chapter 1.

Example: Putting numbers on the decomposition

At a test point, a model has Bias$^2 = 4$, Var $= 3$ and noise $\sigma^2 = 2$.

$$\text{Expected test MSE} = 4 + 3 + 2 = 9.$$

If we switch to a more flexible model with Bias$^2 = 1$ and Var $= 5$, the error becomes $1 + 5 + 2 = 8$: lower, even though variance rose, because the bias fell more. If flexibility is pushed further to Bias$^2 = 0.5$, Var $= 9$, error is $0.5 + 9 + 2 = 11.5$: worse. This is the trade-off.

Write-up for the Exam: Bias–Variance Trade-off

Definition

The expected squared prediction error at a point splits into three parts: $\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. **Bias** is the error from simplifying assumptions; **variance** is the sensitivity of $\hat{f}$ to the particular training sample; σ^2 is irreducible noise. The **trade-off** is that lowering one usually raises the other.

Explanation

1. Simple, inflexible model: *high bias, low variance* $\Rightarrow$ underfitting.
2. Complex, flexible model: *low bias, high variance* $\Rightarrow$ overfitting.
3. Test error is U-shaped in flexibility; the best model lies at the bottom.

How to control

- Choose flexibility with cross-validation
- Regularise to reduce variance
- Collect more data to reduce variance
- Ensemble methods (Chapter 5) reduce variance further

Example

With $\text{Bias}^2 = 4$, $\text{Var} = 3$, $\sigma^2 = 2$ the expected test MSE is 9; a more flexible model with $\text{Bias}^2 = 1$, $\text{Var} = 5$ gives 8 (better), while $\text{Bias}^2 = 0.5$, $\text{Var} = 9$ gives 11.5 (worse).

2.4.2 Other Trade-offs

1. **Flexibility vs. interpretability.** Simple models (linear regression) are easy to explain but may be too rigid; flexible models (boosting, deep networks) can fit complex patterns but act as black boxes.
2. **Prediction accuracy vs. inference.** If you need to *explain* effects, prefer simple models; if you only need accurate predictions, flexible models are fine.
3. **Model complexity vs. amount of data.** Complex models need much more data. With little data, a simple model usually generalises better.
4. **Training cost vs. accuracy.** More flexible models and more data take longer to train and need more memory.

Reading the Figure

Each box is a family of methods placed by two properties. Moving **right**, models become more flexible (can fit more complicated shapes). Moving **down**, they become harder to interpret. The dashed arrow shows the general trend: *what you gain in flexibility you usually pay for in interpretability.* The positions are rough guides, not exact measurements;

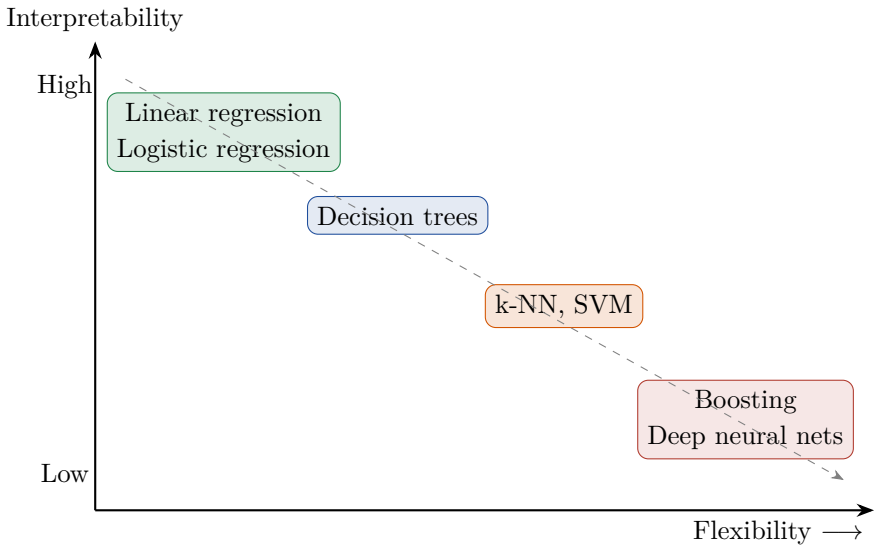


Figure 2.5. Typical position of methods on the flexibility–interpretability trade-off (qualitative).

these methods are covered in Chapters 3–9.

Exam Tip

The bias–variance trade-off is the single most-asked topic of Unit I. Be ready to (i) state Equation (2.3), (ii) sketch Figure 2.4 with the four curves, and (iii) link high bias to underfitting and high variance to overfitting.

2.5 Estimating Risk Statistics

We cannot compute the true risk $\mathcal{R}(\hat{f})$, so we *estimate* it. A good estimate should be nearly unbiased and have low variance.

2.5.1 Hold-out (Validation-set) Estimate

Split once into training and test parts and compute $\hat{\mathcal{R}}_{\text{test}}$. It is simple and fast, but (i) the number depends on *which* points fell in the test set, and (ii) the model is trained on less data, so it may look slightly worse than it is.

Uncertainty of the estimate. If the m test losses $\ell_j = L(y'_j, \hat{f}(x'_j))$ have sample mean $\bar{\ell} = \hat{\mathcal{R}}_{\text{test}}$ and sample standard deviation s, then

$$\text{SE}(\bar{\ell}) = \frac{s}{\sqrt{m}}, \qquad \text{approximate 95\% CI for the risk:} \quad \bar{\ell} \pm 1.96\ \text{SE}(\bar{\ell}).$$

Example: Confidence interval for test MSE

On $m = 100$ test points the squared errors have mean 4.1 and standard deviation $s = 3$. Then $\text{SE} = 3/\sqrt{100} = 0.3$ and the 95% interval is $4.1 \pm 1.96(0.3) = 4.1 \pm 0.59$, i.e. [3.51, 4.69]. So we may report "test MSE ≈ 4.1, likely between 3.5 and 4.7". Quadrupling the test set to $m = 400$ would halve the standard error to 0.15.

2.5.2 k-Fold Cross-Validation

When data are scarce, we want to use *all* of it for both training and evaluation.

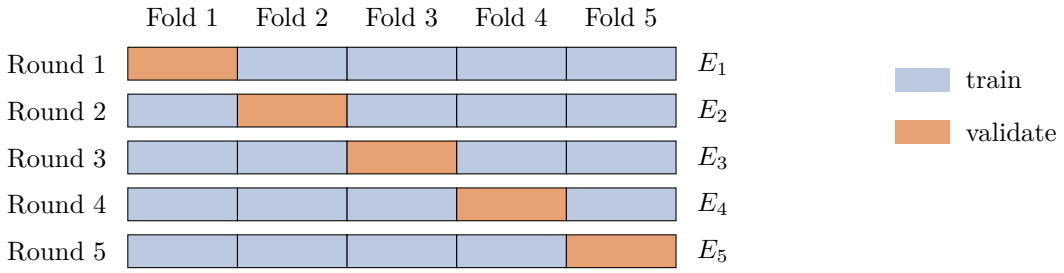


Figure 2.6. 5-fold cross-validation. Each round uses a different fold (orange) for validation and the other four (blue) for training.

Reading the Figure

The dataset is cut into $k = 5$ equal **folds** (the columns). Each **row** is one training round. In Round 1 the model is trained on Folds 2–5 (blue) and its error E_1 is measured on Fold 1 (orange). In Round 2 the orange fold moves to Fold 2, and so on. After five rounds *every point has been used for validation exactly once*, and the final estimate is the average $E_1, \dots, E_5$ of the five errors.

Definition: k-fold cross-validation estimate

Split the data into k folds. For $i = 1, \dots, k$ train on all folds except the i-th and compute the error E_i on the i-th. Then

$$\text{CV}_{(k)} = \frac{1}{k} \sum_{i=1}^{k} E_i.$$

Example: 5-fold CV with numbers

Validation errors: $E = (4.2,\ 3.8,\ 4.6,\ 4.0,\ 3.9)$.

$$\text{CV}_{(5)} = \frac{4.2 + 3.8 + 4.6 + 4.0 + 3.9}{5} = \frac{20.5}{5} = 4.1.$$

Deviations from 4.1: 0.1, −0.3, 0.5, −0.1, −0.2; squares sum to 0.40; $s^2 = 0.40/4 = 0.10$, $s = 0.316$; $\text{SE} = s/\sqrt{5} = 0.141$. So the estimated risk is 4.1 ± 0.14.

Write-up for the Exam: k-fold Cross-Validation

Definition

k-fold cross-validation estimates the risk of a model by splitting the data into k equal folds, training on $k-1$ folds and validating on the remaining fold, repeating k times, and averaging the k validation errors.

Working

1. Shuffle the data and split it into k folds.
2. For $i = 1, \dots, k$: train on all folds except fold i; compute the error E_i on fold i.
3. Report $\mathrm{CV}_{(k)} = \frac{1}{k}\sum_i E_i$ (and its standard error).

Advantages

- Uses all data for both training and validation
- More reliable than a single hold-out split
- Standard tool for model selection and tuning

Disadvantages

- Needs k model fits, so it is k times slower
- Not suitable as-is for time-series data (order matters)

Applications

- Hyperparameter tuning
- Comparing different models
- Small datasets

Choosing k. Leave-one-out CV (LOOCV) is the extreme $k = n$: nearly unbiased but expensive and with high variance. In practice $k = 5$ or $k = 10$ gives a good bias–variance balance. For classification use *stratified* folds, which keep the class proportions the same in each fold.

	Hold-out	**k-fold CV**	**LOOCV**
Number of fits	1	k	n
Computation	Cheapest	Moderate	Most expensive
Bias of estimate	Higher (less training data)	Low–moderate	Lowest
Variance of estimate	High (one split)	Moderate	High
Typical use	Large datasets	Default choice	Very small datasets

2.5.3 Risk Statistics for Classification

For classification with 0–1 loss, the risk is the **error rate**. We summarise the test predictions in a **confusion matrix**.

		Predicted	
		Positive	Negative
Actual	Positive	TP (true positive)	FN (false negative)
	Negative	FP (false positive)	TN (true negative)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN},$$

$$\text{Specificity} = \frac{TN}{TN+FP}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision answers "of the items I flagged, how many were truly positive?"; *recall* answers "of all true positives, how many did I find?"; F_1 is their harmonic mean.

Write-up for the Exam: Confusion Matrix and Classification Metrics

Definition

The **confusion matrix** tabulates a classifier's predictions against the true classes: true positives (TP), false negatives (FN), false positives (FP) and true negatives (TN). All classification metrics are computed from these four counts.

Metrics

- **Accuracy** $= \frac{TP+TN}{TP+TN+FP+FN}$
- **Precision** $= \frac{TP}{TP+FP}$ (how many flagged items were correct)
- **Recall** $= \frac{TP}{TP+FN}$ (how many true positives were found)
- F_1**-score** $= \frac{2PR}{P+R}$ (harmonic mean of precision and recall)

Advantages

- Shows *what kind* of errors are made, not only how many
- Works for imbalanced data when precision/recall are used

Disadvantages

- Accuracy misleads on imbalanced classes
- Depends on the chosen decision threshold

Applications

- Medical screening, fraud detection, spam filtering

Example: Spam filter on 100 test e-mails

Actual spam $= 50$, actual non-spam $= 50$. The filter finds $TP = 40$ spam mails, misses $FN = 10$, wrongly flags $FP = 5$ good mails and correctly passes $TN = 45$.

$$\text{Accuracy} = \tfrac{40+45}{100} = 0.85, \qquad \text{Precision} = \tfrac{40}{45} \approx 0.889,$$
$$\text{Recall} = \tfrac{40}{50} = 0.80, \qquad \text{Specificity} = \tfrac{45}{50} = 0.90,$$
$$F_1 = \tfrac{2(0.889)(0.80)}{0.889+0.80} \approx 0.842.$$

Common Mistake

Accuracy can mislead on imbalanced data. If only 1% of transactions are fraud, a program that always says "not fraud" is 99% accurate yet has recall 0 and is useless. Use precision, recall and F_1 instead.

2.6 Sampling Distribution of an Estimator

Everything we compute from data (a mean, a test error, a regression coefficient) is a *statistic* calculated from a random sample. Draw a different sample and you get a different number. This section studies that randomness.

Definition: Estimator and estimate

An **estimator** $\hat{\theta} = g(X_1, \ldots, X_n)$ is a *rule* (a function of the sample) for guessing an unknown population quantity θ. An **estimate** is the *number* that the rule gives for one particular sample. Since the sample is random, the estimator $\hat{\theta}$ is itself a **random variable**.

Definition: Sampling distribution

The **sampling distribution** of an estimator is its probability distribution over all possible samples of size n from the population. Its *standard deviation* is called the **standard error**, $\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$.

Write-up for the Exam: Sampling Distribution of an Estimator

Definition

An **estimator** $\hat{\theta} = g(X_1, \ldots, X_n)$ is a rule for guessing a population quantity θ from a sample. Because the sample is random, $\hat{\theta}$ is a random variable; its probability distribution over all possible samples of size n is its **sampling distribution**, and the standard deviation of that distribution is the **standard error** $\mathrm{SE}(\hat{\theta})$.

Working

1. Imagine drawing many samples of size n from the population.
2. Compute the statistic $\hat{\theta}$ for every sample.
3. The spread of these values is the sampling distribution; its mean shows bias, its standard deviation gives the standard error.

Key properties

- **Unbiased:** $\mathbb{E}[\hat{\theta}] = \theta$
- **Low variance / standard error**: precise estimates
- **Consistent:** $\hat{\theta} \to \theta$ as $n \to \infty$

Example

For the sample mean, $\mathbb{E}[\bar{X}] = \mu$ and $\mathrm{SE}(\bar{X}) = \sigma/\sqrt{n}$; by the Central Limit Theorem $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n.

Importance

- Gives confidence intervals: $\bar{x} \pm 1.96$ SE
- Tells us how much to trust an estimate such as test error

2.6.1 Properties of Good Estimators

Property	Definition	Meaning
Bias	$\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$	Average error; **unbiased** if $= 0$
Variance	$\mathrm{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2]$	Spread across samples
Standard error	$\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$	Typical size of the estimation error
MSE	$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2$	Overall accuracy (same decomposition as before!)
Consistency	$\hat{\theta} \to \theta$ as $n \to \infty$	More data gives a better estimate

2.6.2 The Sample Mean

Let $X_1, \ldots, X_n$ be i.i.d. with population mean μ and variance σ^2, and let $\bar{X} = \frac{1}{n}\sum X_i$. Then

$$\mathbb{E}[\bar{X}] = \mu \quad \text{(unbiased)}, \qquad \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}, \qquad \mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

Proof. $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \frac{1}{n}(n\mu) = \mu$, and, by independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. ■

Key Idea

The standard error shrinks like $1/\sqrt{n}$. To make an estimate *twice* as precise you need *four times* as much data.

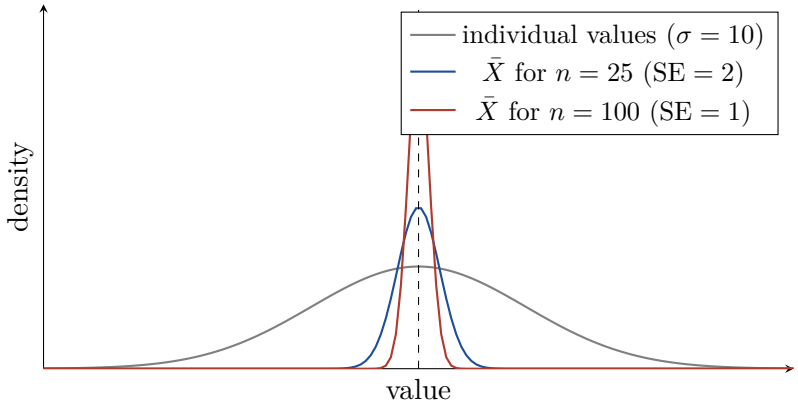


Figure 2.7. Sampling distribution of the sample mean for population mean $\mu = 50$ and $\sigma = 10$.

Reading the Figure

Three bell curves are drawn, all centred on the true mean $\mu = 50$ (dashed line).

- **Grey (widest):** the spread of *individual* observations, $\sigma = 10$.
- **Blue:** the distribution of the *average of 25 observations*. It is still centred at 50 (*unbiased*) but is 5× narrower, SE $= 10/\sqrt{25} = 2$.
- **Red (narrowest and tallest):** the average of 100 observations, SE $= 1$.

The *height* of a curve at a value shows how likely it is that the estimate lands there. The bigger the sample, the more tightly the estimates cluster around the truth. Averages are much more reliable than single observations.

Example: Using the standard error

A population has $\mu = 50$ and $\sigma = 10$. For samples of size $n = 25$, what is $P(\bar{X} > 53)$? SE $= 10/\sqrt{25} = 2$, so $z = \dfrac{53 - 50}{2} = 1.5$ and $P(\bar{X} > 53) = P(Z > 1.5) \approx 0.0668$, about 6.7%. With $n = 100$, SE $= 1$, $z = 3$, and the probability drops to about 0.0013.

2.6.3 Central Limit Theorem

Definition: Central Limit Theorem (CLT)

Let $X_1, \dots, X_n$ be i.i.d. with mean μ and finite variance σ^2. For large n the sampling distribution of $\bar{X}$ is approximately normal:

$$\bar{X} \;\approx\; \mathcal{N}\!\left(\mu,\, \frac{\sigma^2}{n}\right), \qquad \text{equivalently} \qquad \frac{\bar{X}-\mu}{\sigma/\sqrt{n}} \;\approx\; \mathcal{N}(0,1),$$

whatever the shape of the original population.

The CLT is the reason the normal curve appears everywhere: test errors, accuracies and averages of many small effects are sums of many small random pieces. It justifies the confidence intervals used in the previous section (a rule of thumb: $n \geq 30$ is usually enough).

95% confidence interval for a mean. $\bar{x} \pm 1.96\,\dfrac{s}{\sqrt{n}}$.

Example: Confidence interval

A sample of $n = 100$ gives $\bar{x} = 52$ and $s = 10$. SE $= 10/10 = 1$, so the 95% interval is 52 ± 1.96, i.e. [50.04, 53.96]. If we repeated the experiment many times, about 95% of intervals built this way would contain the true μ.

2.6.4 A Biased Estimator: the Variance

Not all natural estimators are unbiased. Consider estimating σ^2:

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2, \qquad S_{n-1}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

It can be shown that $\mathbb{E}[S_n^2] = \dfrac{n-1}{n}\sigma^2 < \sigma^2$, so S_n^2 is **biased** (it underestimates), while S_{n-1}^2 is unbiased. The reason is that deviations are measured from $\bar{X}$, which is itself fitted to the same data and so sits *closer* to the sample than the true μ does. This is the same "fits its own data too well" effect that makes training loss optimistic.

Example: Size of the bias

With $n = 5$ and $\sigma^2 = 10$: $\mathbb{E}[S_n^2] = \frac{4}{5}(10) = 8$ (bias $= -2$), whereas $\mathbb{E}[S_{n-1}^2] = 10$.

Try It Yourself

Verify the standard-error formula and the bias of S_n^2 by simulation:

```
import numpy as np
rng = np.random.default_rng(1)

for n in [5, 25, 100]:
    means = [rng.normal(50, 10, n).mean() for _ in range(10000)]
    print(n, round(np.mean(means), 2), round(np.std(means), 2),
          round(10/np.sqrt(n), 2))       # theory: SE = sigma/sqrt(n)

v0 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=0) for _ in range(20000)]
v1 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=1) for _ in range(20000)]
print(round(np.mean(v0), 2), round(np.mean(v1), 2))
```

Typical output:

n	mean of $\bar{X}$	std of $\bar{X}$	theory $10/\sqrt{n}$
5	49.92	4.39	4.47
25	49.98	2.00	2.00
100	50.00	1.01	1.00

The means stay near 50 (unbiased) and the spread follows $\sigma/\sqrt{n}$. The two variance estimates average about 8.0 (ddof $=0$, biased) and 9.9 (ddof $=1$, unbiased, close to the true 10).

Key Idea

Estimators (including a trained model's predictions and its test error) are random. To trust a number, we need to know **how much it would vary from sample to sample**: that is its **standard error**.

Chapter Summary

- Statistical model: $Y = f(X) + \varepsilon$; we estimate f for *prediction* or *inference*. Error = reducible + irreducible (σ^2).
- Supervised learning estimates $P(Y \mid X)$ (best predictors: conditional mean, Bayes classifier). Unsupervised learning studies $P(X)$.
- Loss measures a single mistake (squared, absolute, 0–1, cross-entropy); **risk** is expected loss; **training loss** (empirical risk) is optimistic; **test loss** is an honest estimate.
- $\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. More flexibility lowers bias but raises variance; the

best model balances them.

- Other trade-offs: flexibility vs. interpretability, prediction vs. inference, complexity vs. data size.
- Estimate risk by hold-out, k-fold CV ($k = 5$ or 10) or LOOCV; for classification use the confusion matrix, precision, recall and F_1.
- An estimator is a random variable; its spread is the **standard error**. For the sample mean $\text{SE} = \sigma/\sqrt{n}$, and the **CLT** makes $\bar{X}$ approximately normal.
- S_n^2 (divide by n) is biased; S_{n-1}^2 is unbiased.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Write the statistical model $Y = f(X) + \varepsilon$ and name each term. **[2 marks]**
2. Differentiate between prediction and inference. **[2 marks]**
3. What is irreducible error? **[2 marks]**
4. Define a loss function. Give two examples. **[2 marks]**
5. Why is training loss an optimistic estimate of the risk? **[2 marks]**
6. Define bias and variance of a learning method. **[2 marks]**
7. What is k-fold cross-validation? **[2 marks]**
8. Define standard error of an estimator. **[2 marks]**
9. State the Central Limit Theorem. **[2 marks]**
10. Define precision and recall. **[2 marks]**

Model Answer: Q5 and Q8

Q5. The model parameters are chosen to *minimise the loss on the training data itself*, so the model partly memorises its noise; on fresh data it performs worse. Hence training loss $\leq$ expected test loss.

Q8. The standard error of an estimator $\hat{\theta}$ is the standard deviation of its sampling distribution, $\text{SE}(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})}$. For the sample mean it equals $\sigma/\sqrt{n}$.

B. Short Answer Questions (5 marks each)

1. Distinguish parametric and non-parametric methods with examples. **[5 marks]**

2. Explain training loss, test loss and risk. Why do we need a separate test set? **[5 marks]**
3. Explain the bias–variance trade-off with a diagram. **[5 marks]**
4. Explain k-fold cross-validation with a diagram. Compare it with the hold-out method. **[5 marks]**
5. Explain the confusion matrix and define accuracy, precision, recall and F_1. **[5 marks]**
6. What is the sampling distribution of an estimator? Show that the sample mean is unbiased with $\mathrm{Var}(\bar{X}) = \sigma^2/n$. **[5 marks]**
7. Why is accuracy a poor measure for imbalanced data? Illustrate with an example. **[5 marks]**

Model Answer: B6 (outline for 5 marks)

An *estimator* $\hat{\theta} = g(X_1, \ldots, X_n)$ is a function of a random sample, hence a random variable; its probability distribution over repeated samples of size n is its *sampling distribution*. For i.i.d. X_i with mean μ and variance σ^2: $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \mu$, so $\bar{X}$ is unbiased. By independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. Hence $\mathrm{SE}(\bar{X}) = \sigma/\sqrt{n}$; precision improves as n grows. By the CLT, $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n.

C. Long Answer Questions (10 marks each)

1. Derive the bias–variance decomposition of the expected squared prediction error. Explain its implications for model selection and sketch the curves. **[10 marks]**
2. Describe the different methods of estimating the risk of a model (hold-out, k-fold, LOOCV) with their advantages and disadvantages. **[10 marks]**
3. Explain the sampling distribution of the sample mean. State the CLT and explain, with an example, how it is used to construct a confidence interval. **[10 marks]**
4. Compare supervised and unsupervised learning from a statistical point of view, and discuss the trade-offs in statistical learning. **[10 marks]**
5. A classifier is tested on 200 samples: $TP = 70$, $FN = 30$, $FP = 20$, $TN = 80$. Compute accuracy, precision, recall, specificity and F_1. **[10 marks]**

Model Answer: C5 (numerical)

$n = 200$. Accuracy $= \frac{70+80}{200} = 0.75$. Precision $= \frac{70}{70+20} = 0.778$. Recall $= \frac{70}{70+30} = 0.70$. Specificity $= \frac{80}{80+20} = 0.80$.
$F_1 = \frac{2(0.778)(0.70)}{0.778+0.70} = \frac{1.089}{1.478} \approx \mathbf{0.737}$.

D. Numerical Problems

1. True $y = (2, 4, 6, 8)$, predicted $\hat{y} = (3, 3, 6, 10)$. Compute MAE, MSE and RMSE. *(Answer: $MAE = 1$, $MSE = 1.5$, $RMSE = 1.22$.)*

2. A model has $\text{Bias}^2 = 2.5$, $\text{Var} = 1.5$, $\sigma^2 = 1$. Find the expected test MSE. After switching to a simpler model, $\text{Bias}^2 = 4$, $\text{Var} = 0.5$: which is better? *(Answer: 5; the new one gives 5.5, so the first is better.)*

3. 4-fold CV errors are 6, 5, 7, 6. Find the CV estimate and its standard error. *(Answer: 6; $s = 0.816$, $\text{SE} = 0.408$.)*

4. A population has $\mu = 100$ and $\sigma = 20$. For $n = 64$, find $\text{SE}(\bar{X})$ and $P(\bar{X} < 97)$. *(Answer: $\text{SE} = 2.5$; $z = -1.2$; $P \approx 0.115$.)*

5. Test squared errors over 400 samples have mean 5 and $s = 4$. Give a 95% CI for the risk. *(Answer: $\text{SE} = 0.2$; $5 \pm 0.392 = [4.61, 5.39]$.)*

E. Multiple-Choice Questions

1. The irreducible error in $Y = f(X) + \varepsilon$ is:
   (a) Bias^2 (b) $\text{Var}(\hat{f})$ (c) $\text{Var}(\varepsilon)$ (d) $\hat{\mathcal{R}}_{\text{train}}$

2. As model flexibility increases, typically:
   (a) bias↑, variance↓ (b) bias↓, variance↑ (c) both ↑ (d) both ↓

3. The best predictor under squared loss is:
   (a) the median of Y (b) the mode of Y (c) $\mathbb{E}[Y \mid X]$ (d) $\mathbb{E}[Y]$

4. In 10-fold CV, each observation is used for validation:
   (a) once (b) ten times (c) never (d) a random number of times

5. If $\sigma = 12$ and $n = 36$, then $\text{SE}(\bar{X})$ is:
   (a) 12 (b) 6 (c) 2 (d) 0.33

6. Which estimator of σ^2 is unbiased?
   (a) $\frac{1}{n}\sum(X_i - \bar{X})^2$ (b) $\frac{1}{n-1}\sum(X_i - \bar{X})^2$ (c) $\frac{1}{n+1}\sum(X_i - \bar{X})^2$ (d) none

7. Recall is defined as:
   (a) $\frac{TP}{TP+FP}$ (b) $\frac{TP}{TP+FN}$ (c) $\frac{TN}{TN+FP}$ (d) $\frac{TP+TN}{n}$

8. Which loss is most robust to outliers?
   (a) squared (b) absolute (c) cubic (d) exponential

Answers: 1-c, 2-b, 3-c, 4-a, 5-c, 6-b, 7-b, 8-b.

F. Practice Problems

1. In Figure 2.1, suppose $f(x) = 2x + 1$ and $\sigma = 0.5$. What is the smallest possible expected squared error for any model, and why?

2. Run the polynomial experiment with degrees 2, 5 and 12 and sketch train/test MSE against degree. At which degree does the test MSE start rising?

3. **Coding:** implement 5-fold cross-validation with `cross_val_score` for `LinearRegression` and report the mean and standard error of the scores.

4. Simulate sampling distributions from a *skewed* population (e.g. `rng.exponential(1, n)`) and verify with histograms that $\bar{X}$ becomes bell-shaped as n grows (CLT).

Quick Revision

Topic	**One-line recall**
Model	$Y = f(X) + \varepsilon$, $\mathbb{E}\varepsilon = 0$, $\operatorname{Var}\varepsilon = \sigma^2$.
Prediction / Inference	Accurate $\hat{Y}$ / understand effect of each X_j.
Error split	Reducible + irreducible σ^2.
Best predictors	$\mathbb{E}[Y \mid X]$ (regression); Bayes classifier $\arg\max_k P(k \mid x)$.
Risk	$\mathbb{E}[L(Y, f(X))]$ (unknown); train loss optimistic; test loss honest.
Bias–variance	$\mathbb{E}[(y_0 - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$; U-shaped test error.
Hold-out / k-fold / LOOCV	1 fit / k fits (use 5 or 10) / n fits.
Classification stats	Acc $= \frac{TP+TN}{n}$, Prec $= \frac{TP}{TP+FP}$, Rec $= \frac{TP}{TP+FN}$, F_1 harmonic mean.
Sampling distribution	Distribution of an estimator over repeated samples; spread = SE.
Sample mean	Unbiased; SE $= \sigma/\sqrt{n}$; 4× data ⇒ 2× precision.
CLT	$\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n; 95% CI $\bar{x} \pm 1.96\,\text{SE}$.
Variance estimator	Divide by $n - 1$ for unbiasedness.

Chapter 3

Distance-Based Methods, Decision Trees and Naive Bayes

Learning Objectives

After studying this chapter you will be able to:

- compute **Euclidean, Manhattan, Minkowski, Chebyshev, Hamming** and **cosine** measures and explain why **feature scaling** matters;
- apply the **k-Nearest Neighbours** algorithm to classification and regression, and explain the effect of k;
- build a **decision tree** by hand using **entropy, information gain** and **Gini index**;
- explain **overfitting in trees** and **pruning**;
- state **Bayes' theorem** and apply the **Naive Bayes** classifier, including **Laplace smoothing** and the **Gaussian** variant;
- compare the three methods and choose between them;
- answer 2-, 5- and 10-mark university questions on these topics.

> *Birds of a feather flock together: the nearest neighbour of a point carries a lot of information about its class.*
>
> — Cover & Hart, 1967 (on nearest neighbours)

This chapter begins Unit II (Supervised Learning). The three methods here are among the simplest and most intuitive classifiers, and each uses a different idea:

Method	Core idea	Type (Chapter 1 / 2)
k-Nearest Neighbours	"You are like your closest neighbours"	instance-based, non-parametric
Decision tree	"Ask a sequence of yes/no questions"	model-based, non-parametric
Naive Bayes	"Use probability and Bayes' rule"	model-based, parametric

3.1 Distance-Based Methods

Many learning algorithms (k-NN, K-Means, SVMs with kernels) depend on one question: *how similar are two instances?* We answer it with a **distance** (small = similar) or a **similarity** (large = similar).

Definition: Distance metric

A function $d(\boldsymbol{a}, \boldsymbol{b})$ is a **metric** if for all points: (i) $d(\boldsymbol{a}, \boldsymbol{b}) \geq 0$ and $d(\boldsymbol{a}, \boldsymbol{b}) = 0 \iff \boldsymbol{a} = \boldsymbol{b}$; (ii) symmetry $d(\boldsymbol{a}, \boldsymbol{b}) = d(\boldsymbol{b}, \boldsymbol{a})$; (iii) triangle inequality $d(\boldsymbol{a}, \boldsymbol{c}) \leq d(\boldsymbol{a}, \boldsymbol{b}) + d(\boldsymbol{b}, \boldsymbol{c})$.

3.1.1 Common Distance Measures

For two d-dimensional points $\boldsymbol{a} = (a_1, \ldots, a_d)$ and $\boldsymbol{b} = (b_1, \ldots, b_d)$:

Measure	Formula	Use
Euclidean (L_2)	$\sqrt{\sum_i (a_i - b_i)^2}$	Default for numeric data
Manhattan (L_1)	$\sum_i \lvert a_i - b_i \rvert$	Grid-like paths; less sensitive to outliers
Minkowski (L_p)	$\left(\sum_i \lvert a_i - b_i \rvert^p\right)^{1/p}$	General family ($p = 1, 2$ above)
Chebyshev (L_∞)	$\max_i \lvert a_i - b_i \rvert$	Largest single difference
Hamming	number of positions where $a_i \neq b_i$	Binary / categorical strings
Cosine similarity	$\dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{\lVert \boldsymbol{a} \rVert \, \lVert \boldsymbol{b} \rVert}$	Text; direction, not size

Example: Computing distances

Let $\boldsymbol{a} = (1, 2, 3)$ and $\boldsymbol{b} = (4, 6, 3)$. The coordinate differences are $3, 4, 0$.

$$d_{L_2} = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{25} = 5, \qquad d_{L_1} = 3 + 4 + 0 = 7,$$
$$d_{L_\infty} = \max(3, 4, 0) = 4, \qquad d_{L_3} = (27 + 64 + 0)^{1/3} = 91^{1/3} \approx 4.50.$$

Cosine similarity: $\boldsymbol{a} \cdot \boldsymbol{b} = 4 + 12 + 9 = 25$, $\lVert \boldsymbol{a} \rVert = \sqrt{14} \approx 3.742$, $\lVert \boldsymbol{b} \rVert = \sqrt{61} \approx 7.810$, so $\cos\theta = 25/29.22 \approx 0.855$.
Hamming: for the bit strings 10110 and 11100 the strings differ in positions 2 and 4, so $d_H = 2$.
Note the ordering $L_\infty \leq L_3 \leq L_2 \leq L_1$ for the same pair of points.

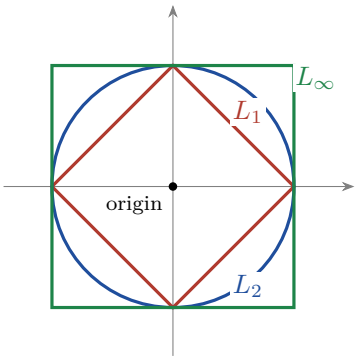


Figure 3.1. The set of all points at distance exactly 1 from the origin, for three metrics.

Reading the Figure

Each coloured outline is the set of points that are *exactly one unit away* from the origin under a different distance. Under **Euclidean** distance (L_2, blue) this set is a *circle*. Under **Manhattan** (L_1, red) it is a *diamond*, because you may only move along the axes. Under **Chebyshev** (L_∞, green) it is a *square*. The point $(1, 1)$ lies on the green square but far outside the red diamond: the *same pair of points* is counted as "close" or "far" depending on the metric. Hence the choice of metric changes which points are neighbours.

Write-up for the Exam: Distance-Based Methods

Definition

Distance-based methods make predictions or form groups using the *distance (or similarity) between instances* in feature space: close points are assumed to be similar.

Working

1. Choose a distance measure (Euclidean, Manhattan, Minkowski, Hamming, cosine).
2. Scale the features so that none dominates.
3. Compute distances between the query and the data.
4. Use the nearest or most similar instances to predict or to form clusters.

Assumptions

- Nearby points tend to share the same label or behaviour
- The chosen metric reflects real similarity

Advantages

- Intuitive and easy to implement
- Little or no training
- Works for classification, regression and clustering

Disadvantages

- Very sensitive to feature scale and irrelevant features

- Slow for large datasets
- Loses meaning in high dimensions (curse of dimensionality)

Applications

- k-NN and K-Means clustering
- Recommender systems
- Anomaly detection

3.1.2 Feature Scaling

Distance measures add up differences across all features, so a feature with a *large numeric range* dominates the sum. We must put features on a comparable scale.

Method	Formula	Result
Min–max scaling	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	Values in $[0, 1]$
Standardisation (z-score)	$x' = \dfrac{x - \mu}{\sigma}$	Mean 0, standard deviation 1

Example: Why scaling matters

Three people: P = (age 25, salary 40000), Q = (45, 42000), R = (26, 80000). Raw Euclidean distances:

$$d(P, Q) = \sqrt{20^2 + 2000^2} \approx 2000.1, \qquad d(P, R) = \sqrt{1^2 + 40000^2} \approx 40000.0.$$

The salary difference swamps age, so Q (20 years older) looks far closer to P than R (1 year older) does, which is silly. Now min–max scale with age range 20–60 and salary range 20 000–100 000:

$$P' = (0.125,\ 0.250), \quad Q' = (0.625,\ 0.275), \quad R' = (0.150,\ 0.750).$$

$d(P', Q') = \sqrt{0.5^2 + 0.025^2} \approx 0.5006$ and $d(P', R') = \sqrt{0.025^2 + 0.5^2} \approx 0.5006$: after scaling, age and salary contribute fairly.

Common Mistake

Always compute the scaling parameters $(x_{\min}, x_{\max}, \mu, \sigma)$ on the **training set only** and then apply the *same* numbers to the test set. Using the test data to compute them leaks information.

3.2 Nearest Neighbours (k-NN)

3.2.1 The Algorithm

k-NN is the simplest learning algorithm: it has **no training phase**. It just stores the training data (a *lazy learner*) and does all the work at prediction time.

Definition: k-Nearest Neighbours

To predict for a new point $\boldsymbol{x}_q$:

1. Compute the distance from $\boldsymbol{x}_q$ to every training point.
2. Pick the k training points with the smallest distances (the *neighbourhood* $N_k(\boldsymbol{x}_q)$).
3. **Classification:** output the *majority class* among the k neighbours.
   Regression: output the *mean* of their target values.

Example: k-NN classification by hand

Training data (two features, two classes) and the query $\boldsymbol{x}_q = (5, 4)$:

Point	x_1	x_2	Class	Distance to $(5,4)$	Rank
P_4	6	5	B	$\sqrt{1+1} = 1.414$	1
P_5	7	4	B	$\sqrt{4+0} = 2.000$	2–3
P_6	5	6	B	$\sqrt{0+4} = 2.000$	2–3
P_7	4	2	A	$\sqrt{1+4} = 2.236$	4
P_3	3	2	A	$\sqrt{4+4} = 2.828$	5
P_2	2	3	A	$\sqrt{9+1} = 3.162$	6
P_1	1	1	A	$\sqrt{16+9} = 5.000$	7

k	Neighbours	Prediction
1	P_4 (B)	**B**
3	P_4, P_5, P_6 = B,B,B	**B**
5	B,B,B,A,A (3 vs 2)	**B**
7	all points: 4 A vs 3 B	**A**

The prediction *changes* from B to A when k grows to 7, which shows how sensitive k-NN is to k.

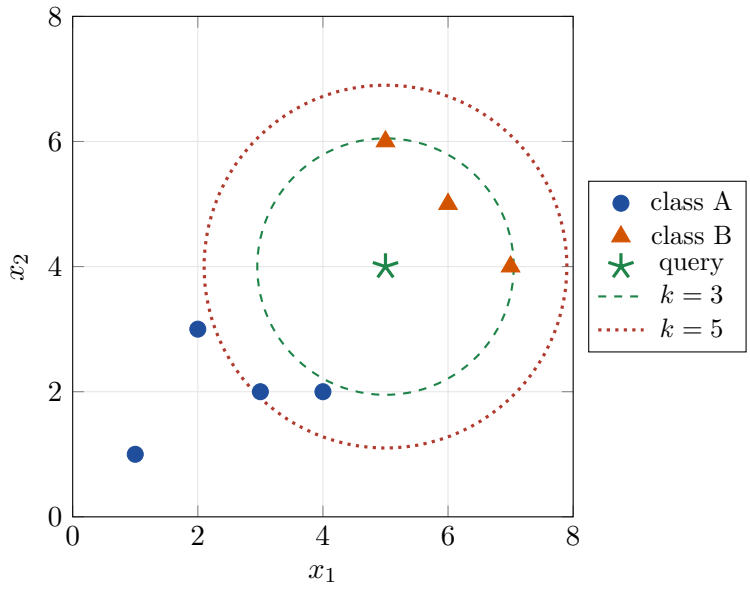


Figure 3.2. Neighbourhoods of the query point for $k = 3$ and $k = 5$.

Reading the Figure

The **green star** is the query point. Blue circles are class A and orange triangles are class B. The **dashed green circle** is just big enough to contain the 3 nearest points: all three are orange, so $k = 3$ votes B. The **dotted red circle** is large enough to contain 5 points: three orange and two blue, so $k = 5$ still votes B, but by a smaller margin. If we kept enlarging the circle to include all 7 points the blue majority would win. The *radius adapts* to the local density of data; this is why k-NN is called *local*.

Write-up for the Exam: k-Nearest Neighbours (k-NN)

Definition

k-NN is a *supervised, non-parametric, instance-based (lazy)* algorithm that predicts the output for a query point from the labels of its k closest training points: the *majority class* for classification or the *mean* value for regression.

Working

1. Choose k and a distance measure; scale the features.
2. Compute the distance from the query to every training point.
3. Select the k points with the smallest distance.
4. Classification: majority vote; regression: average of their targets (optionally weighted by $1/d$).

Assumptions

- Similar points have similar labels
- Features are on comparable scales

Advantages

- Simple; no training; handles many classes
- No assumption about the data distribution
- Improves as more data is added

Disadvantages

- Slow prediction and large memory
- Sensitive to k, scaling and irrelevant features
- Poor in high dimensions

Applications

- Recommendation systems
- Handwriting and pattern recognition
- Credit rating and anomaly detection

3.2.2 Choosing k

	Small k (e.g. 1)	**Large k**
Decision boundary	Very jagged, follows every point	Smooth
Bias	Low	High
Variance	High (sensitive to noise)	Low
Risk	Overfitting	Underfitting

k is a **hyperparameter**, chosen by cross-validation (Chapter 2). Use an *odd* k for two-class problems to avoid ties. A common starting point is $k \approx \sqrt{n}$.

Key Idea

$1/k$ is a measure of model flexibility: $k = 1$ is the *most flexible* (and most variable) k-NN; $k = n$ always predicts the overall majority class (the least flexible). This is the bias–variance trade-off of Chapter 2 in action.

3.2.3 Weighted k-NN and k-NN Regression

In **weighted k-NN**, closer neighbours get more influence, with a weight $w_i = 1/d_i$ (or $1/d_i^2$).

Example: k-NN regression

Use the house data of Chapter 1: size $x = 1, 2, 3, 4, 5$ with price $y = 15, 26, 29, 44, 46$. Predict the price at $x_q = 3.4$ with $k = 2$. The two nearest sizes are $x = 3$ (distance 0.4, price 29) and $x = 4$ (distance 0.6, price 44).

- **Plain average:** $\hat{y} = (29 + 44)/2 = 36.5$.
- **Weighted ($w = 1/d$):** $w_1 = 2.5$, $w_2 = 1.667$; $\hat{y} = \dfrac{2.5(29) + 1.667(44)}{2.5 + 1.667} = \dfrac{145.83}{4.167} = 35.0$.

The weighted answer is pulled towards the closer house (price 29). For comparison, the linear model $8 + 8x$ from Chapter 1 predicts 35.2.

3.2.4 Strengths and Weaknesses

Advantages	Disadvantages
Very simple; no training	Slow prediction: must scan all n points ($O(nd)$ per query)
Naturally handles multiple classes	Needs a lot of memory (stores everything)
No assumption about the data shape	Sensitive to feature scale and irrelevant features
Improves as more data arrives	Suffers from the *curse of dimensionality*

Curse of dimensionality. In high dimensions, points become almost equally far apart, so the idea of a "nearest" neighbour loses meaning and k-NN degrades. Remedies: feature selection and dimensionality reduction such as PCA (Chapter 8).

3.3 Decision Trees

3.3.1 Idea and Structure

A decision tree predicts by asking a sequence of simple questions about the features, like a game of "twenty questions".

Definition: Decision tree

A **decision tree** is a flowchart-like model in which each **internal node** tests one feature, each **branch** is an outcome of the test, and each **leaf** holds a prediction (a class label for *classification trees*, or a number for *regression trees*). The top node is the **root**.

The learning problem is: *which feature should we test at each node?* We choose the test that makes the resulting groups as **pure** as possible, i.e. each group dominated by one class.

3.3.2 Measures of Impurity

Let a node contain examples of C classes with proportions $p_1, \ldots, p_C$.

Definition: Entropy

$$H = -\sum_{i=1}^{C} p_i \log_2 p_i \qquad (0 \log 0 := 0).$$

Entropy is 0 when the node is pure and is maximal ($\log_2 C$) when all classes are equally likely. It is measured in *bits*.

Definition: Gini index

$$G = 1 - \sum_{i=1}^{C} p_i^2.$$

Gini is 0 for a pure node and equals 0.5 for a 50/50 two-class node. It is the probability that two items drawn at random from the node have different classes.

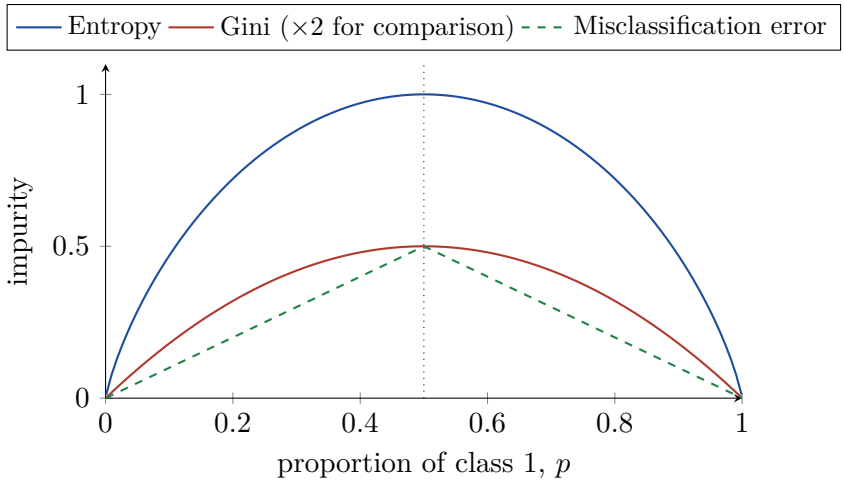


Figure 3.3. Impurity measures for a two-class node as a function of p.

Reading the Figure

The horizontal axis is the **proportion of class 1** in a node: $p = 0$ or $p = 1$ means a *pure* node and $p = 0.5$ is the most mixed. All three curves are **zero at the ends** (pure) and **highest at the middle** (maximum confusion). **Entropy** (blue) is slightly more curved than **Gini** (red; drawn here as $2p(1-p)$ so that it peaks at 1 like entropy). In practice they almost always choose the same splits. The dashed green *misclassification error* has a sharp

corner and is less sensitive to changes, which is why it is rarely used for growing trees.

Definition: Information gain

If a split on feature A divides a set S into subsets S_v (one for each value v of A), then

$$\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v).$$

It is the *reduction in entropy* achieved by the split. At each node the tree picks the feature with the **highest information gain**. (The ID3 algorithm does exactly this; C4.5 uses the *gain ratio* to avoid favouring features with many values; CART uses the Gini index and binary splits.)

3.3.3 Worked Example: Building a Tree with ID3

We want to decide whether to **play tennis** from the weather. There are 14 past days.

Day	Outlook	Temp.	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Step 1: entropy of the whole set. There are 9 Yes and 5 No:

$$H(S) = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = 0.410 + 0.530 = \mathbf{0.940}.$$

Step 2: information gain of every feature.

Split	Values (Yes, No)	Entropies	Gain
Outlook	Sunny (2,3), Overcast (4,0), Rain (3,2)	0.971, 0, 0.971	**0.247**
Humidity	High (3,4), Normal (6,1)	0.985, 0.592	0.152
Wind	Weak (6,2), Strong (3,3)	0.811, 1.000	0.048
Temperature	Hot (2,2), Mild (4,2), Cool (3,1)	1.000, 0.918, 0.811	0.029

Detail for Outlook:

$$\text{Gain} = 0.940 - \tfrac{5}{14}(0.971) - \tfrac{4}{14}(0) - \tfrac{5}{14}(0.971) = 0.940 - 0.694 = 0.247.$$

Outlook has the highest gain, so it becomes the root.

Step 3: recurse on each branch.

- **Overcast:** all 4 days are Yes ⇒ pure ⇒ leaf *Yes*.
- **Sunny** (D1, D2, D8, D9, D11; 2 Yes, 3 No): splitting on Humidity gives High → {D1, D2, D8} all No, Normal → {D9, D11} all Yes. Both pure, so Humidity is chosen.
- **Rain** (D4, D5, D6, D10, D14; 3 Yes, 2 No): splitting on Wind gives Weak → {D4, D5, D10} all Yes, Strong → {D6, D14} all No.

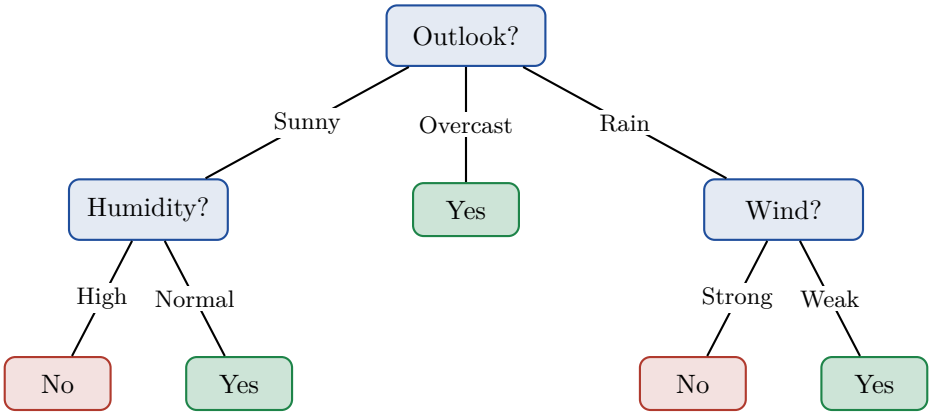


Figure 3.4. The decision tree learned by ID3 from the Play-Tennis data.

Reading the Figure

Blue boxes are decision nodes (questions), **green** and **red** boxes are leaves (final answers: Yes/No). To classify a new day, start at the *root* and follow the branch matching the day's value at each question until you reach a leaf. For example, a day that is *Rain* and *Weak wind*: Outlook = Rain → Wind? → Weak → **Yes**. The tree also reads as rules: *IF Outlook=Overcast THEN Play=Yes; IF Outlook=Sunny AND Humidity=High THEN No;* ... This is why trees are so easy to *interpret*.

The same split using the Gini index. $G(S) = 1 - (\frac{9}{14})^2 - (\frac{5}{14})^2 = 0.459$. For Outlook, $G(\text{Sunny}) = G(\text{Rain}) = 1 - (0.4)^2 - (0.6)^2 = 0.48$ and $G(\text{Overcast}) = 0$, so the weighted Gini is $\frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343$ and the Gini gain is $0.459 - 0.343 = 0.116$, again the best feature.

3.3.4 Algorithm Summary

1. Start with all training data at the root.
2. If the node is pure (or a stopping rule applies), make it a leaf labelled with the majority class.
3. Otherwise, evaluate every candidate split; choose the one with the highest gain (or largest drop in Gini).
4. Split the data and **recurse** on each child.

Continuous features. Sort the values, try thresholds halfway between consecutive values (e.g. "Age ≤ 30?"), and choose the threshold with the best gain.

Regression trees. Leaves store the *mean* target of their training points; the best split minimises the sum of squared errors (variance) in the two children.

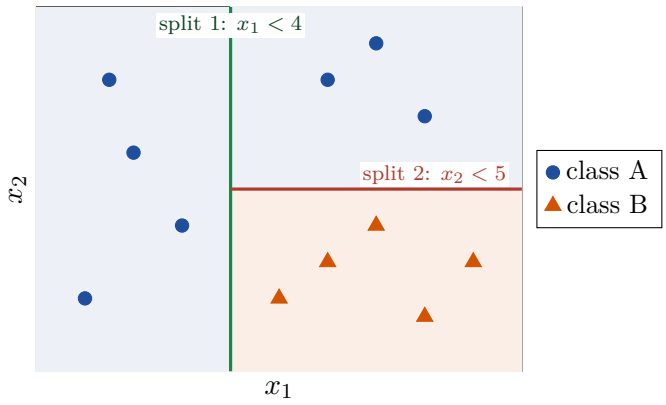


Figure 3.5. A decision tree with two splits carves the feature space into axis-parallel rectangles.

Reading the Figure

Every split in a tree tests *one feature against a threshold*, which draws a **straight line parallel to an axis**. The **green vertical line** is the root test $x_1 < 4$: everything to its left is class A. On the right, the **red horizontal line** tests $x_2 < 5$: below it is class B (orange), above it class A. The final result is three *rectangular regions*, each assigned one class. Trees therefore produce *boxy* decision boundaries, which suit data with axis-aligned structure but approximate diagonal boundaries only with many small steps.

Write-up for the Exam: Decision Tree

Definition

A **decision tree** is a *supervised, non-parametric* model that predicts by asking a sequence of feature tests arranged as a tree: internal nodes test a feature, branches are outcomes and leaves hold the prediction (class or value).

Working (ID3 / CART)

1. Start with all training data at the root.
2. For every feature (and threshold) compute the **information gain** (entropy) or **Gini reduction** of the split.
3. Split on the best feature and create child nodes.
4. Repeat recursively until nodes are pure or a stopping rule is met; label leaves by majority class (or mean).
5. Prune the tree to avoid overfitting.

Assumptions

- Classes can be separated by axis-parallel splits
- Training and test data come from the same distribution

Advantages

- Easy to understand, visualise and convert to rules
- Handles numeric and categorical features; no scaling needed
- Fast prediction

Disadvantages

- Prone to overfitting; needs pruning
- Unstable: small data changes can change the tree
- Only axis-parallel (boxy) boundaries

Applications

- Credit approval and medical diagnosis
- Customer churn prediction
- Rule extraction; base learner in Random Forests

3.3.5 Overfitting and Pruning

A tree allowed to grow until every leaf is pure will memorise the training data, noise included (low bias, **high variance**; see Chapter 2). We control complexity by **pruning**.

Pre-pruning (early stopping)	Stop growing early: limit the depth (`max_depth`), require a minimum number of samples to split (`min_samples_split`) or in a leaf (`min_samples_leaf`), or require a minimum gain.
Post-pruning	Grow the full tree, then *cut back* branches that do not improve accuracy on a validation set (reduced-error pruning) or minimise cost $=$ error $+ \alpha \cdot$ (number of leaves) (cost-complexity pruning).

3.3.6 Strengths and Weaknesses

Advantages	**Disadvantages**
Easy to understand and visualise; human-readable rules	Prone to overfitting
Handles numeric and categorical features	Unstable: small data changes can give a very different tree
No feature scaling needed	Axis-parallel (boxy) boundaries only
Fast prediction; performs feature selection implicitly	Biased towards features with many values

The instability of single trees is fixed by combining many of them into *ensembles* such as Random Forests (Chapter 5).

3.4 Naive Bayes

3.4.1 Bayes' Theorem

Definition: Bayes' theorem

For a class C and observed evidence $\boldsymbol{x}$,

$$P(C \mid \boldsymbol{x}) = \frac{P(\boldsymbol{x} \mid C)\, P(C)}{P(\boldsymbol{x})}, \qquad P(\boldsymbol{x}) = \sum_{c} P(\boldsymbol{x} \mid c) P(c).$$

Here $P(C \mid \boldsymbol{x})$ is the **posterior**, $P(\boldsymbol{x} \mid C)$ the **likelihood**, $P(C)$ the **prior** and $P(\boldsymbol{x})$ the **evidence**.

It follows directly from the definition of conditional probability, $P(C \mid \boldsymbol{x})P(\boldsymbol{x}) = P(\boldsymbol{x}, C) = P(\boldsymbol{x} \mid C)P(C)$. In words: *posterior* (what we believe after seeing the evidence) $\propto$ *likelihood* (how well the class explains the evidence) $\times$ *prior* (what we believed before).

Example: Spam and the word “offer”

Suppose 30% of all e-mail is spam, so $P(\text{spam}) = 0.3$ and $P(\text{ham}) = 0.7$. The word “offer” occurs in 60% of spam, $P(\text{offer} \mid \text{spam}) = 0.6$, but only in 10% of ham, $P(\text{offer} \mid \text{ham}) = 0.1$. An e-mail contains “offer”. How likely is it to be spam?

$$P(\text{spam} \mid \text{offer}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.1 \times 0.7} = \frac{0.18}{0.18 + 0.07} = \frac{0.18}{0.25} = \mathbf{0.72}.$$

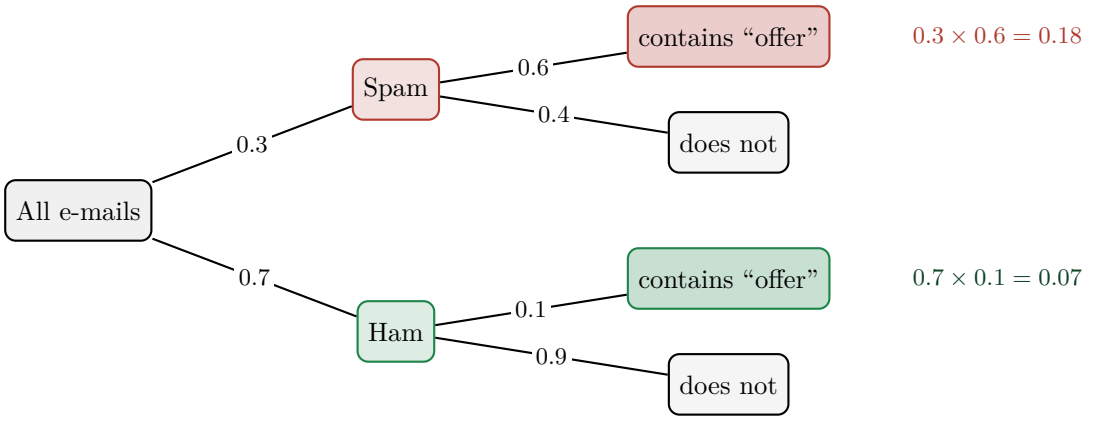


Figure 3.6. Probability tree for the spam example. Only the two “offer” leaves matter once we know the word appeared.

Reading the Figure

Read the tree **left to right**, *multiplying probabilities along a path.* First the e-mail is spam (0.3) or ham (0.7). Then, given its class, it contains “offer” with probability 0.6 (spam) or 0.1 (ham). The **two paths that end in “contains offer”** have joint probabilities 0.18 (spam) and 0.07 (ham). Because we *know* the word appeared, we only look at those two paths, and the posterior is the spam share of their total: $0.18/(0.18 + 0.07) = 0.72$. The prior of 30% has been **updated** to 72% by the evidence.

3.4.2 The Naive Bayes Classifier

With many features $\boldsymbol{x} = (x_1, \ldots, x_d)$, the likelihood $P(x_1, \ldots, x_d \mid C)$ would need an enormous number of probabilities. Naive Bayes makes one simplifying assumption.

Definition: Naive (conditional independence) assumption

Given the class, the features are **independent**:

$$P(x_1, \ldots, x_d \mid C) = \prod_{j=1}^{d} P(x_j \mid C).$$

The classification rule becomes (the denominator $P(\boldsymbol{x})$ is the same for every class, so it can be

dropped):

$$\hat{y} = \arg\max_{c} \; P(c) \prod_{j=1}^{d} P(x_j \mid c). \tag{3.1}$$

The assumption is usually *false* (in e-mail, "free" and "offer" tend to appear together), hence "naive", yet the classifier works surprisingly well, because we only need the class with the *largest* score, not exact probabilities.

Training = counting. Estimate $P(c) = N_c/N$ and $P(x_j = v \mid c) = \dfrac{\text{count}(x_j = v,\ \text{class } c)}{N_c}$ from the training data.

3.4.3 Worked Example: Play-Tennis Again

Classify the new day $\boldsymbol{x} = (\text{Sunny},\ \text{Cool},\ \text{High},\ \text{Strong})$ using the 14-day table.

Priors: $P(\text{Yes}) = \frac{9}{14}$, $P(\text{No}) = \frac{5}{14}$.

Likelihood	Given Yes (9 days)	Given No (5 days)
$P(\text{Sunny} \mid \cdot)$	2/9	3/5
$P(\text{Cool} \mid \cdot)$	3/9	1/5
$P(\text{High} \mid \cdot)$	3/9	4/5
$P(\text{Strong} \mid \cdot)$	3/9	3/5

$$\text{score}(\text{Yes}) = \tfrac{9}{14} \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} = 0.00529, \qquad \text{score}(\text{No}) = \tfrac{5}{14} \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} \cdot \tfrac{4}{5} \cdot \tfrac{3}{5} = 0.02057.$$

Decision: $0.02057 > 0.00529 \Rightarrow$ predict **No**.

Posterior probabilities (normalise): $P(\text{No} \mid \boldsymbol{x}) = \dfrac{0.02057}{0.02057 + 0.00529} = 0.795$ and $P(\text{Yes} \mid \boldsymbol{x}) = 0.205$.

Write-up for the Exam: Naive Bayes Classifier

Definition

Naive Bayes is a *supervised, probabilistic* classifier based on **Bayes' theorem** with the "naive" assumption that features are *conditionally independent* given the class: $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$.

Working

1. Estimate priors $P(c) = N_c/N$ from the training data.
2. Estimate likelihoods $P(x_j \mid c)$ by counting (or with a Gaussian density for continuous features), using Laplace smoothing.
3. For a new instance compute $P(c) \prod_j P(x_j \mid c)$ for every class.
4. Predict the class with the largest score (normalise to get posterior probabilities).

Assumptions

- Features are conditionally independent given the class
- Each feature follows the assumed distribution (Gaussian, multinomial or Bernoulli)

Advantages

- Very fast to train and predict
- Works well with little data and many features
- Excellent baseline for text classification

Disadvantages

- Independence assumption is rarely true
- Probability estimates can be poorly calibrated
- Zero-frequency problem (needs smoothing)

Applications

- Spam filtering
- Sentiment analysis
- Document and news classification
- Medical diagnosis

3.4.4 Laplace Smoothing

Common Mistake

The zero-frequency problem. If a value never occurs with a class in training (e.g. "Overcast" never occurs with "No"), then $P(\text{Overcast} \mid \text{No}) = 0$ and the *whole product becomes zero*, whatever the other features say.

Laplace (add-one) smoothing adds a pseudo-count α (usually 1) to every count:

$$P(x_j = v \mid c) = \frac{\text{count}(x_j = v, c) + \alpha}{N_c + \alpha V_j},$$

where V_j is the number of different values of feature j.

Example: Smoothing

$\text{count}(\text{Overcast}, \text{No}) = 0$, $N_{\text{No}} = 5$, and Outlook has $V = 3$ values. With $\alpha = 1$: $P(\text{Overcast} \mid \text{No}) = \dfrac{0+1}{5+3} = \dfrac{1}{8} = 0.125$ instead of 0.

3.4.5 Variants of Naive Bayes

Variant	Feature type	Model for $P(x_j \mid c)$
Gaussian	Continuous	Normal density with class-wise mean μ_{jc} and variance σ_{jc}^2
Multinomial	Counts (word counts)	Relative frequency of word in class (with smoothing)
Bernoulli	Binary (word present/absent)	Probability of presence/absence

For the **Gaussian** variant,

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

Example: Gaussian Naive Bayes with one feature

Heights: males have $\mu = 175$, $\sigma = 7$ cm; females have $\mu = 162$, $\sigma = 6$ cm; priors are equal. Classify a person of height 170 cm.

$$P(170 \mid \text{male}) = \tfrac{1}{7\sqrt{2\pi}} e^{-25/98} = 0.0570 \times 0.775 = 0.0442,$$
$$P(170 \mid \text{female}) = \tfrac{1}{6\sqrt{2\pi}} e^{-64/72} = 0.0665 \times 0.411 = 0.0273.$$

Since the priors are equal, the larger likelihood wins: **male**, with posterior $0.0442/(0.0442 + 0.0273) \approx 0.62$. (Only 62% confident because 170 is plausible for both.)

Key Idea

Numerical tip. Multiplying many small probabilities underflows to 0. Real implementations add **logarithms** instead: $\hat{y} = \arg\max_c \left[\ln P(c) + \sum_j \ln P(x_j \mid c)\right]$.

Advantages	Disadvantages
Very fast to train and predict (just counting)	Independence assumption rarely true
Works well with little data and many features	Probability estimates can be poorly calibrated
Excellent baseline for text (spam, sentiment)	Zero-frequency problem (needs smoothing)
Handles multiple classes naturally	Cannot learn interactions between features

3.5 Putting It Together: The Three Methods in Python

Try It Yourself

Compare all three on the classic *Iris* flower dataset (150 flowers, 4 measurements, 3 species).

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,
    cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

models = {
  "k-NN (k=5)": make_pipeline(StandardScaler(),
                              KNeighborsClassifier(n_neighbors=5)
                                 ),
  "Decision tree": DecisionTreeClassifier(max_depth=3,
     random_state=0),
  "Naive Bayes": GaussianNB(),
}
for name, m in models.items():
    m.fit(Xtr, ytr)
    cv = cross_val_score(m, X, y, cv=5).mean()
    print(f"{name:14s} test = {m.score(Xte, yte):.3f}  CV = {cv
       :.3f}")
```

Output:

Model	Test accuracy	5-fold CV accuracy
k-NN ($k = 5$)	0.911	0.960
Decision tree (depth 3)	0.933	0.960
Naive Bayes	0.911	0.953

All three do well here. The single test split (45 flowers) is noisy, so the *cross-validated* numbers are more reliable and show the methods are practically tied.

Effect of k (same data, scaled):

k	Train acc.	Test acc.	Comment
1	1.000	0.933	memorises training data
3	0.971	0.911	
5	0.981	0.911	
15	0.981	0.933	smooth boundary
51	0.876	0.822	**underfits**: both fall

A huge k averages over too much of the data and accuracy drops on *both* sets: underfitting, exactly as predicted.

A tiny tree you can read (`max_depth=2`, printed with `export_text`):

```
|--- petal_wid <= 0.70            -> class 0 (setosa)
|--- petal_wid >  0.70
|   |--- petal_wid <= 1.55        -> class 1 (versicolor)
|   |--- petal_wid >  1.55        -> class 2 (virginica)
```

Just *two questions about petal width* separate the three species.

3.6 Comparison of the Three Methods

	k-NN	**Decision tree**	**Naive Bayes**
Learner type	Lazy, instance-based	Eager, model-based	Eager, model-based
Training cost	None	Moderate	Very low
Prediction cost	High	Very low	Very low
Assumption	Similar points share classes	Axis-aligned splits	Features independent given class
Needs scaling?	**Yes**	No	No (Gaussian: no)
Interpretable?	Low	**High**	Moderate
Main hyperparameter	k, metric	depth, min samples	smoothing α
Typical weakness	Slow; high dimensions	Overfits; unstable	Independence assumption

Chapter Summary

- Distance measures (L_1, L_2, L_p, L_∞, Hamming, cosine) define similarity; **scale features** first (min–max or z-score, fitted on training data only).
- **k-NN** stores data and predicts by majority vote (classification) or mean (regression) of

the k closest points. Small $k \Rightarrow$ high variance; large $k \Rightarrow$ high bias.

- **Decision trees** split on the feature with highest **information gain** (entropy) or largest Gini reduction. Full trees overfit; control with **pruning**.
- Entropy $H = -\sum p_i \log_2 p_i$; Gini $= 1 - \sum p_i^2$; Gain $= H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$.
- **Naive Bayes**: $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$, assuming conditional independence; use **Laplace smoothing** and log-probabilities; variants: Gaussian, multinomial, Bernoulli.
- In the Play-Tennis example the tree root is Outlook (gain 0.247) and Naive Bayes predicts "No" with posterior 0.795 for (Sunny, Cool, High, Strong).

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define Euclidean and Manhattan distance. **[2 marks]**
2. Why is feature scaling necessary for k-NN? **[2 marks]**
3. Why is k-NN called a lazy learner? **[2 marks]**
4. What happens when $k = 1$ and when $k = n$ in k-NN? **[2 marks]**
5. Define entropy and Gini index. **[2 marks]**
6. What is information gain? **[2 marks]**
7. What is pruning? Name its two types. **[2 marks]**
8. State Bayes' theorem. **[2 marks]**
9. Why is Naive Bayes called "naive"? **[2 marks]**
10. What is Laplace smoothing and why is it needed? **[2 marks]**

Model Answer: Q4 and Q10

Q4. For $k = 1$ the prediction is the label of the single nearest point: the boundary is very jagged, bias is low, variance is high (overfitting). For $k = n$ every query gets the overall majority class: the model is a constant, bias is high and variance is zero (underfitting).
Q10. Laplace smoothing adds a pseudo-count (usually 1) to every feature-value count so that $P(x_j \mid c) = \frac{\text{count}+1}{N_c + V_j}$. It is needed to avoid zero probabilities for unseen feature–class combinations, which would force the whole product $\prod P(x_j \mid c)$ to zero.

B. Short Answer Questions (5 marks each)

1. Explain the k-NN algorithm for classification with an example. **[5 marks]**
2. Discuss the effect of the value of k on k-NN (bias, variance, over/underfitting). **[5 marks]**
3. List the advantages and disadvantages of k-NN. What is the curse of dimensionality? **[5 marks]**
4. Explain the structure of a decision tree. Differentiate entropy and Gini index. **[5 marks]**
5. Explain overfitting in decision trees. How can it be reduced? **[5 marks]**
6. Explain the Naive Bayes classifier and its independence assumption. **[5 marks]**
7. Compare k-NN, decision trees and Naive Bayes in a table. **[5 marks]**

Model Answer: B4 (outline for 5 marks)

Structure: root node (first test), internal nodes (feature tests), branches (outcomes), leaves (class label or value); classification of a new point follows the path from root to a leaf. *Entropy* $H = -\sum p_i \log_2 p_i$: measures disorder of class labels; 0 for a pure node, 1 for a 50/50 binary node. *Gini* $= 1 - \sum p_i^2$: probability of mis-classifying a random item; 0 for pure, 0.5 for 50/50.

	Entropy	**Gini**
Formula	$-\sum p_i \log_2 p_i$	$1 - \sum p_i^2$
Range (2 classes)	0 to 1	0 to 0.5
Cost	Needs logarithm	Cheaper (no log)
Used in	ID3, C4.5	CART

C. Long Answer Questions (10 marks each)

1. Explain the ID3 algorithm. Using the 14-day Play-Tennis table, compute the information gain of *Outlook* and show that it is chosen as the root. **[10 marks]**
2. Explain the Naive Bayes classifier with Laplace smoothing. Use the Play-Tennis data to classify (Sunny, Cool, High, Strong). **[10 marks]**
3. Describe the k-NN algorithm for classification and regression. Illustrate with a numerical example and discuss the choice of k and distance metric. **[10 marks]**
4. Explain distance measures used in machine learning with examples. Why must features be scaled? **[10 marks]**
5. Discuss pre-pruning and post-pruning in decision trees. How do trees compare to k-NN and Naive Bayes in terms of interpretability and cost? **[10 marks]**

Model Answer: C1 (key steps)

1. $H(S) = 0.940$ (9 Yes, 5 No). 2. Outlook: Sunny (2Y,3N) $H = 0.971$; Overcast (4Y,0N) $H = 0$; Rain (3Y,2N) $H = 0.971$. 3. Gain $= 0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) = 0.247$. 4. Gains of the others: Humidity 0.152, Wind 0.048, Temperature 0.029. Outlook is largest, so it is the root. 5. Overcast is pure (Yes); Sunny splits on Humidity; Rain splits on Wind. Draw the tree of Figure 3.4.

D. Numerical Problems

1. Find the Euclidean and Manhattan distance between $(2, 3)$ and $(6, 6)$. *(Answer: 5 and 7.)*
2. Training points: $(1, 2)$ A, $(2, 1)$ A, $(3, 3)$ A, $(4, 5)$ B, $(5, 4)$ B. Classify the query $(4, 4)$ using $k = 3$ and $k = 5$ (Euclidean). *(Distances: $1, 1, 1.414, 3.606, 3.606$. Answer: $k = 3$: B,B,A $\rightarrow$ **B**; $k = 5$: 2B vs 3A $\rightarrow$ **A**.)*
3. Distances to a query are 1 and 2 with target values 10 and 20. Give the weighted $(1/d)$ k-NN regression value for $k = 2$. *(Answer: $\frac{1 \cdot 10 + 0.5 \cdot 20}{1.5} = 13.33$.)*
4. Compute the entropy of a node with 6 positive and 2 negative examples. *(Answer: $-0.75 \log_2 0.75 - 0.25 \log_2 0.25 = 0.811$.)*
5. A node with 4 positives and 4 negatives is split into $(3+, 1-)$ and $(1+, 3-)$. Compute the information gain. *(Answer: $1 - \frac{4}{8}(0.811) - \frac{4}{8}(0.811) = 0.189$.)*
6. Compute the Gini index of a node with 3 "yes" and 1 "no". *(Answer: $1 - 0.5625 - 0.0625 = 0.375$.)*
7. $P(\text{spam}) = 0.4$, $P(\text{"free"} \mid \text{spam}) = 0.5$, $P(\text{"free"} \mid \text{ham}) = 0.05$. Find $P(\text{spam} \mid \text{"free"})$. *(Answer: $\frac{0.4 \cdot 0.5}{0.4 \cdot 0.5 + 0.6 \cdot 0.05} = \frac{0.20}{0.23} = 0.870$.)*
8. A word never appears in the 100 words of class "ham"; vocabulary size is 50. Find its smoothed probability. *(Answer: $\frac{0+1}{100+50} = 0.0067$.)*

E. Multiple-Choice Questions

1. k-NN does its main computation during:
   (a) training (b) prediction (c) pruning (d) scaling
2. Which distance is the sum of absolute coordinate differences?
   (a) Euclidean (b) Chebyshev (c) Manhattan (d) Cosine
3. The entropy of a pure node is:
   (a) 1 (b) 0.5 (c) 0 (d) ∞
4. A decision tree chooses the split with the:
   (a) lowest gain (b) highest information gain (c) most branches (d) fewest samples
5. Which method needs feature scaling the most?
   (a) Decision tree (b) Naive Bayes (c) k-NN (d) none

6. Overfitting of a decision tree is reduced by:
   (a) growing it deeper (b) pruning (c) removing the root (d) adding duplicate data

7. In Naive Bayes the features are assumed to be:
   (a) identical (b) correlated (c) conditionally independent given the class (d) Gaussian only

8. Laplace smoothing prevents:
   (a) overfitting of trees (b) zero probabilities (c) scaling errors (d) ties in k-NN

9. The Gini index of a 50/50 two-class node is:
   (a) 1 (b) 0.5 (c) 0.25 (d) 0

10. A very large k in k-NN typically causes:
    (a) overfitting (b) underfitting (c) zero error (d) no change

Answers: 1-b, 2-c, 3-c, 4-b, 5-c, 6-b, 7-c, 8-b, 9-b, 10-b.

F. Practice Problems

1. Using the Play-Tennis table, find the best feature to split on inside the *Sunny* branch using information gain, and verify that Humidity gives gain 0.971.

2. Classify (Overcast, Mild, High, Weak) with Naive Bayes using add-one smoothing.

3. **Coding:** using `sklearn`, plot training and test accuracy of k-NN on Iris for $k = 1, 3, \dots, 31$ and identify the best k by 5-fold cross-validation.

4. **Coding:** fit `DecisionTreeClassifier` with `max_depth` from 1 to 10 and plot train vs. test accuracy. At which depth does overfitting begin?

Quick Revision

Topic	One-line recall
Euclidean / Manhattan	$\sqrt{\sum (a_i - b_i)^2}$ / $\sum \lvert a_i - b_i \rvert$.
Scaling	Min–max $\frac{x - x_{\min}}{x_{\max} - x_{\min}}$; z-score $\frac{x - \mu}{\sigma}$; fit on train only.
k-NN	Lazy; majority vote / mean of k closest; small k overfits, large k underfits.
Weighted k-NN	$w_i = 1/d_i$; closer neighbours count more.
Entropy / Gini	$-\sum p \log_2 p$ / $1 - \sum p^2$; both 0 if pure.
Information gain	$H(S) - \sum \frac{\lvert S_v \rvert}{\lvert S \rvert} H(S_v)$; choose the maximum.
Pruning	Pre (max depth, min samples) / post (cut back, cost-complexity).
Tree boundary	Axis-parallel rectangles.
Bayes' theorem	Posterior = likelihood × prior / evidence.
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$; conditional independence; Laplace $\frac{n+1}{N_c + V}$.
Play-Tennis results	Root Outlook (gain 0.247); NB for (S,C,H,Str): No, $P = 0.795$.

Chapter 4

Linear Models and Support Vector Machines

Learning Objectives

After studying this chapter you will be able to:

- **define** linear regression, logistic regression, generalized linear models and support vector machines, and write each as a standard exam answer;
- fit a regression line by hand, using both the **least-squares formula** and **gradient descent**, and judge it with **MSE** and R^2;
- explain the **sigmoid**, **odds**, **log-odds** and **cross-entropy loss**, and compute logistic-regression probabilities, boundaries and accuracy end to end;
- describe the three components of a **GLM** and work with **Poisson regression**;
- explain **binary classification**, the **maximum-margin hyperplane**, **support vectors** and the **soft margin**, and solve a small SVM by hand;
- compare linear and logistic regression and say when to use each;
- answer 2-, 5- and 10-mark university questions on Unit II.

> *Simple models are not simple-minded: a good linear model often beats a clever complicated one.*
>
> — Leo Breiman

In Chapter 3 the models were built from *distances*, *questions* and *probabilities*. This chapter studies the most widely used family of models in all of statistics and machine learning: **linear models**, which predict from a *weighted sum* of the features.

Definition: Linear model

A **linear model** computes a score

$$z = \boldsymbol{w} \cdot \boldsymbol{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b,$$

where $\boldsymbol{w}$ are the **weights** (coefficients) and b is the **bias** (intercept). The final prediction is a function of z: z itself (linear regression), a probability $\sigma(z)$ (logistic regression), or the sign of z (SVM).

Model	Prediction from the score z	Output	Real-life task
Linear regression	$\hat{y} = z$	any real number	Marks from study hours
Logistic regression	$\hat{p} = \dfrac{1}{1+e^{-z}}$	probability in $(0,1)$	Will the student pass?
GLM (Poisson)	$\hat{\mu} = e^{z}$	non-negative count	Number of calls per day
SVM	$\hat{y} = \text{sign}(z)$	class -1 or $+1$	Spam or not spam

4.1 Linear Regression

Write-up for the Exam: Linear Regression

Definition

Linear regression is a *supervised, parametric* learning method for *regression* problems. It models the target y as a **linear function of the input features** plus random error, $y = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d + \varepsilon$, and chooses the parameters that minimise the sum of squared prediction errors (**method of least squares**).

Working

1. Assume the linear model $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$.
2. Define the loss $J(\theta) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2$.
3. Find θ that minimises J using the *normal equation* or *gradient descent*.
4. Predict for new inputs and evaluate with MSE, RMSE and R^2 on test data

Assumptions

- Linearity of the relationship
- Independence of errors
- Constant error variance (homoscedasticity)
- Normally distributed errors (for inference)
- No strong multicollinearity among features

Advantages

- Simple, fast, easy to interpret (each coefficient is the effect of one feature)
- Needs little data
- A good baseline

Disadvantages

- Cannot capture non-linear relationships unless features are transformed
- Sensitive to outliers
- Affected by correlated features

Applications

- House-price prediction
- Sales forecasting
- Estimating marks or salary
- Demand and risk modelling

4.1.1 Simple Linear Regression

With one feature x the model is a straight line

$$\hat{y} = \theta_0 + \theta_1 x,$$

where θ_1 is the **slope** (change in y per unit change in x) and θ_0 the **intercept** (value of y at $x = 0$). The error of a prediction is the **residual** $e_i = y_i - \hat{y}_i$. The method of **least squares** chooses θ_0, θ_1 to minimise the **sum of squared errors**

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2.$$

Setting the partial derivatives $\partial\text{SSE}/\partial\theta_0$ and $\partial\text{SSE}/\partial\theta_1$ to zero gives the **closed-form solution**

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}. \tag{4.1}$$

Real-life example: study hours and exam marks

A teacher records the hours studied x and the marks y (out of 100) of five students.

Student	A	B	C	D	E	Mean
Hours x	1	2	3	4	5	$\bar{x} = 3$
Marks y	35	45	50	65	70	$\bar{y} = 53$

Example: End-to-end solution

Step 1: deviations from the mean.

$x - \bar{x}$	-2	-1	0	1	2	
$y - \bar{y}$	-18	-8	-3	12	17	
$(x-\bar{x})(y-\bar{y})$	36	8	0	12	34	$S_{xy} = 90$
$(x-\bar{x})^2$	4	1	0	1	4	$S_{xx} = 10$

Step 2: parameters. $\theta_1 = \dfrac{90}{10} = 9$ and $\theta_0 = 53 - 9(3) = 26$. **Model:** $\widehat{\text{marks}} = 26 + 9 \times \text{hours}$.

Interpretation: each extra hour of study adds about **9 marks**; a student who studies 0 hours is expected to score 26.

Step 3: predict. For $x = 6$ hours: $\hat{y} = 26 + 9(6) = \mathbf{80}$ marks.

Step 4: check the fit.

x	1	2	3	4	5	
y	35	45	50	65	70	
$\hat{y} = 26 + 9x$	35	44	53	62	71	
Residual $y - \hat{y}$	0	1	-3	3	-1	$\sum e = 0$
e^2	0	1	9	9	1	SSE $= 20$
$(y-\bar{y})^2$	324	64	9	144	289	SST $= 830$

$$\text{MSE} = \frac{\text{SSE}}{n} = \frac{20}{5} = 4, \quad \text{RMSE} = 2, \quad R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{20}{830} = \mathbf{0.976}.$$

The line explains about **97.6%** of the variation in marks. The residuals add up to 0, a property of least squares with an intercept.

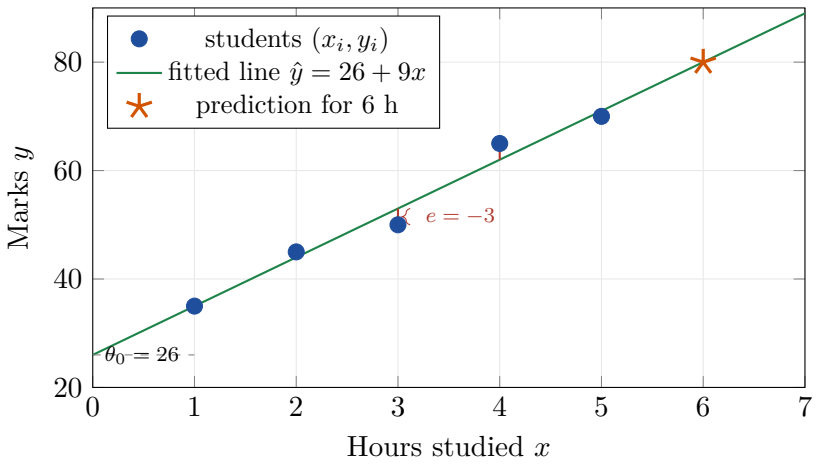


Figure 4.1. Least-squares line for study hours vs. marks. Red segments are residuals.

Reading the Figure

Each **blue dot** is a student. The **green line** is the best straight line, chosen so that the *sum of the squared lengths of the red vertical segments* is as small as possible. A red segment is a **residual**: the gap between the actual marks and the line's prediction. For student C the line predicts 53 but the actual mark is 50, so the residual is -3. The **slope** of the line is 9 (marks per hour), and where it meets the vertical axis it gives the intercept 26. The **orange star** is the prediction for a student who studies 6 hours: we read the height of the line at $x = 6$, giving 80.

4.1.2 Normal Equation (Matrix Form)

For one or many features, stack the inputs into a matrix $\mathbf{X}$ with a leading column of ones (for the intercept) and the targets into a vector $\boldsymbol{y}$. Minimising J gives the **normal equation**

$$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}. \tag{4.2}$$

Example: Same problem in matrix form

With $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}$ and $\boldsymbol{y} = (35, 45, 50, 65, 70)^\top$:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix}, \quad \mathbf{X}^\top \boldsymbol{y} = \begin{bmatrix} 265 \\ 885 \end{bmatrix}, \quad (\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{50} \begin{bmatrix} 55 & -15 \\ -15 & 5 \end{bmatrix}.$$

$$\boldsymbol{\theta} = \frac{1}{50} \begin{bmatrix} 55(265) - 15(885) \\ -15(265) + 5(885) \end{bmatrix} = \frac{1}{50} \begin{bmatrix} 1300 \\ 450 \end{bmatrix} = \begin{bmatrix} 26 \\ 9 \end{bmatrix}.$$

This is the same answer, $\theta_0 = 26$, $\theta_1 = 9$.

The normal equation is exact but requires inverting a $d \times d$ matrix (cost $\approx O(d^3)$), which is slow for thousands of features. For large problems we use an iterative method.

4.1.3 Gradient Descent

Definition: Gradient descent

Gradient descent is an iterative optimisation algorithm that minimises a differentiable loss $J(\boldsymbol{\theta})$ by repeatedly moving the parameters a small step in the direction of *steepest decrease*, i.e. opposite to the gradient:

$$\theta_j \;\leftarrow\; \theta_j - \alpha \,\frac{\partial J}{\partial \theta_j}, \qquad j = 0, 1, \ldots, d,$$

where $\alpha > 0$ is the **learning rate**. All parameters are updated *simultaneously*.

For linear regression with $J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2$:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{n}\sum_i (\hat{y}_i - y_i), \qquad \frac{\partial J}{\partial \theta_1} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)\, x_i.$$

Example: Gradient descent by hand on the marks data

Start at $\theta_0 = \theta_1 = 0$ and use $\alpha = 0.01$.

Iteration 1. Predictions are all 0, so the errors $\hat{y} - y$ are $-35, -45, -50, -65, -70$.

$$\frac{\partial J}{\partial \theta_0} = \frac{-265}{5} = -53, \qquad \frac{\partial J}{\partial \theta_1} = \frac{-35-90-150-260-350}{5} = \frac{-885}{5} = -177.$$

$$\theta_0 = 0 - 0.01(-53) = 0.53, \qquad \theta_1 = 0 - 0.01(-177) = 1.77.$$

Iteration 2. Predictions $0.53 + 1.77x$ are 2.30, 4.07, 5.84, 7.61, 9.38; errors $-32.70, -40.93, -44.16, -57.39, -60.62$.

$$\frac{\partial J}{\partial \theta_0} = -47.16, \quad \frac{\partial J}{\partial \theta_1} = -155.94 \;\Rightarrow\; \theta_0 = 1.00,\ \theta_1 = 3.33.$$

Iteration	θ_0	θ_1	Loss J
0	0	0	1487.5
1	0.53	1.77	1166.3
2	1.00	3.33	916.6
3	1.42	4.70	722.4
⋮			
many	$\to 26$	$\to 9$	$\to 2.0$

The loss falls every step and the parameters move towards the exact answer $(26, 9)$ found earlier; with $\alpha = 0.01$ about 20 000 steps are needed to reach it to many decimals (a larger α such as 0.05 is faster, but see the next figure).

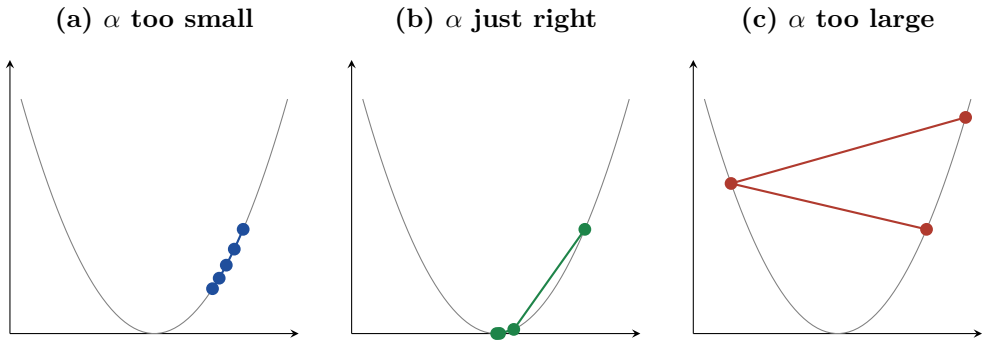


Figure 4.2. Gradient descent on the loss $J(\theta) = \theta^2$ starting from $\theta = 4$ with three learning rates.

Reading the Figure

The grey bowl is a simple loss curve $J(\theta) = \theta^2$; the lowest point (the minimum) is at $\theta = 0$. Each dot is one iteration, joined in order from the start at $\theta = 4$. The update is $\theta \leftarrow \theta(1 - 2\alpha)$.
(a) $\alpha = 0.05$ **(too small):** every step multiplies θ by 0.9: safe, but after five steps we are still far from the bottom. Training is *slow*.
(b) $\alpha = 0.4$ **(good):** each step multiplies by 0.2, and within three steps we are essentially at the minimum.
(c) $\alpha = 1.1$ **(too large):** each step multiplies by -1.2, so we *overshoot* the minimum and land higher on the opposite side each time: the loss *increases* and the algorithm **diverges**.
Lesson: if the loss goes up, reduce the learning rate.

	Normal equation	**Gradient descent**
Learning rate α	Not needed	Must be chosen
Iterations	None (one formula)	Many
Many features (d large)	Slow, $O(d^3)$	Works well
Feature scaling	Not needed	**Needed** for fast convergence
Works for other models	No (linear regression only)	Yes (logistic regression, neural networks)

4.1.4 Multiple Linear Regression

With d features, $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d = \boldsymbol{\theta}^\top \boldsymbol{x}$. Each coefficient θ_j is the change in $\hat{y}$ when x_j increases by one unit *while all other features are held fixed.*

Example: House price

A fitted model for house price (lakh Rs) is

$$\widehat{\text{price}} = 5 + 0.4\,(\text{area in m}^2) + 6\,(\text{bedrooms}) - 0.5\,(\text{age in years}).$$

For a house with 100 m^2, 3 bedrooms and 10 years old: $5 + 0.4(100) + 6(3) - 0.5(10) = 5 + 40 + 18 - 5 = \mathbf{58}$ lakh. Reading the coefficients: one extra bedroom adds 6 lakh and each year of age subtracts 0.5 lakh, other things equal.

Practical points.

- **Categorical features** (e.g. city) are converted to 0/1 *dummy variables* (one-hot encoding).
- **Scale features** before gradient descent so that all weights learn at a similar speed.
- **Multicollinearity:** if two features are strongly correlated (area in m^2 and in ft^2) the coefficients become unstable. Drop one feature or use regularisation.
- **Polynomial regression:** adding $x^2, x^3, \ldots$ as new features gives a *curved* fit, yet the model is still *linear in the parameters*, so all the theory applies. A high degree overfits (Chapter 2).

4.1.5 Evaluating a Regression Model

Metric	Formula	Meaning
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	Average squared error; punishes big errors
RMSE	$\sqrt{\text{MSE}}$	Error in the *units of* y (e.g. marks)
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Average absolute error; robust to outliers
R^2	$1 - \dfrac{\text{SSE}}{\text{SST}}$	Fraction of variance explained; 1 is perfect, 0 is no better than predicting $\bar{y}$
Adjusted R^2	$1 - (1 - R^2)\dfrac{n-1}{n-p-1}$	R^2 penalised for the number of features p

For the marks example, $R^2 = 0.976$ and adjusted $R^2 = 1 - 0.0241 \cdot \frac{4}{3} = 0.968$.

Common Mistake

A high R^2 on the *training* data does not guarantee good predictions on new data. Always compute the metrics on a held-out test set. Also remember that **correlation is not causation**: studying more is associated with higher marks, but the regression alone cannot prove it causes them.

4.1.6 Regularised Linear Regression

When there are many features, the least-squares weights can become large and overfit. **Regularisation** adds a penalty on the size of the weights:

$$\text{Ridge (L2):}\;\; J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^{d} \theta_j^2, \qquad \text{Lasso (L1):}\;\; J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^{d} |\theta_j|.$$

- **Ridge** shrinks all weights smoothly towards zero and handles correlated features well.
- **Lasso** can shrink some weights *exactly to zero*, so it performs **automatic feature selection**.
- $\lambda \geq 0$ is a hyperparameter (chosen by cross-validation): $\lambda = 0$ gives ordinary least squares; large λ gives an almost flat model (underfitting).

Try It Yourself

Reproduce the hand results with `scikit-learn`:

```
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge

hours = np.array([[1], [2], [3], [4], [5]])
marks = np.array([35, 45, 50, 65, 70])

model = LinearRegression().fit(hours, marks)
print(model.intercept_, model.coef_[0])       # 26.0  9.0
print(model.predict([[6]]))                   # [80.]
print(model.score(hours, marks))              # R^2 = 0.9759

# regularised version: weights shrink a little
print(Ridge(alpha=1.0).fit(hours, marks).coef_)
```

Challenge: add a student with 2 hours and 80 marks (an outlier). How do the slope and R^2 change?

4.2 Logistic Regression

Write-up for the Exam: Logistic Regression

Definition

Logistic regression is a *supervised, parametric* method for *classification*. It models the **probability** that an instance belongs to the positive class by passing a linear score through the **sigmoid** function, $P(y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w} \cdot \boldsymbol{x} + b) = \dfrac{1}{1 + e^{-(\boldsymbol{w} \cdot \boldsymbol{x} + b)}}$, and predicts class 1 if this probability exceeds a threshold (usually 0.5). Despite its name it is a classification algorithm.

Working

1. Compute $z = \boldsymbol{w} \cdot \boldsymbol{x} + b$.
2. Convert to a probability $p = \sigma(z)$.
3. Learn $\boldsymbol{w}, b$ by minimising the **cross-entropy (log) loss**, equivalent to *maximum likelihood*, using gradient descent.
4. Predict class 1 if $p \geq 0.5$ (that is, $z \geq 0$), else class 0

Assumptions

- Binary (or categorical) target
- Independent observations
- Log-odds is a linear function of the features
- Little multicollinearity
- Enough data per class

Advantages

- Gives *probabilities*, not only labels
- Simple, fast and interpretable (coefficients are log-odds ratios)
- Linear decision boundary
- Works well as a baseline
- Can be regularised

Disadvantages

- Only linear boundaries unless features are transformed
- May underperform on complex data
- Sensitive to outliers and correlated features
- Fails on perfectly separable data without regularisation

Applications

- Spam detection
- Disease diagnosis
- Loan default
- Customer churn
- Pass/fail prediction

4.2.1 Why Not Use Linear Regression for Classes?

Suppose we code "fail = 0, pass = 1" and fit a straight line. The line gives predictions like -0.3 or 1.4, which are *not valid probabilities*, and one extreme point can tilt the whole line. We need a function that squeezes any real number into $(0, 1)$: the **sigmoid**.

Definition: Sigmoid (logistic) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma(0) = 0.5, \qquad \sigma(-z) = 1 - \sigma(z), \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

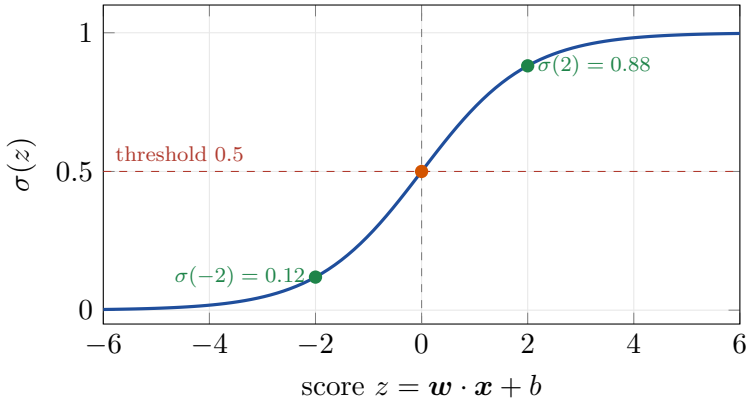


Figure 4.3. The sigmoid function maps any real score to a probability between 0 and 1.

Reading the Figure

The horizontal axis is the **linear score** z (which can be any real number) and the vertical axis is the **probability**. Large positive scores give probabilities near 1 (confident "yes"); large negative scores give probabilities near 0 (confident "no"). At $z = 0$ the curve passes through 0.5, the **decision threshold** (dashed red line): a score above 0 predicts class 1 and a score below 0 predicts class 0. The S-shape is steepest near $z = 0$, where the model is least sure, and flat at the ends, where it is almost certain. The two green dots show the symmetry $\sigma(-2) = 1 - \sigma(2)$.

4.2.2 Odds and Log-odds

Definition: Odds and logit

If an event has probability p, its **odds** are $\dfrac{p}{1-p}$ and its **log-odds (logit)** is $\ln \dfrac{p}{1-p}$.

Logistic regression says that the *log-odds is linear in the features*:

$$\ln \frac{p}{1-p} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d.$$

Hence e^{θ_j} is the **odds ratio**: the factor by which the odds are *multiplied* when x_j increases by one unit. For instance, if $p = 0.8$ the odds are $0.8/0.2 = 4$ ("4 to 1"); if $p = 0.5$ the odds are 1.

4.2.3 Loss Function: Cross-entropy from Maximum Likelihood

For one example with label $y \in \{0, 1\}$ and predicted probability $p = \sigma(z)$, the probability of the observed label is $p^y(1-p)^{1-y}$. Maximising the likelihood of all n independent examples is the same as minimising the **cross-entropy (log) loss**

$$J(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\Big]. \tag{4.3}$$

Why this loss? If $y = 1$ the loss is $-\ln p$: it is 0 when $p = 1$ and grows to ∞ as $p \to 0$, so *confident wrong answers are punished heavily.* There is no closed-form solution, so we use gradient descent. The gradient has the same neat form as in linear regression:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\, x_{ij}.$$

Example: Computing the log-loss

True y	Predicted p	Loss	Comment
1	0.80	$-\ln 0.80 = 0.223$	good
0	0.30	$-\ln 0.70 = 0.357$	fairly good
1	0.10	$-\ln 0.10 = 2.303$	confident and wrong: large loss

Average of the first two: $(0.223 + 0.357)/2 = 0.290$.

4.2.4 Real-life Example: Will the Student Pass?

Twenty students record how many hours they studied and whether they passed. Training logistic regression by gradient descent (equivalently by `scikit-learn`) gives

$$z = -4.08 + 1.50 \times \text{hours}, \qquad P(\text{pass}) = \sigma(z).$$

Example: End-to-end use of the trained model

(a) Probabilities. $P(\text{pass}) = \sigma(-4.08 + 1.50\,h)$:

Hours h	1	2	3	4	5
Score z	-2.58	-1.08	0.42	1.92	3.42
$P(\text{pass}) = \frac{1}{1+e^{-z}}$	0.070	0.254	0.604	0.872	0.968
Prediction (≥ 0.5)	Fail	Fail	Pass	Pass	Pass

For $h = 3$: $e^{-0.42} = 0.657$, so $P = \dfrac{1}{1 + 0.657} = 0.604$.

(b) Decision boundary. $P = 0.5 \iff z = 0 \iff h = \dfrac{4.08}{1.50} = \mathbf{2.72}$ hours: students who study more than about 2.7 hours are predicted to pass.

(c) Meaning of the coefficient. The odds ratio is $e^{1.50} = 4.48$: each extra hour of study multiplies the odds of passing by about **4.5**. (At $h = 2$ the odds are $0.254/0.746 = 0.34$; at $h = 3$ they are $0.604/0.396 = 1.52$; and $1.52/0.34 \approx 4.5$.)

(d) Accuracy. Using the 0.5 rule on the 20 training students: 10 students studied less than 2.72 h (predicted Fail; 8 really failed, 2 passed) and 10 studied more (predicted Pass; 8 passed, 2 failed). So

		Predicted	
		Pass	Fail
Actual	Pass	$TP = 8$	$FN = 2$
	Fail	$FP = 2$	$TN = 8$

Accuracy $= 16/20 = 0.80$, precision $= 8/10 = 0.80$, recall $= 8/10 = 0.80$ (formulas of Chapter 2).

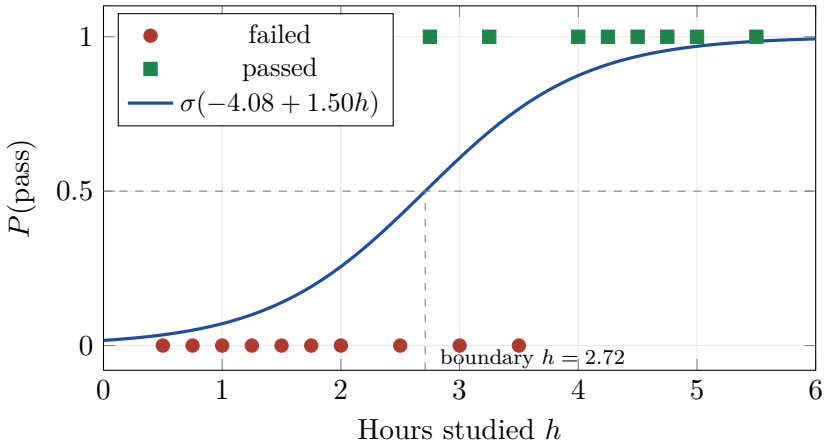


Figure 4.4. Logistic regression fitted to 20 students: the S-curve is the predicted probability of passing.

Reading the Figure

Red dots at height 0 are students who **failed**; green squares at height 1 are students who **passed**. The **blue S-curve** is the model's predicted probability of passing for any number of study hours. It rises slowly at first, climbs quickly around 2–3.5 hours and flattens near 1. Where the curve crosses the horizontal dashed line (probability 0.5), the vertical dashed line marks the **decision boundary** at about 2.7 hours. Note the overlap: some students who studied about 2–3 hours passed and others failed, so the model is naturally *uncertain* there and the curve gives a probability, not a hard yes/no. A straight regression line through these 0/1 points would go below 0 and above 1, which makes no sense for probabilities.

4.2.5 Training by Gradient Descent: One Step by Hand

Example: One gradient step of logistic regression

Five students: $x = (1, 2, 3, 4, 5)$ hours and $y = (0, 0, 1, 0, 1)$. Start with $\theta_0 = \theta_1 = 0$, so $z = 0$ and $p = \sigma(0) = 0.5$ for everyone. Use $\alpha = 0.5$.
Errors $p - y = (0.5,\ 0.5,\ -0.5,\ 0.5,\ -0.5)$.

$$\frac{\partial J}{\partial \theta_0} = \frac{0.5+0.5-0.5+0.5-0.5}{5} = 0.1, \qquad \frac{\partial J}{\partial \theta_1} = \frac{0.5(1)+0.5(2)-0.5(3)+0.5(4)-0.5(5)}{5} = \frac{-0.5}{5} = -0.1.$$

$$\theta_0 = 0 - 0.5(0.1) = -0.05, \qquad \theta_1 = 0 - 0.5(-0.1) = 0.05.$$

The new score is $z = -0.05 + 0.05x$; the 5-hour student now gets $\sigma(0.20) = 0.55$ instead of 0.50. Repeating the step many times converges to the best weights.

4.2.6 Multiclass Logistic Regression

For $K > 2$ classes there are two standard approaches.

- **One-vs-Rest (OvR):** train K binary classifiers ("class k vs. all others") and pick the class with the highest probability.
- **Softmax (multinomial) regression:** compute a score z_k for each class and convert them to probabilities that sum to 1,

$$P(y = k \mid \boldsymbol{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Example: Softmax

Scores $z = (2.0,\ 1.0,\ 0.1)$ for three classes: $e^z = (7.389,\ 2.718,\ 1.105)$, sum $= 11.212$, so the probabilities are $(0.659,\ 0.242,\ 0.099)$ and the predicted class is the first.

4.2.7 Choosing the Threshold and Regularisation

- The threshold 0.5 is not sacred. For *cancer screening* a missed patient (false negative) is very costly, so we lower the threshold (say to 0.2) to raise **recall**, accepting more false alarms. For *spam filtering* a good mail sent to spam (false positive) is costly, so we raise it to improve **precision**.
- Logistic regression can be **regularised** just like linear regression (L2 or L1 penalty). In `scikit-learn` the parameter `C` is the *inverse* of regularisation strength: smaller `C` means stronger regularisation.

	Linear regression	**Logistic regression**
Task	Regression (predict a number)	Classification (predict a class / probability)
Output	Any real value	Probability in $(0, 1)$
Function	$\hat{y} = \boldsymbol{\theta}^\top \boldsymbol{x}$	$\hat{p} = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x})$
Loss	Squared error	Cross-entropy (log loss)
Solution	Closed form or gradient descent	Gradient descent only
Metric	MSE, RMSE, R^2	Accuracy, precision, recall, F_1, log-loss
Example	Marks from study hours	Pass/fail from study hours

Try It Yourself

Reproduce the pass/fail model:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

h = np.array([.5,.75,1,1.25,1.5,1.75,1.75,2,2.25,2.5,
              2.75,3,3.25,3.5,4,4.25,4.5,4.75,5,5.5]).reshape(-1,
                  1)
p = np.array([0,0,0,0,0,0,1,0,1,0, 1,0,1,0,1,1,1,1,1,1])

# a very large C = almost no regularisation
clf = LogisticRegression(C=1e6).fit(h, p)
print(clf.intercept_, clf.coef_)            # about -4.08  1.50
print(clf.predict_proba([[3]])[0, 1])       # P(pass | 3 h) =
    0.607
print(clf.score(h, p))                      # accuracy = 0.80
```

Fitted coefficients: intercept -4.079, slope 1.505, boundary at 2.71 hours, training accuracy 0.80, matching the hand calculation. *Challenge:* predict the probabilities for 1, 2, 4 and 5 hours (answer: 0.07, 0.26, 0.88, 0.97).

4.3 Generalized Linear Models (GLMs)

Write-up for the Exam: Generalized Linear Model

Definition

A **generalized linear model (GLM)** is a flexible extension of linear regression that allows the response y to follow any distribution from the **exponential family** (normal, Bernoulli/binomial, Poisson, gamma, ...) and relates its **mean** $\mu = \mathbb{E}[y \mid \boldsymbol{x}]$ to a linear predictor through a **link function**: $g(\mu) = \boldsymbol{\theta}^\top \boldsymbol{x}$.

Three components

1. *Random component:* the distribution of y (exponential family).
2. *Systematic component:* the linear predictor $\eta = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$.
3. *Link function:* g, with $g(\mu) = \eta$, i.e. $\mu = g^{-1}(\eta)$

Working

Choose the distribution suited to the type of target, choose the matching link, and estimate $\boldsymbol{\theta}$ by **maximum likelihood** (usually by iteratively re-weighted least squares, IRLS, or gradient descent).

Advantages

- One unified framework
- Handles counts, proportions and binary data properly
- Predictions always stay in the valid range
- Coefficients remain interpretable

Disadvantages

- Still assumes a linear relationship on the link scale
- Sensitive to outliers
- The distribution must be chosen correctly (e.g. over-dispersed counts break the Poisson model)

Applications

- Insurance claim counts
- Accident/call counts
- Binary outcomes (credit default)
- Proportions
- Waiting times

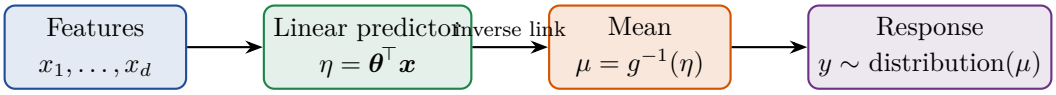


Figure 4.5. Structure of a generalized linear model.

Reading the Figure

Read left to right. The **features** are combined linearly into a score η (green box), exactly as in linear regression. The **inverse link** g^{-1} (arrow label) then converts this unbounded score into the **mean** μ of the response, forcing it into a sensible range (for example $(0, 1)$ for probabilities or $(0, \infty)$ for counts). Finally the **actual outcome** y is a random draw from a distribution (normal, Bernoulli, Poisson, ...) whose mean is μ. Changing the *link* and the *distribution* gives different models: this single picture covers linear, logistic and Poisson regression.

Model	**Distribution of y**	**Link $g(\mu)$**	**Mean μ**	**Target type**
Linear regression	Normal	identity: μ	η	real number
Logistic regression	Bernoulli	logit: $\ln \frac{\mu}{1-\mu}$	$\frac{1}{1+e^{-\eta}}$	0/1 class
Poisson regression	Poisson	log: $\ln \mu$	e^{η}	count $0, 1, 2, \ldots$
Gamma regression	Gamma	inverse: $1/\mu$	$1/\eta$	positive, skewed

Key Idea

Linear regression and logistic regression are both GLMs. Linear regression uses a Normal distribution with the identity link; logistic regression uses a Bernoulli distribution with the logit link. This is why they share the same gradient formula $\frac{1}{n}\sum(\hat{\mu}_i - y_i)x_{ij}$.

4.3.1 Real-life Example: Poisson Regression for Emergency Calls

A city fire-control room counts the number of emergency calls per day. The count depends on the rainfall x (mm): $y \sim \text{Poisson}(\mu)$ with

$$\ln \mu = 1 + 0.2\, x \qquad \Longleftrightarrow \qquad \mu = e^{\,1+0.2x}.$$

Example: Using the model

Expected calls on a dry day ($x = 0$): $\mu = e^1 = 2.72$.
On a day with 5 mm rain: $\mu = e^{1+1.0} = e^2 = 7.39$ calls.
On a day with 10 mm rain: $\mu = e^3 = 20.09$ calls.

Interpretation of the coefficient: each extra millimetre of rain multiplies the expected number of calls by $e^{0.2} = 1.22$, i.e. an increase of about **22%** per mm. Notice that the prediction is always positive, which an ordinary linear model could not guarantee.
Probability of exactly 7 calls on the 5 mm day: for a Poisson variable $P(y = k) = \frac{\mu^k e^{-\mu}}{k!}$, so $P(y = 7) = \frac{7.39^7 e^{-7.39}}{5040} \approx 0.15$.

4.4 Binary Classification and Support Vector Machines

4.4.1 Binary Classification

Definition: Binary classification

Binary classification is the supervised task of assigning each input to one of exactly **two classes**, called the **positive** and **negative** class (coded $\{0, 1\}$ in logistic regression or $\{-1, +1\}$ in SVMs). The classifier learns a **decision function** $f(\boldsymbol{x})$ and predicts the positive class when $f(\boldsymbol{x})$ exceeds a threshold.

A **linear classifier** uses $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$; the set $\{\boldsymbol{x} : \boldsymbol{w} \cdot \boldsymbol{x} + b = 0\}$ is a **hyperplane** (a line in 2-D, a plane in 3-D) that cuts the space into the two decision regions. **Examples:** spam / not spam, fraud / genuine, disease / healthy, loan default / repay. Performance is judged with the confusion matrix, precision, recall, F_1 (Chapter 2) and the **ROC curve** (true-positive rate against false-positive rate for all thresholds; the area under it, **AUC**, is 1 for a perfect classifier and 0.5 for random guessing).

Common Mistake

Many different lines can separate the same two classes perfectly. Which one is *best*? Logistic regression picks the one with the best likelihood; the **SVM** picks the one **farthest from both classes**, which tends to generalise better.

4.4.2 The Support Vector Machine (Linear, Hard Margin)

Write-up for the Exam: Support Vector Machine

Definition

A **Support Vector Machine (SVM)** is a *supervised* learning algorithm for *classification* (and regression) that finds the **maximum-margin hyperplane**: the separating hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ that is **as far as possible from the nearest training points of both classes**. The nearest points that fix the boundary are called **support vectors**.

Working

1. Label the classes $y_i \in \{-1, +1\}$.
2. Find $\boldsymbol{w}, b$ with the widest margin such that every point is on the correct side: $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$.
3. The margin width is $2/\|\boldsymbol{w}\|$, so we *minimise* $\frac{1}{2}\|\boldsymbol{w}\|^2$ subject to those constraints.
4. Classify new points by $\text{sign}(\boldsymbol{w} \cdot \boldsymbol{x} + b)$. For overlapping classes, a *soft margin* (parameter C) allows some errors; for non-linear data, *kernels* are used (Chapter 6)

Assumptions

- Data are (nearly) linearly separable in the chosen feature space
- Features should be scaled

Advantages

- Effective in high dimensions
- Only the support vectors matter, so it is memory-efficient
- Strong generalisation because of the maximum margin
- Flexible through kernels

Disadvantages

- Slow on very large datasets
- Requires feature scaling
- Sensitive to the choice of C and the kernel
- Does not directly output probabilities
- Hard to interpret with non-linear kernels

Applications

- Text and spam classification
- Image and face recognition
- Handwriting recognition
- Bioinformatics (gene and protein classification)

Geometry. The distance from a point $\boldsymbol{x}_0$ to the hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ is $\dfrac{|\boldsymbol{w} \cdot \boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$. We scale $\boldsymbol{w}, b$ so that the closest points satisfy $|\boldsymbol{w} \cdot \boldsymbol{x} + b| = 1$. Then the two *margin lines* are $\boldsymbol{w} \cdot \boldsymbol{x} + b = +1$ and $\boldsymbol{w} \cdot \boldsymbol{x} + b = -1$ and the **margin width** is

$$\frac{1}{\|\boldsymbol{w}\|} + \frac{1}{\|\boldsymbol{w}\|} = \frac{2}{\|\boldsymbol{w}\|}.$$

Making the margin wide means making $\|\boldsymbol{w}\|$ small. The **hard-margin SVM** is therefore the optimisation problem

$$\min_{\boldsymbol{w},b} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 \qquad \text{subject to} \qquad y_i\,(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \ \geq \ 1 \quad \text{for all } i. \tag{4.4}$$

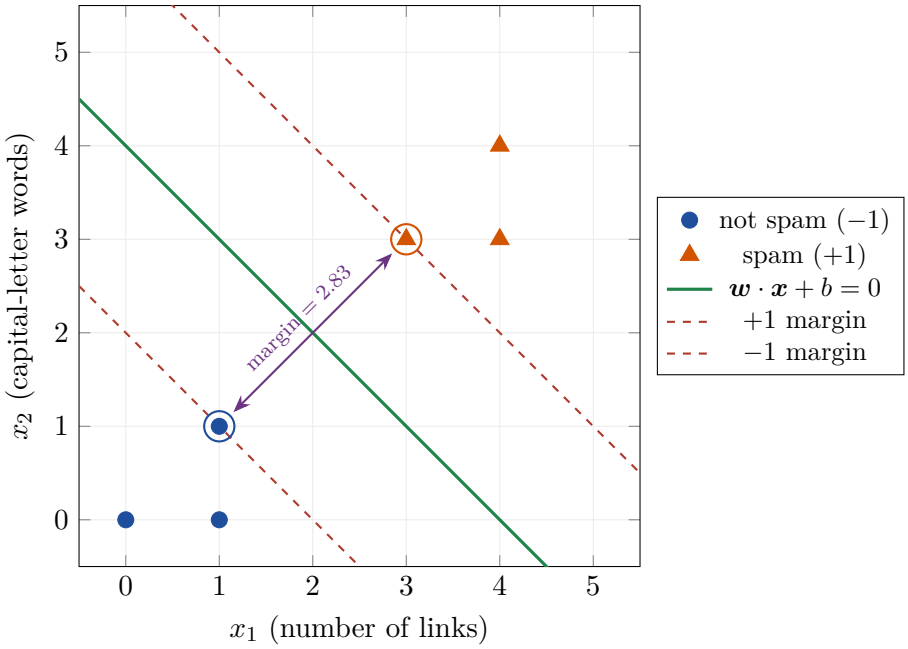


Figure 4.6. Maximum-margin separation of spam and non-spam e-mails. Circled points are the support vectors.

Reading the Figure

Each point is an e-mail described by two features. The **solid green line** is the decision boundary. The two **dashed red lines** are the **margin**: they pass through the nearest point of each class, the **circled support vectors** $(1,1)$ and $(3,3)$. The **purple arrow** is the margin width, the gap that the SVM tries to make as large as possible. Only the circled points decide where the line goes. If you moved or deleted the other four points (without crossing the margin), the boundary would *not change*. A different separating line (say, tilted) could also split the classes, but it would pass closer to some points and generalise worse.

Example: SVM by hand on the spam data

Data. Three non-spam mails (class -1): $(1,1)$, $(0,0)$, $(1,0)$. Three spam mails (class $+1$): $(3,3)$, $(4,4)$, $(4,3)$. Features: number of links x_1 and number of words in capitals x_2.
Step 1: find the closest opposite pair. The nearest points are $(1,1)$ and $(3,3)$ (distance $\sqrt{8} = 2.83$). They are the support vectors; by symmetry the boundary passes midway, perpendicular to the line joining them, so $\boldsymbol{w} = (a, a)$.
Step 2: impose the margin conditions.

$$\boldsymbol{w}\cdot(3,3)+b=+1 \ \Rightarrow \ 6a+b=1, \qquad \boldsymbol{w}\cdot(1,1)+b=-1 \ \Rightarrow \ 2a+b=-1.$$

Subtracting: $4a = 2 \Rightarrow a = 0.5$; then $b = -1 - 2(0.5) = -2$. So $\boldsymbol{w} = (0.5,\, 0.5)$, $b = -2$ and the boundary is $0.5x_1 + 0.5x_2 - 2 = 0$, i.e. $x_1 + x_2 = 4$.
Step 3: verify all six constraints $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$:

Point	$(1,1)$	$(0,0)$	$(1,0)$	$(3,3)$	$(4,4)$	$(4,3)$
$f = 0.5x_1 + 0.5x_2 - 2$	-1	-2	-1.5	1	2	1.5
$y\, f$	1	2	1.5	1	2	1.5
Satisfied?	on margin	yes	yes	on margin	yes	yes

All $y_i f(\boldsymbol{x}_i) \geq 1$, and exactly two points equal 1: they are the support vectors.
Step 4: margin. $\|\boldsymbol{w}\| = \sqrt{0.25 + 0.25} = 0.707$, so the margin is $2/0.707 = \mathbf{2.83}$.
Step 5: predict. A new mail with 4 links and 1 capital word: $f = 0.5(4) + 0.5(1) - 2 = 0.5 > 0 \Rightarrow$ **spam**. A mail with $(1, 2)$: $f = 0.5 + 1 - 2 = -0.5 < 0 \Rightarrow$ **not spam**.

4.4.3 Soft-Margin SVM

Real data usually overlap, so no hyperplane separates the classes perfectly (and noisy points would make a hard margin very narrow). The **soft-margin SVM** allows some points to violate the margin by introducing **slack variables** $\xi_i \geq 0$:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n}\xi_i \qquad \text{subject to} \qquad y_i(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \tag{4.5}$$

Here $\xi_i = 0$ for a correctly classified point outside the margin, $0 < \xi_i \leq 1$ for a point inside the margin but on the correct side, and $\xi_i > 1$ for a misclassified point. The hyperparameter C controls the trade-off:

	Large C	**Small** C
Priority	Few training errors	Wide margin
Margin	Narrow	Wide
Bias / variance	Low bias, high variance	Higher bias, lower variance
Risk	Overfitting	Underfitting

Equivalently, the soft-margin SVM minimises the **hinge loss**

$$L_{\text{hinge}} = \max\left(0,\ 1 - y\, f(\boldsymbol{x})\right),$$

which is 0 for points correctly classified *beyond* the margin and grows linearly otherwise. For example, with $y = +1$: $f = 2 \Rightarrow$ loss 0; $f = 0.3 \Rightarrow 0.7$; $f = -0.5 \Rightarrow 1.5$.

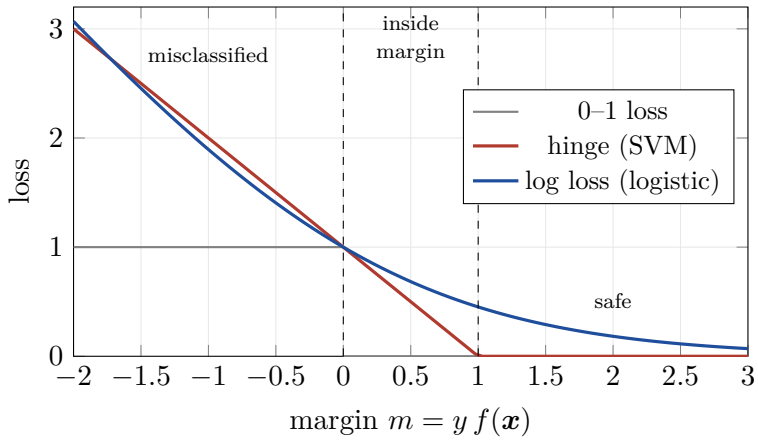


Figure 4.7. Loss as a function of the margin $m = y\,f(\boldsymbol{x})$: 0–1 loss, hinge loss (SVM) and log loss (logistic regression).

Reading the Figure

The horizontal axis is the **margin** $m = y\,f(\boldsymbol{x})$: it is positive when the point is on the *correct* side and negative when on the *wrong* side, and bigger values mean greater confidence. **Grey (0–1 loss):** counts a mistake as 1 and a correct answer as 0, but it is a cliff, so we cannot use gradients on it. **Red (hinge loss):** zero once $m \geq 1$ ("correct with room to spare"), so points beyond the margin contribute *nothing*, and grows linearly for $m < 1$, including points that are right but inside the margin. **Blue (log loss):** never exactly zero, so every point still matters a little, and it decreases smoothly. Both red and blue curves are smooth-ish *upper bounds* on the 0–1 loss that can be optimised. This single picture shows why an SVM depends only on a few support vectors, while logistic regression uses all points.

Key Idea

SVM vs logistic regression. Both learn a linear boundary. Logistic regression minimises the log loss and outputs *probabilities*; the SVM minimises the hinge loss and maximises the *margin*, depending only on the support vectors. When classes are nearly separable the SVM often generalises slightly better; when you need probabilities, use logistic regression. The *kernel trick* that lets SVMs learn *non-linear* boundaries is covered in Chapter 6.

Try It Yourself

Verify the hand-solved SVM:

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[1,1],[0,0],[1,0],[3,3],[4,4],[4,3]])    # [links, capitals]
y = np.array([-1,-1,-1, 1, 1, 1])                       # -1 ham, +1 spam
```

```
svm = SVC(kernel="linear", C=1e6).fit(X, y)    # huge C = hard
    margin
print(svm.coef_, svm.intercept_)        # [[0.5 0.5]] [-2.]
print(svm.support_vectors_)             # [[1. 1.] [3. 3.]]
print(svm.predict([[4, 1]]))            # [1]   -> spam
print(svm.decision_function([[4, 1]])) # [0.5]
```

The output reproduces $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$, the two support vectors and $f(4,1) = 0.5$. *Challenge:* set `C=0.01` and watch what happens to the margin and the support vectors.

4.5 Choosing Among the Methods of Unit II

	k-NN	**Decision tree**	**Naive Bayes**	**Linear / Logistic / SVM**
Boundary	Irregular, local	Axis-parallel boxes	Smooth (probabilistic)	Linear hyperplane
Interpretable?	Low	High	Moderate	High (coefficients)
Needs scaling?	Yes	No	No	Yes
Training speed	None	Fast	Very fast	Fast–moderate
Outputs probabilities?	Roughly	Roughly	Yes	Logistic: yes; SVM: no
Good when	Small data, local patterns	Mixed features, rules needed	Text, little data	Many features, linear structure

Chapter Summary

- **Linear regression:** $\hat{y} = \boldsymbol{\theta}^\top \boldsymbol{x}$, fitted by least squares: $\theta_1 = S_{xy}/S_{xx}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$, or $\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}$, or gradient descent $\theta_j \leftarrow \theta_j - \alpha\, \partial J / \partial \theta_j$. Evaluate with MSE, RMSE, R^2. Ridge (L2) and Lasso (L1) regularise.
- **Logistic regression:** $P(y = 1) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x})$; log-odds is linear; trained by minimising cross-entropy (maximum likelihood); predicts class 1 if $p \geq 0.5$; e^{θ_j} is the odds ratio. Softmax extends it to many classes.
- **GLM:** distribution from the exponential family + linear predictor + link function $g(\mu) = \eta$. Linear (identity), logistic (logit) and Poisson (log) regression are special cases.
- **Binary classification:** two classes, decision function and hyperplane; evaluate with confusion matrix, precision, recall, F_1, ROC–AUC.
- **SVM:** maximum-margin hyperplane; margin $= 2/\|\boldsymbol{w}\|$; minimise $\frac{1}{2}\|\boldsymbol{w}\|^2$ s.t. $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$; only **support vectors** matter; soft margin adds slack and C; hinge loss

$\max(0, 1 - yf)$.

- Worked examples: marks $= 26 + 9 \times$hours ($R^2 = 0.976$); pass probability $\sigma(-4.08 + 1.5h)$ with boundary 2.72 h; Poisson calls $e^{1+0.2x}$; SVM $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$, margin 2.83.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define linear regression. **[2 marks]**
2. What is a residual? What does the method of least squares minimise? **[2 marks]**
3. Write the gradient-descent update rule. What is the role of the learning rate? **[2 marks]**
4. Define R^2. What does $R^2 = 0.9$ mean? **[2 marks]**
5. Differentiate Ridge and Lasso regression. **[2 marks]**
6. Define the sigmoid function. What is its range? **[2 marks]**
7. Define odds and log-odds. **[2 marks]**
8. Why is logistic regression a classification algorithm even though it has "regression" in its name? **[2 marks]**
9. Define a generalized linear model. What is a link function? **[2 marks]**
10. What are support vectors? What is the margin of an SVM? **[2 marks]**
11. What is the role of the parameter C in a soft-margin SVM? **[2 marks]**
12. Define hinge loss. **[2 marks]**

Model Answer: Q2, Q7 and Q10

Q2. A *residual* is the difference between the observed and predicted value, $e_i = y_i - \hat{y}_i$. Least squares chooses the parameters that minimise the *sum of squared residuals* $\sum e_i^2$.
Q7. If an event has probability p, its *odds* are $p/(1-p)$ and its *log-odds* (logit) is $\ln[p/(1-p)]$. Logistic regression models the log-odds as a linear function of the features.
Q10. *Support vectors* are the training points that lie exactly on the margin boundaries (or inside the margin) and that alone determine the position of the separating hyperplane. The *margin* is the distance between the two margin lines, $2/\|\boldsymbol{w}\|$.

B. Short Answer Questions (5 marks each)

1. Explain simple linear regression and derive the least-squares estimates of θ_0 and θ_1. **[5 marks]**

2. Explain gradient descent for linear regression. Discuss the effect of the learning rate. **[5 marks]**
3. Compare the normal equation and gradient descent. **[5 marks]**
4. State the assumptions of linear regression. Explain R^2 and adjusted R^2. **[5 marks]**
5. Explain logistic regression with the sigmoid function. Why is it preferred to linear regression for classification? **[5 marks]**
6. Explain the cross-entropy loss used in logistic regression. **[5 marks]**
7. Explain generalized linear models with the three components and examples. **[5 marks]**
8. Explain the maximum-margin classifier and support vectors with a diagram. **[5 marks]**
9. Differentiate between linear regression and logistic regression. **[5 marks]**
10. Differentiate between hard-margin and soft-margin SVM. **[5 marks]**

Model Answer: B5 (outline for 5 marks)

Definition: logistic regression models $P(y = 1 \mid \boldsymbol{x}) = \sigma(z) = 1/(1 + e^{-z})$ with $z = \boldsymbol{w} \cdot \boldsymbol{x} + b$; it predicts class 1 if $P \geq 0.5$.
Sigmoid: maps $(-\infty, \infty) \to (0, 1)$; $\sigma(0) = 0.5$; S-shaped (draw the curve).
Why not linear regression? (i) a line can give outputs < 0 or > 1, not valid probabilities; (ii) squared error is a poor loss for 0/1 targets; (iii) outliers can shift the line and change predictions; (iv) errors are not normal or of constant variance.
Training: minimise cross-entropy $J = -\frac{1}{n} \sum [y \ln p + (1 - y) \ln(1 - p)]$ by gradient descent.
Example: pass/fail from study hours.

C. Long Answer Questions (10 marks each)

1. Explain linear regression in detail: model, least-squares method, normal equation, gradient descent, assumptions and evaluation metrics. Fit a line to the data $(1, 2), (2, 4), (3, 5), (4, 4), (5, 5)$. **[10 marks]**
2. Explain logistic regression: sigmoid, odds and log-odds, decision boundary, cross-entropy loss and gradient-descent training. Explain how it is extended to multiple classes. **[10 marks]**
3. Explain generalized linear models. Show that linear and logistic regression are special cases. Explain Poisson regression with an example. **[10 marks]**
4. Explain the support vector machine: maximum margin, support vectors, hard-margin optimisation problem, soft margin and the role of C. Solve a small numerical example. **[10 marks]**
5. Compare linear regression, logistic regression and SVM with respect to task, loss function, output, advantages and disadvantages. **[10 marks]**

6. Explain overfitting in linear models and how regularisation (Ridge and Lasso) controls it. **[10 marks]**

Model Answer: C1 (numerical part)

$x = (1,2,3,4,5)$, $y = (2,4,5,4,5)$. $\bar{x} = 3$, $\bar{y} = 4$.
$(x-\bar{x})(y-\bar{y}) = (-2)(-2)+(-1)(0)+(0)(1)+(1)(0)+(2)(1) = 4+0+0+0+2 = 6 = S_{xy}$;
$S_{xx} = 4+1+0+1+4 = 10$.
$\theta_1 = 6/10 = 0.6$, $\theta_0 = 4 - 0.6(3) = 2.2$. **Model:** $\hat{y} = 2.2 + 0.6x$. Prediction for $x = 6$: $2.2 + 3.6 = 5.8$.
Residuals: $\hat{y} = (2.8, 3.4, 4.0, 4.6, 5.2)$, $e = (-0.8, 0.6, 1, -0.6, -0.2)$, SSE $= 0.64 + 0.36 + 1 + 0.36 + 0.04 = 2.4$, SST $= 4+0+1+0+1 = 6$, $R^2 = 1 - 2.4/6 = \mathbf{0.60}$.

D. Numerical Problems

1. Fit a line to $x = (2,4,6,8)$, $y = (3,7,5,10)$ and predict y at $x = 10$. *(Answer: $\bar{x} = 5$, $\bar{y} = 6.25$, $S_{xy} = 19$, $S_{xx} = 20$, $\theta_1 = 0.95$, $\theta_0 = 1.5$; $\hat{y}(10) = 11$.)*

2. For problem 1 compute SSE and R^2. *(Answer: $\hat{y} = 3.4, 5.3, 7.2, 9.1$; residuals $-0.4, 1.7, -2.2, 0.9$; SSE $= 8.70$; SST $= 26.75$; $R^2 = 0.675$.)*

3. One gradient-descent step for $\hat{y} = \theta_1 x$ on $(1,2),(2,4)$ with $\theta_1 = 0$, $\alpha = 0.1$, $J = \frac{1}{2n}\sum(\hat{y}-y)^2$. *(Answer: gradient $= \frac{1}{n}\sum(\hat{y}-y)x = \frac{(-2)(1)+(-4)(2)}{2} = -5$; $\theta_1 = 0 - 0.1(-5) = 0.5$.)*

4. Evaluate $\sigma(0)$, $\sigma(2)$ and $\sigma(-2)$. *(Answer: 0.5, 0.881, 0.119.)*

5. A logistic model has $z = -6 + 2x$. Find $P(y=1)$ at $x = 3$ and at $x = 4$, the decision boundary and the odds ratio per unit of x. *(Answer: $\sigma(0) = 0.5$; $\sigma(2) = 0.881$; boundary $x = 3$; $e^2 = 7.39$.)*

6. Compute the log-loss for $(y = 1, p = 0.8)$ and $(y = 0, p = 0.3)$ and the average. *(Answer: 0.223, 0.357; average 0.290.)*

7. One gradient step of logistic regression on $(x,y) = (0,0),(1,1)$ from $\theta_0 = \theta_1 = 0$ with $\alpha = 1$. *(Answer: errors $0.5, -0.5$; $\partial J/\partial\theta_0 = 0$; $\partial J/\partial\theta_1 = \frac{0+(-0.5)(1)}{2} = -0.25$; new $\theta = (0, 0.25)$.)*

8. Softmax of scores $(1,2,3)$. *(Answer: $e^z = 2.718, 7.389, 20.086$, sum $= 30.193$, probabilities $0.090, 0.245, 0.665$.)*

9. A Poisson model has $\ln\mu = 0.2 + 0.5x$. Find the expected count at $x = 2$ and the rate ratio per unit of x. *(Answer: $\mu = e^{1.2} = 3.32$; ratio $e^{0.5} = 1.65$.)*

10. An SVM has $\boldsymbol{w} = (1,1)$, $b = -3$. Classify $(2,2)$ and $(1,1)$, and find the margin. *(Answer: $f(2,2) = 1 \Rightarrow +1$ (on margin); $f(1,1) = -1 \Rightarrow -1$ (on margin); margin $= 2/\sqrt{2} = 1.41$.)*

11. Compute the hinge loss for $(y = +1, f = 0.3)$, $(y = -1, f = -2)$ and $(y = +1, f = -0.5)$. *(Answer: 0.7, 0, 1.5.)*

E. Multiple-Choice Questions

1. Least squares minimises:
   (a) sum of residuals (b) sum of squared residuals (c) largest residual (d) sum of y
2. In gradient descent, a learning rate that is too large may cause:
   (a) slow learning (b) divergence (c) exact solution (d) underfitting
3. The range of the sigmoid function is:
   (a) $(-1, 1)$ (b) $(0, \infty)$ (c) $(0, 1)$ (d) $(-\infty, \infty)$
4. Logistic regression is used for:
   (a) clustering (b) classification (c) dimensionality reduction (d) only regression
5. If $p = 0.8$, the odds are:
   (a) 0.8 (b) 0.25 (c) 4 (d) 1.25
6. The link function of logistic regression is:
   (a) identity (b) log (c) logit (d) inverse
7. Poisson regression is suitable for:
   (a) binary data (b) count data (c) negative values (d) text
8. The margin of an SVM is:
   (a) $\|\boldsymbol{w}\|$ (b) $1/\|\boldsymbol{w}\|$ (c) $2/\|\boldsymbol{w}\|$ (d) $2\|\boldsymbol{w}\|$
9. Lasso regularisation can:
   (a) make weights exactly zero (b) increase variance (c) remove the intercept (d) only increase weights
10. A large value of C in a soft-margin SVM leads to:
    (a) wide margin, many errors (b) narrow margin, few errors (c) no support vectors (d) a non-linear boundary
11. Ridge regression adds a penalty proportional to:
    (a) $\sum |\theta_j|$ (b) $\sum \theta_j^2$ (c) $\max \theta_j$ (d) number of features
12. Which method gives probabilities directly?
    (a) SVM (b) logistic regression (c) hinge loss (d) least squares

Answers: 1-b, 2-b, 3-c, 4-b, 5-c, 6-c, 7-b, 8-c, 9-a, 10-b, 11-b, 12-b.

F. Practice Problems

1. Collect data on 8 friends: hours of sleep and score in a short test. Fit a line by hand and by `scikit-learn` and compare.
2. Using the pass/fail model, find the number of study hours that gives a pass probability of 0.9. *(Hint: solve* $-4.08 + 1.5h = \ln(0.9/0.1) = 2.197$*; answer* $h \approx 4.2$*.)*
3. **Coding:** implement gradient descent for linear regression from scratch with `numpy` and plot the loss against iterations for $\alpha = 0.001, 0.01, 0.05$.

4. **Coding:** train `LogisticRegression` and a linear `SVC` on the *breast cancer* dataset (`load_breast_cancer`), scale the features, and compare accuracy, precision and recall.

5. Draw a labelled diagram of an SVM with the margin, hyperplane and support vectors, and mark where $w \cdot x + b = +1, 0, -1$.

Quick Revision

Topic	One-line recall
Linear regression	$\hat{y} = \theta_0 + \theta_1 x$; minimise SSE; $\theta_1 = \frac{S_{xy}}{S_{xx}}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$.
Normal equation	$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}$; exact but $O(d^3)$.
Gradient descent	$\theta \leftarrow \theta - \alpha\, \partial J / \partial \theta$; too small $\rightarrow$ slow, too large $\rightarrow$ diverges.
Metrics	MSE, RMSE, MAE, $R^2 = 1 - \text{SSE/SST}$, adjusted R^2.
Ridge / Lasso	L2 shrinks weights; L1 makes some exactly 0 (feature selection).
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$; range $(0, 1)$; $\sigma(0) = 0.5$.
Logistic regression	$\ln \frac{p}{1-p} = \boldsymbol{\theta}^\top \boldsymbol{x}$; loss $= -\frac{1}{n} \sum [y \ln p + (1-y) \ln(1-p)]$; odds ratio e^{θ_j}.
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$ for many classes.
GLM	distribution + linear predictor + link; identity / logit / log.
SVM	max margin $2/\|w\|$; min $\frac{1}{2}\|w\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$; support vectors.
Soft margin	slack ξ_i; large C: narrow margin, overfit; small C: wide margin.
Hinge loss	$\max(0, 1 - yf)$; zero beyond the margin.

Chapter 5

Ensemble Learning

Learning Objectives

After studying this chapter you will be able to:

- **define** ensemble learning and explain *why* combining models improves accuracy;
- compute the accuracy of a **majority vote** and explain the role of **diversity**;
- explain **hard** and **soft voting** classifiers and work out an example;
- explain **bagging** and **pasting**, the **bootstrap** and **out-of-bag (OOB)** evaluation;
- describe how a **Random Forest** works and read **feature importances**;
- run **AdaBoost** by hand and explain **gradient boosting**;
- explain **stacking** with a meta-learner;
- compare bagging, boosting and stacking in terms of **bias and variance**;
- answer 2-, 5- and 10-mark university questions on Unit III (ensembles).

> *At a county fair, 800 people guessed the weight of an ox. No one guessed it exactly, but the* average *of all guesses was off by less than one percent.*
>
> — Francis Galton, 1907 (the "wisdom of crowds")

In Chapters 3 and 4 we studied *single* models. This chapter asks a simple question:

If one model is good, are many models better?

The answer is usually **yes**, provided the models are *different from each other*. Ensembles are behind most winning solutions in machine-learning competitions and many real systems.

5.1 Introduction to Ensemble Learning

5.1.1 The Idea: Wisdom of the Crowd

Think of a **hospital**. For a difficult case, one doctor might make a mistake, but if five independent specialists examine the patient and the *majority* agree on a diagnosis, you trust it much more. Or think of a **jury**: twelve people decide together rather than one judge alone.

Definition: Ensemble learning

Ensemble learning combines the predictions of *several models* (called **base learners**, **weak learners** or **predictors**) to produce one final prediction that is usually *more accurate and more stable* than the prediction of any single model.

Definition: Weak learner and strong learner

A **weak learner** is a model only slightly better than random guessing (for two classes, accuracy a little above 50%), for example a very shallow decision tree. A **strong learner** is a model with high accuracy. A key theoretical result says that many weak learners can be combined into a strong learner.

5.1.2 Why Does It Work? A Simple Calculation

Suppose we have n classifiers, each correct with probability $p > 0.5$, and *assume their mistakes are independent*. If we take the **majority vote**, the ensemble is correct whenever more than half of the classifiers are correct. For n classifiers (odd n) this probability is given by the binomial distribution:

$$P(\text{ensemble correct}) = \sum_{k=\lceil (n+1)/2 \rceil}^{n} \binom{n}{k} p^k (1-p)^{n-k}.$$

Example: Three classifiers, each 70% accurate

The majority is correct when at least 2 of the 3 are correct.

Number correct	Probability	Value
All 3 correct	0.7^3	0.343
Exactly 2 correct	$3 \times 0.7^2 \times 0.3$	0.441
At least 2 correct		**0.784**

So three 70% classifiers give a 78.4% ensemble. With **five** classifiers:

$$P = \binom{5}{3} 0.7^3 0.3^2 + \binom{5}{4} 0.7^4 0.3 + 0.7^5 = 0.3087 + 0.3602 + 0.1681 = \mathbf{0.837}.$$

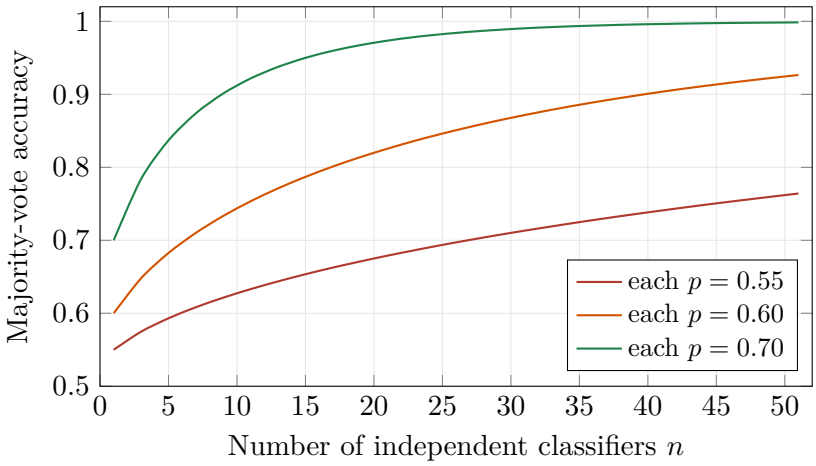


Figure 5.1. Accuracy of a majority vote of n *independent* classifiers, each with accuracy p.

Reading the Figure

The horizontal axis is the **number of classifiers** voting and the vertical axis is the **accuracy of the majority vote**. Each curve is for a different accuracy of the *individual* classifiers (red 55%, orange 60%, green 70%).

Look at the **green curve**: with one classifier the accuracy is 70%; with 5 it is 83.7%; with 21 it is 97.4%; with 51 it is almost 100%. Even the **red curve**, built from classifiers that are barely better than guessing (55%), climbs steadily to about 76% with 51 voters.

Important warning: this picture assumes the classifiers make *independent* errors. In practice they are trained on the same data and their errors are *correlated*, so real gains are smaller. This is why we work hard to make the members **diverse**.

Key Idea

Ensembles work best when the members are (i) each better than random guessing and (ii) as different from each other as possible. Diversity can come from different *algorithms*, different *training samples*, different *features*, or different *hyperparameters*.

5.1.3 Types of Ensemble Methods

Reading the Figure

Read the tree from top to bottom. The three coloured boxes are the three **strategies**.

- **Parallel** (green): the models are trained *independently* (possibly on different samples of the data) and their outputs are combined by voting or averaging. The aim is to reduce **variance**.
- **Sequential** (orange): models are trained *one after another*, and each new model concentrates on the mistakes of the earlier ones. The aim is to reduce **bias**.

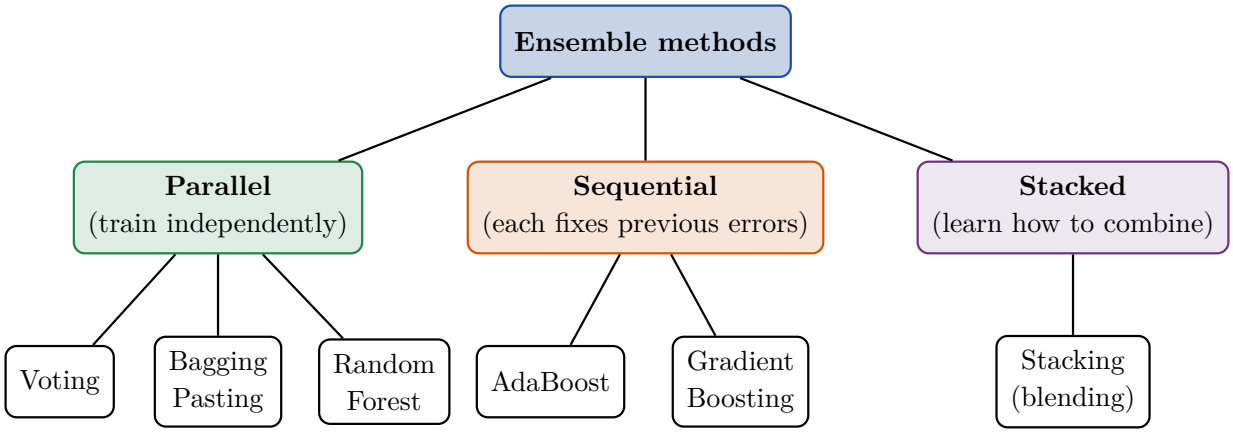


Figure 5.2. Families of ensemble methods covered in this chapter.

- **Stacked** (purple): a second-level model *learns* the best way to combine the first-level models.

Write-up for the Exam: Ensemble Learning

Definition

Ensemble learning is a machine-learning technique that trains *multiple base models* and **combines their predictions** (by voting, averaging or a learned combiner) to obtain better accuracy and robustness than any single model.

Working

1. Create several base learners, making them *diverse* (different algorithms, data samples or features).
2. Train each base learner.
3. Combine the predictions: majority/weighted vote for classification, average for regression, or a meta-model.
4. Evaluate the ensemble on unseen test data.

Assumptions

- Each base learner is better than random guessing
- The learners make *different* (ideally independent) errors

Advantages

- Higher accuracy than a single model
- Reduces overfitting (lower variance) and is more robust to noise
- Works with almost any base algorithm

Disadvantages

- More computation, memory and prediction time

- Harder to interpret than a single model
- Gains are small if the members are too similar

Applications

- Credit scoring and fraud detection
- Medical diagnosis and recommender systems
- Winning solutions in data-science competitions

5.2 Voting Classifiers

Write-up for the Exam: Voting Classifier

Definition

A **voting classifier** combines several *different* classifiers trained on the *same data*. In **hard voting** the final class is the one predicted by the *majority* of the classifiers; in **soft voting** the predicted class *probabilities* are averaged and the class with the highest average probability wins.

Working

1. Train several diverse classifiers (e.g. logistic regression, SVM, random forest) on the same training set.
2. **Hard voting:** each classifier predicts a class; count the votes; choose the most frequent class.
3. **Soft voting:** each classifier outputs $P(\text{class})$; compute $\bar{P}(c) = \frac{1}{m}\sum_j P_j(c)$ (optionally weighted); choose $\arg\max_c \bar{P}(c)$.

Assumptions

- Classifiers are reasonably accurate and diverse
- (Soft voting) every classifier can output well-calibrated probabilities

Advantages

- Very simple to build and often beats the best single member
- Soft voting uses the classifiers' *confidence*

Disadvantages

- Poor members or highly similar members add little
- Needs several trained models, hence more cost

Applications

- Combining different algorithms for text, image or tabular classification

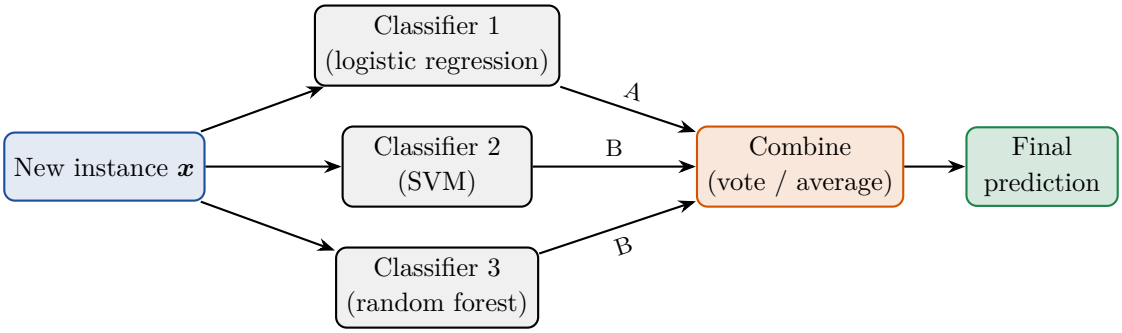


Figure 5.3. A voting ensemble: three classifiers vote on the same instance.

Reading the Figure

Follow the arrows from left to right. The **same instance $\boldsymbol{x}$** is given to three *different* classifiers. Each returns its own answer (labels A, B, B on the arrows). The orange box combines them: in **hard voting** it counts the votes (two B's against one A), so the **final prediction is B**. Notice that classifier 1 made a "mistake" but the ensemble still reached the answer the majority agreed on.

5.2.1 Hard vs. Soft Voting: Worked Example

Example: Which class is the new e-mail?

Three classifiers estimate the probability that an e-mail is **spam** (class A); the other class is not-spam (B).

Classifier	$P(\text{A})$	$P(\text{B})$	**Own prediction**
1 (logistic regression)	0.90	0.10	A
2 (SVM)	0.40	0.60	B
3 (random forest)	0.40	0.60	B

Hard voting: votes are A, B, B $\Rightarrow$ **B** wins, 2 to 1.

Soft voting: average probability of A is

$$\bar{P}(A) = \frac{0.90 + 0.40 + 0.40}{3} = \frac{1.70}{3} = 0.567, \qquad \bar{P}(B) = 0.433 \;\Rightarrow\; \textbf{A wins}.$$

Why the difference? Classifier 1 is *very confident* (90%) while the other two are only *mildly* sure (60%). Soft voting lets confident classifiers count for more.

Weighted soft voting: if we trust classifier 1 three times as much (weights $3, 1, 1$):

$$\bar{P}(A) = \frac{3(0.90) + 1(0.40) + 1(0.40)}{3 + 1 + 1} = \frac{3.5}{5} = 0.70 \;\Rightarrow\; \textbf{A}.$$

For **regression**, the ensemble simply **averages** the predictions: $\hat{y} = \frac{1}{m}\sum_{j=1}^{m}\hat{y}_j$.

	Hard voting	**Soft voting**
Uses	Predicted class labels	Predicted probabilities
Rule	Majority vote	Highest average probability
Needs probabilities?	No	Yes
Accuracy	Good	Usually slightly better

5.3 Bagging and Pasting

Voting uses *different algorithms* on the same data. Another way to create diversity is to use the **same algorithm** but train each copy on a **different random subset of the training data**.

Definition: Bagging and pasting

Both methods train B copies of the same base learner, each on a random sample of the training set, and combine them by voting (classification) or averaging (regression).

- **Bagging** (*bootstrap aggregating*): samples are drawn **with replacement**.
- **Pasting**: samples are drawn **without replacement**.

5.3.1 The Bootstrap

Definition: Bootstrap sample

A **bootstrap sample** of a dataset of size n is obtained by drawing n instances *at random with replacement*. Some instances appear several times and some do not appear at all.

Example: Drawing one bootstrap sample

Original data: instances $\{1, 2, 3, 4, 5\}$ ($n = 5$). One bootstrap sample might be

$$\{2,\ 2,\ 5,\ 1,\ 5\}.$$

Instance 2 and 5 appear twice, instance 1 once, and instances **3 and 4 are missing**. These missing instances are the **out-of-bag (OOB)** instances for this sample; they can be used to test the model trained on the sample.

How many instances are left out? The probability that a particular instance is *not* chosen in one draw is $1 - \frac{1}{n}$; over n draws it is $(1 - \frac{1}{n})^n \to e^{-1} \approx 0.368$ for large n. Hence

- about **63.2%** of the instances appear in each bootstrap sample (at least once);
- about **36.8%** are **out-of-bag**.

(For $n = 100$: $0.99^{100} = 0.366$; a simulation with $n = 1000$ gave 62.2% unique instances.)

5.3.2 Algorithm and Aggregation

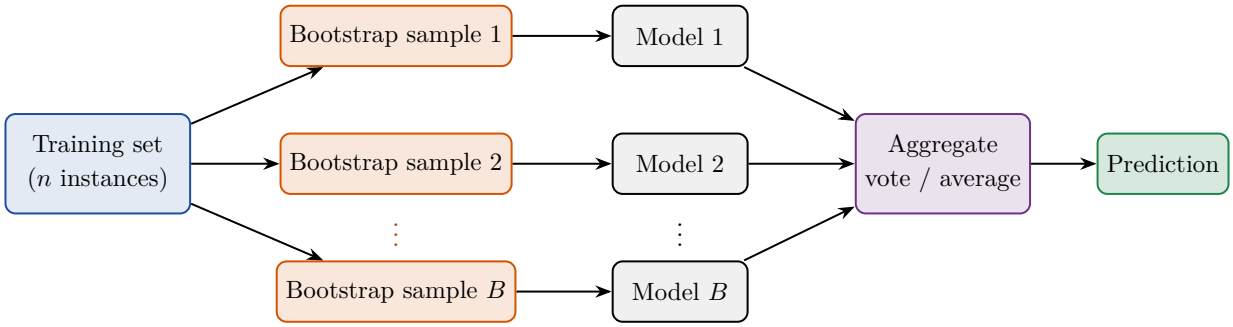


Figure 5.4. The bagging procedure.

Reading the Figure

Start at the blue box on the left. (1) From the training set we draw B different **bootstrap samples** (orange). (2) We train one model on each sample (grey); because the samples differ, the models differ. (3) For a new instance, every model makes a prediction and the purple box **aggregates** them: the **most frequent class** for classification or the **average** for regression. The models can be trained at the same time (in parallel).

Why does bagging reduce variance? Suppose each of B models has prediction variance σ^2 and any two models have correlation ρ. The variance of the average is

$$\mathrm{Var}(\bar{f}) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{5.1}$$

Example: Variance of an average of models

Let $\sigma^2 = 4$ and $B = 10$.

- If the models are **independent** ($\rho = 0$): $\mathrm{Var} = \frac{4}{10} = 0.4$ (ten times smaller).
- If they are **highly correlated** ($\rho = 0.5$): $\mathrm{Var} = 0.5(4) + \frac{0.5}{10}(4) = 2 + 0.2 = 2.2$ (only about half).
- Even with $B \to \infty$ the variance cannot go below $\rho\sigma^2 = 2$.

Lesson: averaging helps most when the models are *uncorrelated*; bagging tries to decorrelate them using different bootstrap samples. Bagging reduces **variance** but *not bias*, so it works best with **low-bias, high-variance** learners such as deep decision trees.

5.3.3 Out-of-Bag (OOB) Evaluation

Each model never saw about 36.8% of the training instances. We can use these OOB instances as a *free validation set*: for each instance, predict using only the models that did **not** have it in their sample, and compute the accuracy. The **OOB score** is a good estimate of test accuracy *without needing a separate validation set.*

Write-up for the Exam: Bagging and Pasting

Definition

Bagging (bootstrap aggregating) trains many copies of the same learner, each on a *bootstrap sample* (random sample with replacement) of the training data, and combines their outputs by voting or averaging. **Pasting** is the same but samples *without replacement.*

Working

1. Draw B random samples from the training set (with replacement for bagging).
2. Train one base learner (e.g. a deep decision tree) on each sample.
3. For a new instance collect the B predictions.
4. Classification: majority vote; regression: average.
5. Optionally estimate accuracy using the out-of-bag instances.

Assumptions

- The base learner is unstable (small data changes cause large changes in the model)
- Bootstrap samples are representative of the population

Advantages

- Reduces variance and overfitting
- Parallel training; free OOB validation
- Works with any base learner

Disadvantages

- Does not reduce bias
- Less interpretable and slower than one model
- Little gain for stable learners (e.g. linear regression)

Applications

- Improving decision trees (Random Forests)

- Noisy data problems, regression and classification

	Bagging	**Pasting**
Sampling	With replacement (bootstrap)	Without replacement
Duplicates in a sample?	Yes	No
OOB evaluation?	Yes (about 37% per model)	Not directly
Diversity	Slightly higher	Slightly lower
Typical bias / variance	Slightly higher bias, lower variance	Slightly lower bias

5.4 Random Forests

A **Random Forest** is the most popular ensemble method. It is **bagging of decision trees** with one extra trick that makes the trees even more different.

Definition: Random Forest

A **Random Forest** is an ensemble of many decision trees, each trained on a *bootstrap sample* of the data, where at *every split* the tree considers only a **random subset of the features** (typically $\sqrt{p}$ for classification and $p/3$ for regression, with p the total number of features). The forest predicts by *majority vote* (classification) or *average* (regression).

Why the random feature subset? If one feature is very strong, *every* bagged tree would split on it first and all trees would look alike (high correlation ρ in Equation (5.1)). Forcing each split to choose from a random subset **decorrelates** the trees, so averaging them reduces the variance much more.

Reading the Figure

The training data (left) is turned into **three different trees**. Each tree is trained on its *own bootstrap sample* and, at each split, can only choose from *its own random features* (shown in brackets), so the trees have different shapes and make different mistakes. The small squares at the bottom of each tree are **leaves**: red or green squares show the class that leaf predicts. For a new instance each tree sends it down to a leaf and **votes** for that leaf's class: Tree 1 says Red; Trees 2 and 3 say Green. The forest takes the **majority**: **Green**.

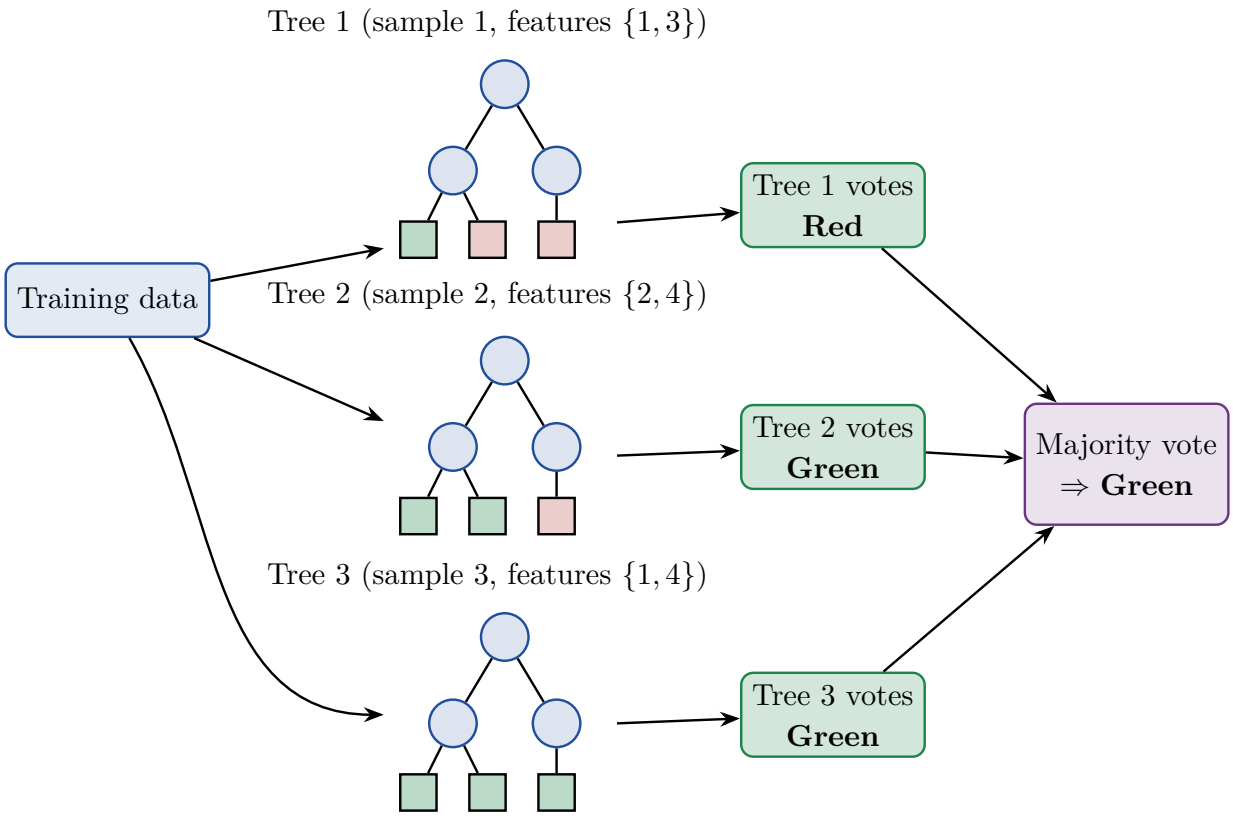


Figure 5.5. A Random Forest of three trees. Each tree sees a different bootstrap sample and different candidate features.

Example: Voting in a forest of five trees

A forest has 5 trees. For a new patient the trees predict: Disease, Healthy, Disease, Disease, Healthy.

	Disease	**Healthy**
Votes	3	2
Estimated probability	$3/5 = 0.60$	$2/5 = 0.40$

The forest predicts **Disease** (with a "confidence" of 60%). For regression the five tree outputs would be averaged.

5.4.1 Feature Importance

A great by-product: a forest tells us **which features matter**. A feature's **importance** is the average total decrease in impurity (Gini or entropy) from all splits that use it, averaged over all trees and normalised so the importances sum to 1.

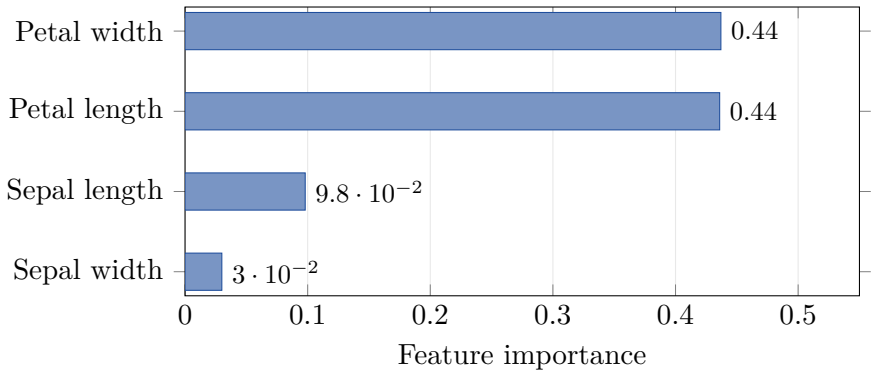


Figure 5.6. Feature importances of a Random Forest (200 trees) on the Iris dataset.

Reading the Figure

Each bar is one of the four measurements of an iris flower; a **longer bar means the forest relied on that feature more**. The two **petal** measurements together account for about 87% of the importance ($0.436 + 0.437$), while the *sepal width* contributes only 3%. This agrees with the single tree of Chapter 3, which separated all three species using only the petal width. Such a ranking helps in **feature selection** and in explaining the model to non-experts.

Write-up for the Exam: Random Forest

Definition

A **Random Forest** is an ensemble of decision trees built by *bagging*, in which each split considers only a *random subset of features*; the final output is the majority vote (classification) or the mean (regression) of the trees.

Working

1. Draw B bootstrap samples from the training data.
2. For each sample grow a decision tree; at *each node* choose the best split from a random subset of $m \approx \sqrt{p}$ features.
3. Grow the trees deep (little or no pruning).
4. Predict by majority vote or average of all trees.
5. Compute the OOB score and the feature importances.

Assumptions

- Individual trees are low-bias but high-variance
- Trees are weakly correlated because of the randomisation

Advantages

- High accuracy; handles many features and mixed data types
- Much less overfitting than a single tree

- Gives feature importance and OOB error; needs little tuning

Disadvantages

- Less interpretable than one tree; larger memory
- Slower prediction with many trees
- May be biased towards features with many categories

Applications

- Credit-risk and churn prediction
- Medical diagnosis, bioinformatics, remote sensing
- Feature selection

Main hyperparameters.

- `n_estimators`: number of trees (more is better but slower; 100–500 is typical).
- `max_features`: features tried at each split (default $\sqrt{p}$).
- `max_depth`, `min_samples_leaf`: limit the size of each tree.

Extra-Trees. *Extremely Randomised Trees* go one step further and also choose the *split thresholds* at random: faster to train and even lower variance, at the price of a little more bias.

5.5 Boosting

Bagging trains models *in parallel* to reduce variance. **Boosting** trains them **one after another**, each model trying to correct the mistakes of the previous ones. It reduces **bias**.

Definition: Boosting

Boosting is a sequential ensemble method that combines many *weak learners* into a strong learner by training them in sequence, where each new learner focuses on the instances that the earlier learners handled badly, and the final prediction is a *weighted* combination of all learners.

Key Idea

Analogy. A student takes a test, then studies *only the questions answered wrongly*, then takes another test, and so on. After several rounds the student has covered the hard questions that a single reading would have missed.

5.5.1 AdaBoost (Adaptive Boosting)

AdaBoost keeps a **weight** on every training instance. After each round it **increases the weights of misclassified instances** so the next learner pays more attention to them. Labels are coded $y_i \in \{-1, +1\}$.

Definition: AdaBoost algorithm

1. Initialise weights $w_i = \dfrac{1}{n}$ for all n instances.
2. For $t = 1, \ldots, T$:
   (a) Train a weak learner h_t on the *weighted* data.
   (b) Weighted error: $\varepsilon_t = \sum_{i:\, h_t(\boldsymbol{x}_i) \neq y_i} w_i$.
   (c) Learner weight ("say"): $\alpha_t = \dfrac{1}{2} \ln \dfrac{1 - \varepsilon_t}{\varepsilon_t}$.
   (d) Update weights: $w_i \leftarrow w_i \, e^{-\alpha_t \, y_i \, h_t(\boldsymbol{x}_i)}$, then divide by the sum so that $\sum w_i = 1$. (Correct instances are multiplied by $e^{-\alpha_t} < 1$, wrong ones by $e^{+\alpha_t} > 1$.)
3. Final classifier: $H(\boldsymbol{x}) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t \, h_t(\boldsymbol{x}) \right)$.

Meaning of α_t. A learner with small error gets a **large** weight, one with error 0.5 (random guessing) gets $\alpha = 0$, and one with error above 0.5 would get a negative weight.

Error ε_t	0.05	0.10	0.20	0.30	0.50
$\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$	1.47	1.10	0.69	0.42	0

End-to-end example: five instances, three rounds

True labels $y = (+1, +1, -1, -1, +1)$ for instances $1, \ldots, 5$. We use weak learners that are each wrong on only one instance.

Example: AdaBoost by hand

Round 1. Weights $w = (0.2, 0.2, 0.2, 0.2, 0.2)$.
Weak learner h_1 predicts $(+, +, -, +, +)$: it is wrong only on **instance 4**.

$$\varepsilon_1 = 0.2, \qquad \alpha_1 = \tfrac{1}{2} \ln \tfrac{0.8}{0.2} = \tfrac{1}{2} \ln 4 = 0.693.$$

Update: correct instances $\times e^{-0.693} = \times 0.5$; instance 4 $\times e^{0.693} = \times 2$.

$$(0.1,\ 0.1,\ 0.1,\ 0.4,\ 0.1) \xrightarrow{\text{sum}=0.8} w = (0.125,\ 0.125,\ 0.125,\ \mathbf{0.5},\ 0.125).$$

Round 2. Instance 4 now carries half of the total weight, so h_2 must get it right. Say h_2 predicts $(-,+,-,-,+)$: wrong only on **instance 1**.

$$\varepsilon_2 = 0.125, \qquad \alpha_2 = \tfrac{1}{2}\ln\tfrac{0.875}{0.125} = \tfrac{1}{2}\ln 7 = 0.973.$$

Correct $\times e^{-0.973} = 0.378$; instance 1 $\times e^{0.973} = 2.646$. After normalising:

$$w = (\mathbf{0.500},\ 0.071,\ 0.071,\ 0.286,\ 0.071).$$

Round 3. Now instance 1 dominates. Say h_3 predicts $(+,+,+,-,+)$: wrong only on **instance 3**.

$$\varepsilon_3 = 0.071, \qquad \alpha_3 = \tfrac{1}{2}\ln\tfrac{0.929}{0.071} = \tfrac{1}{2}\ln 13 = 1.282.$$

Final ensemble: $H(\boldsymbol{x}) = \operatorname{sign}\big(0.693\,h_1 + 0.973\,h_2 + 1.282\,h_3\big)$.

Instance	y	h_1	h_2	h_3	Score	H	Correct?
1	+	+	−	+	$0.693 - 0.973 + 1.282 = +1.00$	+	✓
2	+	+	+	+	$+2.95$	+	✓
3	−	−	−	+	$-0.693 - 0.973 + 1.282 = -0.38$	−	✓
4	−	+	−	−	$0.693 - 0.973 - 1.282 = -1.56$	−	✓
5	+	+	+	+	$+2.95$	+	✓

Conclusion: each weak learner made a mistake, but the weighted ensemble classifies *all five* instances correctly.

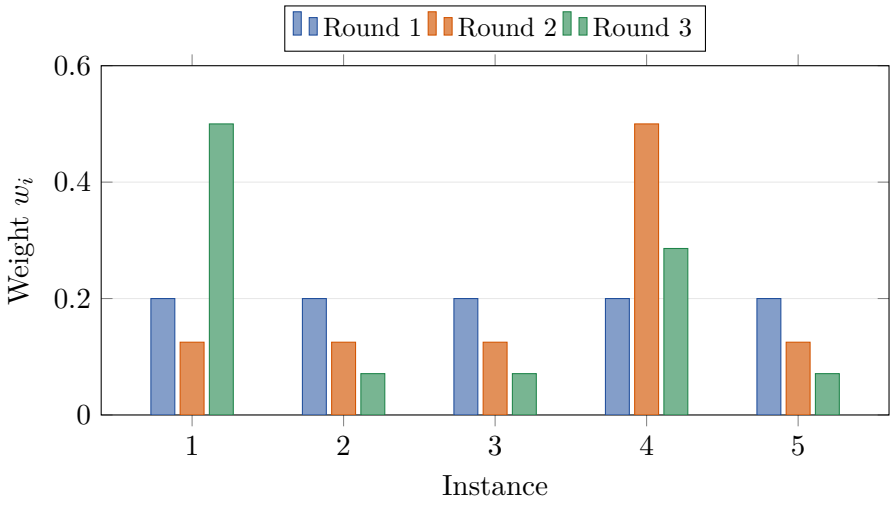


Figure 5.7. How AdaBoost's instance weights change from round to round in the worked example.

Reading the Figure

Each group of bars is one training instance; the three bars show its weight at the start of Round 1 (blue), Round 2 (orange) and Round 3 (green).

- **Round 1 (blue):** all instances are equal (0.2).
- **Round 2 (orange):** instance 4, which learner 1 got wrong, has jumped to 0.5.
- **Round 3 (green):** instance 1, which learner 2 got wrong, now has the biggest weight (0.5) while instance 4 (now handled) has dropped back to 0.286.

The **spotlight moves** to whichever instances are still being misclassified. This is how boosting makes later learners specialise in the hard cases.

Write-up for the Exam: AdaBoost

Definition

AdaBoost (Adaptive Boosting) is a *sequential* ensemble algorithm that builds a strong classifier as a *weighted vote of weak learners* (usually decision stumps), re-weighting the training instances after every round so that misclassified instances get more attention.

Working

1. Start with equal instance weights $w_i = 1/n$.
2. Train a weak learner on the weighted data and compute its weighted error ε_t.
3. Compute its weight $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$.
4. Increase the weights of misclassified instances, decrease the others, and normalise.
5. Repeat for T rounds; predict $H(\boldsymbol{x}) = \operatorname{sign} \sum_t \alpha_t h_t(\boldsymbol{x})$.

Assumptions

- Each weak learner is slightly better than random guessing ($\varepsilon_t < 0.5$)
- Data are not dominated by label noise

Advantages

- Turns weak learners into a strong one; reduces bias
- Few hyperparameters; simple to implement

Disadvantages

- Sensitive to noisy data and outliers (they keep gaining weight)
- Sequential, so it cannot be parallelised

Applications

- Face detection (Viola–Jones)
- Text and customer classification

5.5.2 Gradient Boosting

Gradient boosting generalises the idea. Instead of re-weighting instances it fits each new model to the **residuals** (errors) of the current ensemble.

Definition: Gradient boosting (regression with squared loss)

1. Start with a constant prediction $F_0(\boldsymbol{x}) = \bar{y}$ (the mean).
2. For $m = 1, \dots, M$:
   (a) Compute the **residuals** $r_i = y_i - F_{m-1}(\boldsymbol{x}_i)$ (they are the negative gradient of the squared-error loss).
   (b) Fit a small tree h_m to predict the residuals.
   (c) Update: $F_m(\boldsymbol{x}) = F_{m-1}(\boldsymbol{x}) + \eta\, h_m(\boldsymbol{x})$, where $\eta \in (0, 1]$ is the **learning rate** (shrinkage).
3. The final model is $F_M(\boldsymbol{x}) = \bar{y} + \eta \sum_{m=1}^{M} h_m(\boldsymbol{x})$.

Example: Gradient boosting on four points

Predict a salary (in thousand rupees) from years of experience x:

x	1	2	3	4
y	10	12	20	22

Use a decision stump that splits at $x \le 2.5$ and $\eta = 0.5$.

Step 0. $F_0 = \bar{y} = \frac{10+12+20+22}{4} = 16$ for all x. Residuals $r = (-6, -4, 4, 6)$; MSE $= \frac{36+16+16+36}{4} = \mathbf{26}$.

Step 1. The stump predicts the mean residual on each side: left $\frac{-6-4}{2} = -5$, right $\frac{4+6}{2} = 5$.

$$F_1 = F_0 + 0.5\, h_1 = (16 - 2.5,\ 16 - 2.5,\ 16 + 2.5,\ 16 + 2.5) = (13.5,\ 13.5,\ 18.5,\ 18.5).$$

New residuals $(10 - 13.5,\ 12 - 13.5,\ 20 - 18.5,\ 22 - 18.5) = (-3.5, -1.5, 1.5, 3.5)$; MSE $= \frac{12.25+2.25+2.25+12.25}{4} = \mathbf{7.25}$.

Step 2. Stump on the new residuals: left -2.5, right $+2.5$.

$$F_2 = F_1 + 0.5\, h_2 = (12.25,\ 12.25,\ 19.75,\ 19.75).$$

Residuals $(-2.25, -0.25, 0.25, 2.25)$; MSE $= \frac{5.06+0.06+0.06+5.06}{4} = \mathbf{2.56}$.

Stage	F_0	F_1	F_2
MSE	26.0	7.25	2.56

The error falls at every stage. Prediction for $x = 3$: $F_2(3) = 19.75$ (true value 20).

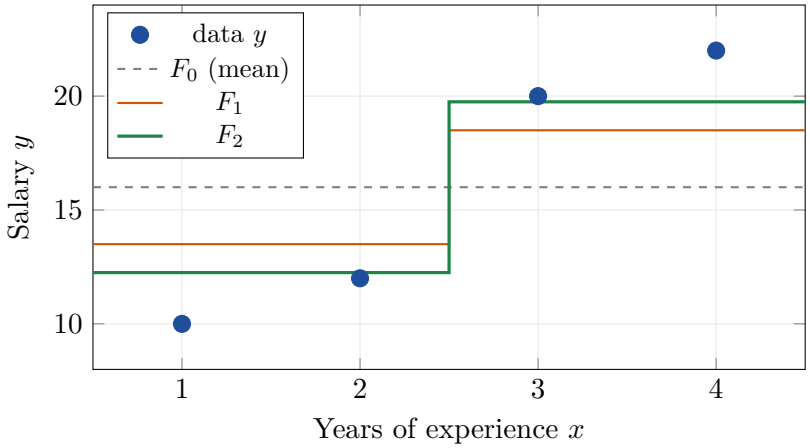


Figure 5.8. Gradient boosting: each stage moves the step-function prediction closer to the data.

Reading the Figure

The blue dots are the data. The **grey dashed line** is the starting guess F_0: the mean, a flat line that is wrong for every point. The **orange step function** F_1 moves the left points down and the right points up, getting closer. The **green step function** F_2 is closer still: its left step (12.25) is near the left points $(10, 12)$ and its right step (19.75) is near $(20, 22)$. Each stage *adds a small correction* to fix the remaining error. With many stages and deeper trees, the curve can follow complex patterns.

Learning rate and number of stages. A small η (e.g. 0.1) makes learning slower but usually generalises better; it must be balanced with the number of trees M (small η needs large M). Too many stages can **overfit**; use early stopping on a validation set. Popular fast implementations are **XGBoost**, **LightGBM** and **CatBoost**.

Write-up for the Exam: Gradient Boosting

Definition

Gradient boosting builds an additive model $F_M(\boldsymbol{x}) = F_0 + \eta \sum_m h_m(\boldsymbol{x})$ *stage by stage*, where each new weak learner h_m is fitted to the **negative gradient of the loss** (the *residuals*, for squared error) of the current model.

Working

1. Initialise with a constant prediction (e.g. the mean).
2. Compute the residuals of the current model.
3. Fit a small decision tree to the residuals.
4. Add the tree to the model scaled by the learning rate η.
5. Repeat for M stages (stop early on a validation set).

Assumptions

- A differentiable loss function suited to the task

- Weak learners are simple (shallow trees)

Advantages

- Very high accuracy on structured/tabular data
- Works for regression, classification and ranking with any differentiable loss

Disadvantages

- Many hyperparameters to tune (learning rate, depth, number of trees)
- Can overfit noisy data; sequential training is slower

Applications

- Web search ranking, click-through-rate prediction
- Credit scoring and sales forecasting

	AdaBoost	**Gradient boosting**
How errors are handled	Re-weights misclassified instances	Fits new learner to the residuals
Loss function	Exponential loss	Any differentiable loss
Combination	Weighted vote $\sum \alpha_t h_t$	Sum $F_0 + \eta \sum h_m$
Weak learner	Usually a stump	Small tree (depth 3–8)

5.6 Stacking

Voting uses a fixed rule (majority, average). **Stacking** (stacked generalisation) asks: *why not train a model to learn the best way to combine the predictions?*

Definition: Stacking

Stacking trains several different **base learners** (level-0) and then trains a **meta-learner** (level-1, also called the *blender*) whose *inputs are the predictions of the base learners* and whose output is the final prediction.

Reading the Figure

The input goes to three **level-0** base learners (grey) that are different algorithms. Their three outputs are collected into a small table of predictions (orange), which becomes the *input features* of the **level-1 meta-learner** (purple). The meta-learner has *learned* how much to trust each base learner: for example it may learn that the SVM is most reliable and give it the largest weight. Its output is the final prediction (green).

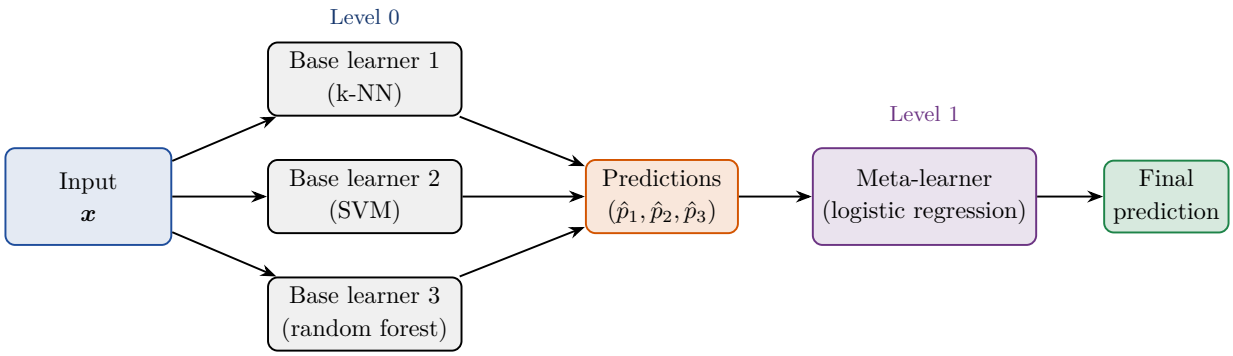


Figure 5.9. Architecture of a stacking ensemble.

Avoiding leakage. If the base learners predict on the *same data they were trained on*, their predictions are too good and the meta-learner learns to over-trust them. The standard fix is to use **out-of-fold predictions**: for each training instance use the prediction of a base model that was trained *without* that instance (cross-validation). A simpler variant, **blending**, holds out a separate validation set to train the meta-learner.

Example: A meta-learner in action

Three trained base models predict the probability that a customer will leave (churn) for one customer: $(0.80,\ 0.30,\ 0.60)$. A linear meta-learner has learned the weights $(0.5,\ 0.2,\ 0.3)$.

$$\hat{p} = 0.5(0.80) + 0.2(0.30) + 0.3(0.60) = 0.40 + 0.06 + 0.18 = \mathbf{0.64}.$$

Since $0.64 \geq 0.5$ the customer is predicted to **churn**. A simple average of the three would give 0.567; stacking gave more influence to the first model because the meta-learner found it most reliable.

Write-up for the Exam: Stacking

Definition

Stacking (stacked generalisation) is an ensemble technique in which the predictions of several diverse *base learners* are used as inputs to a *meta-learner* that is trained to produce the final prediction.

Working

1. Split the training data into folds.
2. Train each base learner and obtain *out-of-fold predictions* for every training instance.
3. Use these predictions as features and the true labels as targets to train the meta-learner.
4. Retrain the base learners on all training data.
5. For a new instance: get the base predictions, then pass them to the meta-learner.

Assumptions

- Base learners are diverse and make different errors

- Meta-learner is simple enough not to overfit

Advantages

- Learns the best combination automatically
- Often more accurate than any single model or simple voting

Disadvantages

- Complex, slow to train and tune
- Risk of information leakage and overfitting
- Hard to interpret

Applications

- Machine-learning competitions (e.g. the Netflix Prize)
- High-stakes prediction where small accuracy gains matter

5.7 Comparison and Practical Advice

	Bagging / Random Forest	**Boosting**	**Stacking**
Training	Parallel	Sequential	Two levels
Data used	Bootstrap samples	Re-weighted data / residuals	Out-of-fold predictions
Main effect	Reduces **variance**	Reduces **bias** (and some variance)	Learns best combination
Base learner	Strong, high variance (deep trees)	Weak, high bias (stumps, shallow trees)	Diverse algorithms
Combination	Vote / average	Weighted sum	Meta-model
Overfitting risk	Low	Higher (noisy data)	Moderate
Parallelisable?	Yes	No	Partly
Examples	Random Forest, Extra-Trees	AdaBoost, XGBoost, LightGBM	StackingClassifier

Try It Yourself

Compare single models and ensembles on the `moons` dataset (two interleaving half circles, with noise).

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.ensemble import (VotingClassifier, BaggingClassifier
    ,
```

```
        RandomForestClassifier, AdaBoostClassifier,
        GradientBoostingClassifier, StackingClassifier)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=42)

lr = LogisticRegression()
rf = RandomForestClassifier(n_estimators=100, random_state=42)
sv = SVC(probability=True, random_state=42)

# soft voting
vote = VotingClassifier([("lr", lr), ("rf", rf), ("svc", sv)],
                        voting="soft").fit(Xtr, ytr)
print("soft voting:", vote.score(Xte, yte))

# bagging of 200 trees with out-of-bag score
bag = BaggingClassifier(DecisionTreeClassifier(), n_estimators
    =200,
        bootstrap=True, oob_score=True, random_state=42).fit(Xtr,
            ytr)
print("bagging:", bag.score(Xte, yte), "OOB:", bag.oob_score_)
```

Results on the 150 test points:

Model	Test accuracy	Model	Test accuracy
Single decision tree	0.853 (train: 1.000)	AdaBoost (200 stumps)	0.880
Logistic regression	0.860	Gradient boosting	0.907
SVM	0.907	Bagging (200 trees)	0.900 (OOB 0.891)
Hard voting	0.913	Pasting (200 trees)	0.900
Soft voting	**0.933**	Random Forest	0.900 (OOB 0.897)
Stacking	0.900		

What to notice: the single tree *memorises* the training set (train accuracy 1.0) but scores only 0.853 on test data; **bagging the same trees raises test accuracy to** 0.90; and the OOB score (0.891, 0.897) is close to the test accuracy without using the test set. *Honest note:* with only 150 test points one point is 0.67%, so differences of 1–2% between ensembles are within noise; we should not claim that one ensemble is always the best.

Common Mistake

Common mistakes with ensembles.

- Combining models that are all *the same* (no diversity) gives no gain.
- Evaluating a stacking model with base predictions made on the training data causes leakage.
- Using boosting on very noisy labels: the algorithm keeps chasing the noise.
- Assuming more trees always overfit: for Random Forests, more trees never hurt much, but for boosting too many stages can.

Chapter Summary

- **Ensemble learning** combines several diverse models; it works because errors partly cancel (three 70% classifiers give 78.4% by majority vote, five give 83.7%).
- **Voting:** hard (majority of labels) or soft (average of probabilities, usually better).
- **Bagging** (with replacement) and **pasting** (without) train the same learner on random subsets; Var $= \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$; OOB $\approx 36.8\%$ gives free validation.
- **Random Forest** = bagged trees + random feature subset at each split; gives feature importances.
- **AdaBoost:** re-weight misclassified instances; $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$; final $H = \text{sign}\sum\alpha_t h_t$.
- **Gradient boosting:** fit each tree to the current residuals; $F_m = F_{m-1} + \eta h_m$.
- **Stacking:** a meta-learner learns to combine base predictions; use out-of-fold predictions.
- Bagging reduces **variance**; boosting reduces **bias**; stacking learns the combination.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define ensemble learning. **[2 marks]**
2. What is a weak learner? **[2 marks]**
3. Differentiate between hard voting and soft voting. **[2 marks]**
4. Differentiate between bagging and pasting. **[2 marks]**
5. What is a bootstrap sample? **[2 marks]**
6. What is an out-of-bag instance? What fraction of the data is OOB? **[2 marks]**
7. How does a Random Forest differ from bagging of decision trees? **[2 marks]**

8. What is feature importance in a Random Forest? **[2 marks]**

9. State the formula for the weight α_t of a learner in AdaBoost. **[2 marks]**

10. What does gradient boosting fit at each stage? **[2 marks]**

11. What is a meta-learner? **[2 marks]**

12. Which ensemble technique reduces variance and which reduces bias? **[2 marks]**

Model Answer: Q6, Q9 and Q12

Q6. When a bootstrap sample of size n is drawn with replacement, some instances are never selected; they are the *out-of-bag* instances of that sample. The chance that an instance is left out is $(1 - 1/n)^n \to e^{-1} \approx 0.368$, so about **36.8%** of the data is OOB for each model.
Q9. $\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$, where ε_t is the weighted error of the t-th weak learner.
Q12. Bagging (and Random Forests) mainly reduce *variance*; boosting mainly reduces *bias*.

B. Short Answer Questions (5 marks each)

1. Explain why ensemble methods improve accuracy. What is the role of diversity? **[5 marks]**

2. Explain hard and soft voting classifiers with an example. **[5 marks]**

3. Explain bagging with a diagram. How does out-of-bag evaluation work? **[5 marks]**

4. Differentiate between bagging and pasting. **[5 marks]**

5. Explain the Random Forest algorithm. Why do we choose a random subset of features at each split? **[5 marks]**

6. Explain the AdaBoost algorithm. **[5 marks]**

7. Explain gradient boosting with a suitable example. **[5 marks]**

8. Explain stacking with a neat diagram. **[5 marks]**

9. Differentiate between bagging and boosting. **[5 marks]**

Model Answer: B9 (comparison for 5 marks)

Basis	Bagging	Boosting
Training	Parallel; models independent	Sequential; each model depends on the previous
Data	Bootstrap samples	Re-weighted instances / residuals
Goal	Reduce variance	Reduce bias
Base learner	Strong, unstable (deep trees)	Weak (stumps)
Combination	Equal-weight vote/average	Weighted vote/sum
Noise	Robust	Sensitive
Example	Random Forest	AdaBoost, XGBoost

C. Long Answer Questions (10 marks each)

1. What is ensemble learning? Explain the different types of ensemble methods with diagrams. **[10 marks]**
2. Explain bagging, pasting and Random Forests. Discuss why they reduce variance. **[10 marks]**
3. Explain the AdaBoost algorithm in detail and work through a numerical example. **[10 marks]**
4. Explain gradient boosting and show with a numerical example how the error decreases over stages. **[10 marks]**
5. Explain voting classifiers (hard, soft, weighted) and stacking. Give examples. **[10 marks]**
6. Compare bagging, boosting and stacking in terms of training, bias–variance effect, advantages and disadvantages. **[10 marks]**

Model Answer: C3 (key steps of the numerical part)

Take 5 instances with equal weights 0.2. A weak learner misclassifies one instance: $\varepsilon_1 = 0.2$. $\alpha_1 = \frac{1}{2}\ln\frac{0.8}{0.2} = 0.693$. Correct instances: $0.2 \times e^{-0.693} = 0.1$; misclassified: $0.2 \times e^{0.693} = 0.4$. Sum $= 4(0.1) + 0.4 = 0.8$.
Normalised weights: correct $= 0.1/0.8 = 0.125$, misclassified $= 0.4/0.8 = 0.5$.
Next learner concentrates on the misclassified instance. Final model $H(\boldsymbol{x}) = \text{sign}\left(\sum \alpha_t h_t(\boldsymbol{x})\right)$.

D. Numerical Problems

1. Three independent classifiers each have accuracy 0.8. Find the accuracy of the majority vote. *(Answer:* $3(0.8)^2(0.2) + 0.8^3 = 0.384 + 0.512 = 0.896$.*)*

2. Three classifiers give $P(\text{class } 1) = (0.9,\ 0.45,\ 0.40)$. Find the hard- and soft-voting predictions. *(Answer: hard votes* $1, 0, 0 \Rightarrow$ *class* 0*; soft:* $\bar{P} = 0.583 \Rightarrow$ *class* 1*.)*

3. Using weights $(2, 1, 1)$ for the classifiers of problem 2, find the weighted soft-voting probability. *(Answer:* $\frac{2(0.9)+0.45+0.40}{4} = \frac{2.65}{4} = 0.6625 \Rightarrow$ *class* 1*.)*

4. For $n = 100$ compute the probability that a given instance is *not* in a bootstrap sample. *(Answer:* $(0.99)^{100} = 0.366$*; so* 63.4% *are in the sample.)*

5. $B = 20$ models each have variance 9 and pairwise correlation 0.3. Find the variance of their average and the limit as $B \to \infty$. *(Answer:* $0.3(9) + \frac{0.7(9)}{20} = 2.7 + 0.315 = 3.015$*; limit* $= 2.7$*.)*

6. Find α_t for $\varepsilon_t = 0.1$, 0.3 and 0.5 in AdaBoost. *(Answer:* $\frac{1}{2}\ln 9 = 1.099$*;* $\frac{1}{2}\ln\frac{7}{3} = 0.424$*;* 0*.)*

7. In AdaBoost with 10 instances (weights 0.1) a learner misclassifies 2. Find ε, α and the new normalised weights. *(Answer:* $\varepsilon = 0.2$*,* $\alpha = 0.693$*; correct* $0.05 \to 0.0625$ *each, wrong* $0.2 \to 0.25$ *each.)*

8. Gradient boosting: $y = (2, 4, 9, 11)$ at $x = 1, 2, 3, 4$. $F_0 = \bar{y}$; a stump splits at $x \le 2.5$; $\eta = 0.1$. Find F_1 and the MSE before and after. *(Answer:* $F_0 = 6.5$*; residuals* $(-4.5, -2.5, 2.5, 4.5)$*; stump* $= (-3.5, 3.5)$*;* $F_1 = (6.15, 6.15, 6.85, 6.85)$*; MSE* $13.25 \to 10.92$*.)*

9. A stacking meta-learner has weights $(0.5, 0.2, 0.3)$ and base predictions $(0.8, 0.3, 0.6)$. Find the final probability. *(Answer:* $0.40 + 0.06 + 0.18 = 0.64$*.)*

E. Multiple-Choice Questions

1. Combining the predictions of several models is called:
   (a) regularisation (b) ensemble learning (c) scaling (d) pruning

2. In soft voting the final class is chosen using:
   (a) majority of labels (b) average of predicted probabilities (c) random choice (d) the first classifier

3. Bagging samples the training data:
   (a) without replacement (b) with replacement (c) only once (d) never

4. The fraction of instances that are OOB for large n is about:
   (a) 10% (b) 25% (c) 36.8% (d) 63.2%

5. Random Forests choose, at each split:
   (a) all features (b) one fixed feature (c) a random subset of features (d) no feature

6. Boosting mainly reduces:
   (a) variance (b) bias (c) data size (d) noise

7. Bagging mainly reduces:
   (a) bias (b) variance (c) irreducible error (d) training time

8. In AdaBoost, a learner with error 0.5 gets weight $\alpha =$
   (a) 1 (b) $\ln 2$ (c) 0 (d) ∞

9. Gradient boosting fits each new tree to the:
   (a) original labels (b) residuals (c) random noise (d) features

10. The final model in stacking is called the:
    (a) base learner (b) meta-learner (c) weak learner (d) stump

11. Which method can be trained fully in parallel?
    (a) AdaBoost (b) Gradient boosting (c) Random Forest (d) all of these

12. Ensembles help most when members are:
    (a) identical (b) accurate and diverse (c) inaccurate (d) correlated

Answers: 1-b, 2-b, 3-b, 4-c, 5-c, 6-b, 7-b, 8-c, 9-b, 10-b, 11-c, 12-b.

F. Practice Problems

1. Compute by hand the accuracy of a majority vote of 7 independent classifiers each 60% accurate. *(Answer:* 0.710*, see Figure 5.1.)*

2. Continue the AdaBoost example: suppose a fourth learner has $\varepsilon_4 = 0.1$. Compute α_4 and explain how it changes the combined vote.

3. **Coding:** train a `RandomForestClassifier` on the *wine* dataset (`load_wine`), print the feature importances sorted, and compare accuracy with a single `DecisionTreeClassifier`.

4. **Coding:** plot test accuracy of `GradientBoostingClassifier` against `n_estimators` (1 to 300) for learning rates 0.5 and 0.05; find where it starts to overfit.

5. Draw a diagram of a stacking ensemble and label base learners, predictions and meta-learner.

Quick Revision

Topic	One-line recall
Ensemble	Many diverse models combined; better and more stable than one.
Majority vote	$P = \sum_{k>n/2} \binom{n}{k} p^k (1-p)^{n-k}$; 3 models of 70% $\to$ 78.4%.
Hard / soft voting	Majority of labels / average of probabilities (soft often better).
Bagging / pasting	Bootstrap (with replacement) / without replacement; vote or average.
OOB	$\approx 36.8\%$ left out of each bootstrap sample; free validation.
Variance of average	$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
Random Forest	Bagged trees + random features per split ($\sqrt{p}$); importances.
AdaBoost	$\alpha = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$; $w \leftarrow we^{\mp\alpha}$; $H = \text{sign}\sum \alpha h$.
Gradient boosting	Fit residuals; $F_m = F_{m-1} + \eta h_m$.
Stacking	Meta-learner on out-of-fold base predictions.
Bias–variance	Bagging: variance $\downarrow$; boosting: bias $\downarrow$.

Chapter 6

Support Vector Machines and Naive Bayes Classifiers

Learning Objectives

After studying this chapter you will be able to:

- explain the **maximum-margin idea** of a linear SVM and the role of **support vectors**;
- explain the **soft margin** and the effect of the parameter C;
- use the **dual form** to show that an SVM depends only on dot products between points;
- explain the **kernel trick** and compute **polynomial** and **RBF** kernels by hand;
- describe **nonlinear SVM classification** and tune C and γ;
- explain **SVM regression** with the ε-tube and the ε-insensitive loss;
- apply **Gaussian**, **multinomial** and **Bernoulli Naive Bayes** classifiers, with Laplace smoothing, end to end;
- answer 2-, 5- and 10-mark university questions on Unit III.

> *Nothing is more practical than a good theory.*
>
> — Vladimir Vapnik, inventor of the SVM

This chapter completes Unit III. It has two halves:

- **Support Vector Machines (SVMs):** linear classification, nonlinear classification with kernels, and regression;
- **Naive Bayes classifiers:** the probabilistic family for text, medical and many other problems.

Throughout the chapter you will meet some cartoon characters who help us picture the ideas: a mayor who builds roads, a strict and a relaxed teacher, a detective who loves clues, and a tunnel full of data points.

6.1 Linear SVM Classification

6.1.1 The Idea: Build the Widest Road

Imagine two colonies of houses, the **Blue Colony** and the **Orange Colony**. The mayor wants to build a road between them so that the two colonies are kept apart. Many roads would do the job, but a good mayor builds the **widest road possible**, so that no house is ever too close to the traffic. The houses that end up right at the *edge* of the road are the ones that decide where the road can go.

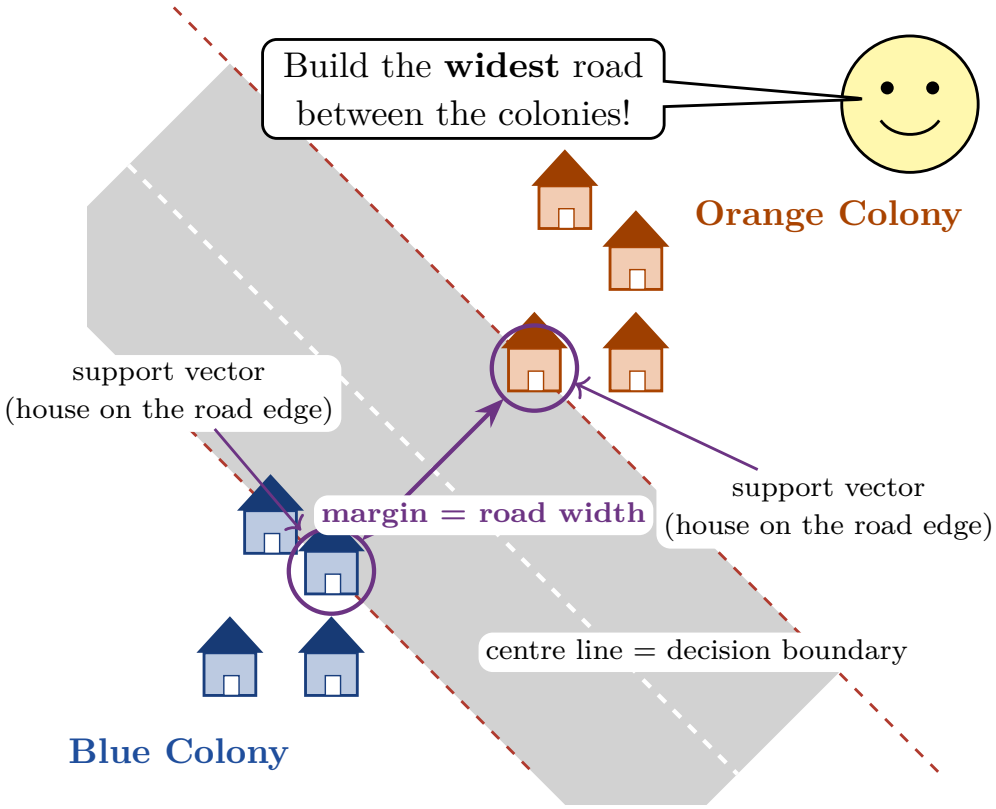


Figure 6.1. A cartoon of the maximum-margin classifier: the widest road between two colonies.

Reading the Figure

Houses are data points: blue houses are one class and orange houses are the other. The **grey road** is the margin and the **white dashed centre line** is the decision boundary. The **red dashed lines** are the edges of the road.

The two **purple circled houses** sit exactly on the edge of the road. They are the **support vectors**. If you moved or removed any *other* house (without letting it cross the edge) the road would stay exactly where it is; but if you moved a circled house, the whole road would shift. The **purple arrow** shows the road width, the **margin**, which the SVM tries to make as large as possible. A wide road means the classifier keeps a safe distance from both classes, so new houses (new data) are less likely to be misclassified.

6.1.2 Definitions

Definition: Support Vector Machine (SVM)

A **Support Vector Machine** is a supervised learning algorithm that classifies data by finding the **maximum-margin hyperplane**: the separating hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ that is *farthest* from the closest training points of both classes. With labels $y_i \in \{-1, +1\}$ the **hard-margin** problem is

$$\min_{\boldsymbol{w}, b} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 \qquad \text{subject to} \qquad y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 \quad \text{for all } i,$$

and a new point is classified by $\hat{y} = \operatorname{sign}(\boldsymbol{w} \cdot \boldsymbol{x} + b)$.

Definition: Margin and support vectors

The **margin** is the width $2/\|\boldsymbol{w}\|$ of the empty strip between the two parallel hyperplanes $\boldsymbol{w} \cdot \boldsymbol{x} + b = \pm 1$. The **support vectors** are the training points that lie *on* these hyperplanes (or, for a soft margin, inside them or on the wrong side); they alone determine $\boldsymbol{w}$ and b.

In Chapter 4 we solved a small SVM by hand. Here is a quick reminder of its answer, which we will reuse: for the spam data the support vectors were $(1, 1)$ with $y = -1$ and $(3, 3)$ with $y = +1$, giving $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$ and margin $2/\|\boldsymbol{w}\| = 2.83$.

6.1.3 Soft-Margin SVM: Strict and Relaxed Teachers

Real data usually overlap, so no road can perfectly separate the colonies. A **soft margin** lets a few houses stand on the road or even on the wrong side, but each such house is fined.

Definition: Soft-margin SVM

Introduce a **slack variable** $\xi_i \geq 0$ for each point that measures how far it violates the margin, and solve

$$\min_{\boldsymbol{w}, b, \boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C \sum_{i=1}^{n} \xi_i \qquad \text{s.t.} \qquad y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The hyperparameter $C > 0$ controls the trade-off. Equivalently the SVM minimises the **hinge loss** $\max(0, 1 - y_i f(\boldsymbol{x}_i))$ plus the penalty $\frac{1}{2C}\|\boldsymbol{w}\|^2$.

Think of C as the **strictness of a teacher**.

- **Large C = strict teacher:** "No student may break the rules!" The road becomes narrow and twists to avoid every violation: low bias, *high variance*, risk of overfitting.
- **Small C = relaxed teacher:** "A few mistakes are fine." The road stays wide and smooth: higher bias, *low variance*, risk of underfitting.

(a) $C = 0.01$**: relaxed teacher wide road, margin** 5.0

(b) $C = 100$**: strict teacher narrow road, margin** 0.53

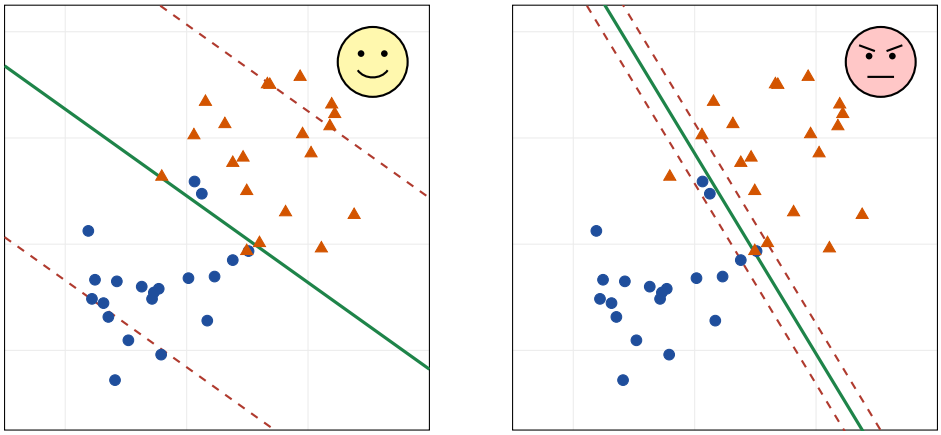


Figure 6.2. The same 40 points fitted with a very small and a very large C (real SVM fits).

Reading the Figure

Both panels show the *same* 40 houses (blue circles and orange triangles; the two colonies overlap a little in the middle). The **green line** is the decision boundary and the **red dashed lines** are the road edges.

(a) Relaxed teacher ($C = 0.01$): the road is very wide (margin 5.0) and covers many houses, so many of them are support vectors. The model pays little attention to the individual rule-breakers and looks only at the overall pattern.
(b) Strict teacher ($C = 100$): the road is narrow (margin 0.53) and the line is tilted to squeeze between the points, because every violation is expensive. The model follows the training data closely.

C	Support vectors	Margin	Train accuracy	Test accuracy (400 pts)
0.01 (relaxed)	32	5.00	0.925	0.953
1	11	1.63	0.900	0.948
100 (strict)	8	0.53	0.950	0.940

Read the table: as C grows, the margin shrinks ($5.00 \to 0.53$), fewer points are support vectors ($32 \to 8$) and the *training* accuracy rises slightly ($0.925 \to 0.950$) while the *test* accuracy drifts down ($0.953 \to 0.940$): the strict teacher fits the training set better but generalises a little worse. (The differences in accuracy are small here, since the two classes are only mildly overlapping and 400 test points is a small sample; the large change in margin and support vectors is the main lesson.)

6.1.4 The Dual Form and Support Vectors

The SVM optimisation problem can be rewritten in an equivalent *dual* form using Lagrange multipliers $\alpha_i \geq 0$, one per training point.

Definition: Dual form of the SVM

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j \, y_i y_j \, (\boldsymbol{x}_i \cdot \boldsymbol{x}_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

The weights are $\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$ and a new point is classified by

$$f(\boldsymbol{x}) = \sum_i \alpha_i \, y_i \, (\boldsymbol{x}_i \cdot \boldsymbol{x}) + b.$$

Only the **support vectors** have $\alpha_i > 0$; all other points have $\alpha_i = 0$.

Two important facts:

1. The prediction needs only the *support vectors*, which makes it compact.
2. The data enter *only through dot products* $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$. This is the key to the **kernel trick** in the next section.

Example: The dual form on the spam data

Support vectors: $(1, 1)$ with $y = -1$ and $(3, 3)$ with $y = +1$; all other points have $\alpha = 0$.

Solving for α: by symmetry $\alpha_1 = \alpha_2 = \alpha$. Then

$$\boldsymbol{w} = \alpha(-1)(1,1) + \alpha(+1)(3,3) = \alpha(2,2) = (0.5, 0.5) \;\Rightarrow\; \alpha = 0.25.$$

Check $\sum \alpha_i y_i = 0.25(-1) + 0.25(+1) = 0$ ✓. The bias is $b = -2$ as before.

Predicting the new e-mail $(4, 1)$ using only dot products with the support vectors:

$$f = \underbrace{0.25(-1)(1 \cdot 4 + 1 \cdot 1)}_{=-1.25} + \underbrace{0.25(+1)(3 \cdot 4 + 3 \cdot 1)}_{=+3.75} - 2 = \mathbf{0.5} > 0 \;\Rightarrow\; \text{spam}.$$

This matches the value $f(4, 1) = 0.5$ found in Chapter 4 directly from $\boldsymbol{w}$.

Dual objective value: $\sum \alpha_i - \frac{1}{2} \sum_{i,j} \ldots = 0.5 - 0.25 = 0.25 = \frac{1}{2}\|\boldsymbol{w}\|^2$, equal to the primal value: the primal and dual give the same answer.

6.1.5 Practical Points

- **Scale the features.** The margin is measured with distances, so a feature with a huge range (salary in rupees) would dominate one with a tiny range (age in years). Standardise first.
- **Choose C by cross-validation** on a grid such as $10^{-2}, 10^{-1}, \ldots, 10^{3}$.

- **More than two classes:** SVMs are binary. For K classes use *one-vs-rest* (K classifiers, pick the highest score) or *one-vs-one* ($K(K-1)/2$ classifiers that vote).
- **No probabilities by default.** The SVM output $f(\boldsymbol{x})$ is a signed distance, not a probability (scikit-learn can fit one afterwards with `probability=True`).

Linear SVM classification

Advantages

- Wide margin gives good generalisation
- Effective with many features (text)
- Only support vectors matter, so it is compact
- Convex problem: a single global optimum

Disadvantages

- No probabilities by default
- Needs scaled features
- Sensitive to the choice of C
- Slow for very large datasets
- Hard margin fails if classes overlap

Applications

- Spam and news-article classification
- Handwritten digit recognition
- Gene-expression and cancer classification
- Image classification

6.2 Nonlinear SVM Classification

6.2.1 When a Straight Line Is Not Enough

Many real problems cannot be solved by a straight line. For instance, if the "blue" houses are in the *middle* of a village and the "orange" houses are all around them, no straight road can separate them.

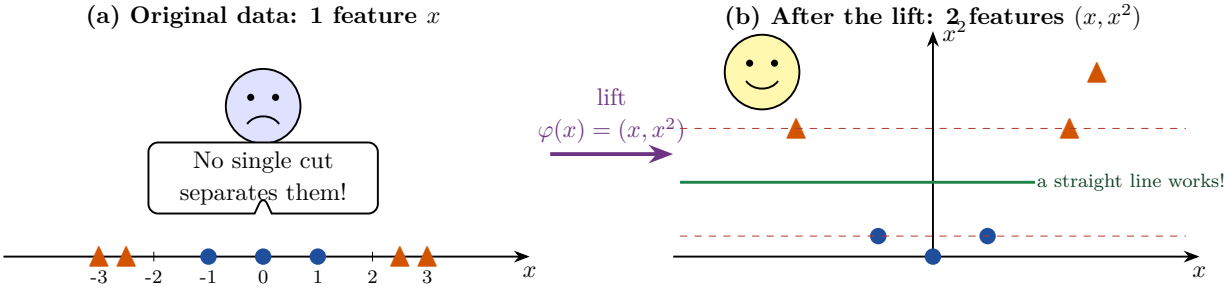


Figure 6.3. The lifting trick: data that cannot be separated in one dimension become separable after adding the feature x^2.

Reading the Figure

Panel (a): the data live on a single number line. The blue circles are in the middle ($x = -1, 0, 1$) and the orange triangles are on both outsides ($x = \pm 2.5, \pm 3$). Wherever we put *one* cut-point on the line, some blue and some orange end up on the same side. The sad face is right: no straight cut works.

Panel (b): we *lift* every point into two dimensions by adding the new feature x^2 (the purple arrow). A point $x = 3$ becomes $(3, 9)$, and $x = 1$ becomes $(1, 1)$. Now the orange

points have large x^2 values and the blue points have small ones, so a *horizontal straight line* (green) separates them perfectly, with red dashed lines marking the margin. Going back to the original line, this horizontal cut corresponds to *two* cut-points, $x \approx -1.9$ and $x \approx +1.9$: a **nonlinear** boundary in x is a **linear** boundary in the lifted space.

Key Idea

A linear model in a higher-dimensional feature space is a nonlinear model in the original space. The SVM can therefore learn curved boundaries by first mapping the data with a function φ and then finding the maximum-margin hyperplane there.

Example: Solving the lifted problem end to end

Data: $x = -1, 0, 1$ are class -1 (blue); $x = \pm 2.5, \pm 3$ are class $+1$ (orange). Map $\varphi(x) = (x, x^2)$.

x	-1	0	1	-2.5	2.5	-3	3
$\varphi(x) = (x, x^2)$	$(-1, 1)$	$(0, 0)$	$(1, 1)$	$(-2.5, 6.25)$	$(2.5, 6.25)$	$(-3, 9)$	$(3, 9)$
Class	$-$	$-$	$-$	$+$	$+$	$+$	$+$

Step 1: closest opposite points. In the lifted space the blue points $(\pm 1, 1)$ have $x^2 = 1$ and the orange points $(\pm 2.5, 6.25)$ have $x^2 = 6.25$. These four are the support vectors. The separating line is horizontal, $\boldsymbol{w} = (0, a)$.

Step 2: margin conditions. $a(6.25) + b = +1$ and $a(1) + b = -1$. Subtracting, $5.25\, a = 2$ so

$$a = 0.381, \qquad b = -1 - 0.381 = -1.381, \qquad f(x) = 0.381\, x^2 - 1.381.$$

Step 3: boundary. $f(x) = 0 \iff x^2 = 3.625 \iff x = \pm 1.90$. The margin is $2/0.381 = 5.25$ units in the x^2 direction.

Step 4: check the others. $f(0) = -1.381 \leq -1$ ✓; $f(\pm 3) = 0.381(9) - 1.381 = 2.05 \geq 1$ ✓.

Step 5: predict. $x = 2$: $f = 0.381(4) - 1.381 = 0.143 > 0 \Rightarrow$ **orange**. $x = 1.5$: $f = 0.381(2.25) - 1.381 = -0.524 < 0 \Rightarrow$ **blue**.

The cost of lifting. Adding more and more features (polynomial features of degree 3, 10, ...) quickly produces an enormous number of columns: d features and degree p give about $\binom{d+p}{p}$ new features. Computing with them directly becomes very slow. The **kernel trick** avoids this cost.

6.2.2 The Kernel Trick

Remember that the dual SVM uses the data only through dot products $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$. After the lift we would need $\varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_j)$. A kernel computes that number *without ever building* φ.

Definition: Kernel function

A **kernel** is a function $K(\boldsymbol{a}, \boldsymbol{b})$ that equals a dot product in some (possibly very high-dimensional) feature space: $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. The **kernel trick** replaces every dot product $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$ in the SVM by $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$, so the SVM works in the high-dimensional space *at the cost of computing a simple formula*:

$$f(\boldsymbol{x}) = \sum_i \alpha_i \, y_i \, K(\boldsymbol{x}_i, \boldsymbol{x}) + b.$$

Kernel	Formula	Parameters
Linear	$K(\boldsymbol{a}, \boldsymbol{b}) = \boldsymbol{a} \cdot \boldsymbol{b}$	none
Polynomial	$K(\boldsymbol{a}, \boldsymbol{b}) = (\gamma \, \boldsymbol{a} \cdot \boldsymbol{b} + r)^d$	degree d, γ, r
RBF (Gaussian)	$K(\boldsymbol{a}, \boldsymbol{b}) = \exp(-\gamma \lVert \boldsymbol{a} - \boldsymbol{b} \rVert^2)$	$\gamma > 0$
Sigmoid	$K(\boldsymbol{a}, \boldsymbol{b}) = \tanh(\gamma \, \boldsymbol{a} \cdot \boldsymbol{b} + r)$	γ, r

Example: The polynomial kernel really equals a dot product

Let $\boldsymbol{a} = (1, 2)$ and $\boldsymbol{b} = (3, 1)$ and take the kernel $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b})^2$.

Using the kernel (one cheap step): $\boldsymbol{a} \cdot \boldsymbol{b} = 3 + 2 = 5$, so $K = 5^2 = \mathbf{25}$.

Using the explicit feature map $\varphi(\boldsymbol{x}) = (x_1^2, \ \sqrt{2}\, x_1 x_2, \ x_2^2)$ (three features):

$$\varphi(\boldsymbol{a}) = (1, \ 2.828, \ 4), \qquad \varphi(\boldsymbol{b}) = (9, \ 4.243, \ 1),$$

$$\varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b}) = 1(9) + 2.828(4.243) + 4(1) = 9 + 12 + 4 = \mathbf{25}.$$

Both ways give 25, but the kernel never needed the three-dimensional vectors.

Example: The RBF kernel as a similarity score

For $\boldsymbol{a} = (1, 2)$ and $\boldsymbol{b} = (3, 1)$: $\lVert \boldsymbol{a} - \boldsymbol{b} \rVert^2 = 2^2 + 1^2 = 5$.

γ	0.1	0.5	2
$K = e^{-5\gamma}$	$e^{-0.5} = 0.607$	$e^{-2.5} = 0.082$	$e^{-10} = 0.00005$

Also $K(\boldsymbol{a}, \boldsymbol{a}) = e^0 = 1$ (identical points are maximally similar). The RBF kernel is a **similarity between** 0 **and** 1 that falls as the points move apart, and γ says *how fast*. The feature space of the RBF kernel has infinitely many dimensions, yet each K is one cheap

calculation.

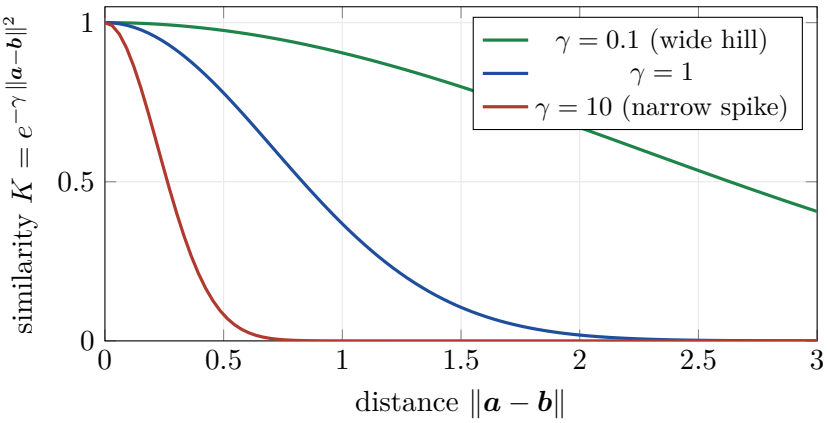


Figure 6.4. The RBF kernel as a function of distance, for three values of γ.

Reading the Figure

The horizontal axis is the **distance** between two points and the vertical axis is their **similarity** (1 = identical, 0 = completely unrelated). Each curve is a different γ.

- **Green ($\gamma = 0.1$):** a wide, gentle hill. Even points that are 2 units apart are still considered fairly similar. Every training point influences a large neighbourhood, so the model is *smooth* (risk of underfitting).
- **Blue ($\gamma = 1$):** a medium hill.
- **Red ($\gamma = 10$):** a narrow spike. Only points very close together are similar. Each training point influences just its immediate surroundings, so the boundary becomes wiggly and tightly follows the data (risk of overfitting).

6.2.3 How an RBF SVM Decides: Hills of Influence

Here is a cartoon way to see the RBF-kernel SVM. Each support vector builds a **hill of influence** around itself: a positive hill for an orange support vector (it "pulls" nearby points to the orange class) and a negative hill (a pit) for a blue one. The decision value $f(\boldsymbol{x})$ at any point is the *sum* of all the hills and pits there.

Reading the Figure

We work in one dimension for clarity, with one blue support vector at $x = 1$ and one orange at $x = 3$.

- The **orange curve** is the positive hill $e^{-(x-3)^2}$ centred on the orange support vector.
- The **blue curve** is the negative pit $-e^{-(x-1)^2}$ centred on the blue support vector.

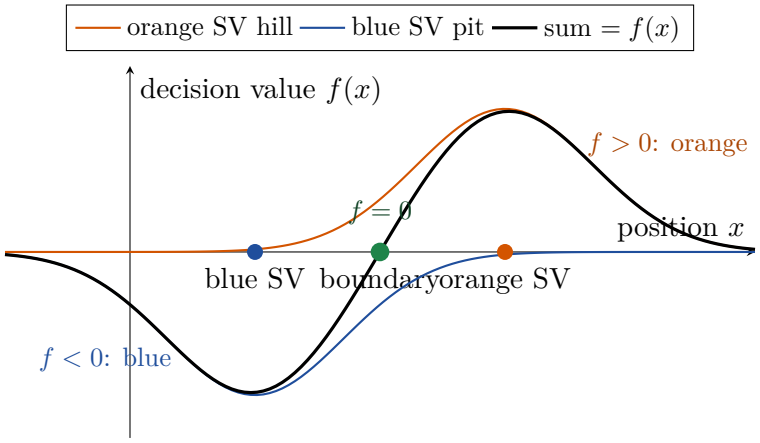


Figure 6.5. Hills of influence: two RBF support vectors ($\gamma = 1$) and the resulting decision value.

- The **black curve** is their sum, the decision value $f(x)$. It is positive on the right (the orange hill wins), negative on the left (the blue pit wins), and crosses **zero at** $x = 2$ (green dot), exactly midway, where both influences are equal.

In more dimensions it is the same picture: the decision boundary is where the hills and pits cancel, which can be a curve of any shape.

6.2.4 Choosing C and γ: Real Results

We trained RBF SVMs on the `moons` data (two interleaving half-circles with noise): 180 points to train and 120 to test.

γ	**Train accuracy**	**Test accuracy**	**Support vectors**	**Diagnosis**
0.1	0.839	0.867	79	too smooth: underfits
1	0.950	**0.992**	54	good
10	0.961	0.975	115	starting to overfit
100	**1.000**	0.850	173	overfits (memorises)

C ($\gamma = 1$)	**Train**	**Test**	**Kernel** ($C = 1$)	**Train**	**Test**
0.1	0.928	0.950	Linear	0.817	0.883
1	0.950	**0.992**	Polynomial (degree 3)	0.944	0.983
10	0.950	0.975	RBF	0.950	**0.992**
1000	0.967	0.933			

Interpretation. (i) With $\gamma = 100$ the training accuracy is a perfect 1.000 but the test accuracy falls to 0.850: the classic overfitting signature. (ii) Too small a γ underfits. (iii) Very large C

(1000) also overfits (0.967 train, 0.933 test). (iv) On this curved data the linear SVM is clearly worse (0.883) than the polynomial and RBF kernels. (Test accuracies are based on 120 points, so one point is 0.8%.)

Try It Yourself

Reproduce the experiment with `scikit-learn`:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC

X, y = make_moons(n_samples=300, noise=0.25, random_state=0)
Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.4,
                                  random_state=0)

for g in [0.1, 1, 10, 100]:
    model = make_pipeline(StandardScaler(),
                          SVC(kernel="rbf", gamma=g, C=1))
    model.fit(Xa, ya)
    print(g, model.score(Xa, ya), model.score(Xb, yb))
```

Challenge: replace the loop by `GridSearchCV` over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{0.1, 1, 10\}$ and report the best pair.

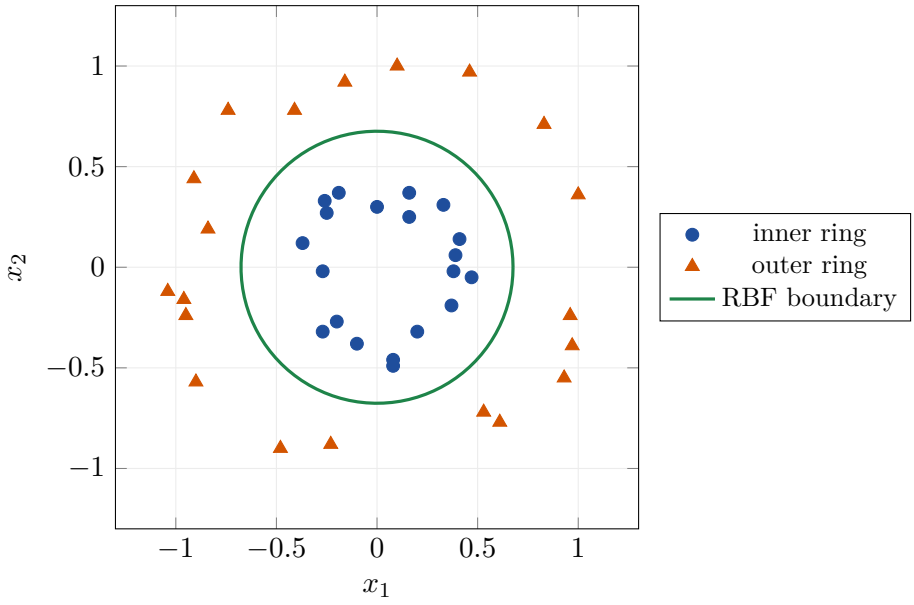


Figure 6.6. An RBF SVM separates two concentric rings that no straight line can (a real fit, 20 points of each class shown).

Reading the Figure

The blue circles form an **inner ring** and the orange triangles an **outer ring**. A straight line cannot separate them: a linear SVM got only 58% of these points right. The **RBF SVM** ($\gamma = 2$, $C = 10$) classified all of them correctly, and its decision boundary (green) is an almost perfect **circle of radius** ≈ 0.68 lying in the gap between the rings. The curved boundary was never specified by us: the kernel allowed the SVM to find it.

Key Idea

Choosing a kernel: start with a *linear* kernel (fast, good for many features and text). If the data look nonlinear, try the *RBF* kernel, which is a good default, and tune C and γ by cross-validation on *scaled* features. Use a polynomial kernel when you believe interactions between features of a known degree matter.

Nonlinear (kernel) SVM

Advantages

- Learns nonlinear boundaries without building features by hand
- Flexible: many kernels
- Works well in high dimensions
- Strong generalisation with the margin

Disadvantages

- Needs careful tuning of C, γ and the kernel
- Training cost grows roughly as n^2 to n^3: slow for $n \gg 10^5$
- Hard to interpret
- Needs feature scaling

Applications

- Face and object recognition
- Handwritten digit recognition
- Protein and DNA classification
- Remote-sensing image classification

6.3 SVM Regression

6.3.1 The Idea: Points Inside the Tunnel Are Free

The SVM idea also works for **regression**. In classification we wanted a road that keeps points *off* the road. In SVM regression we reverse it: we fit a **tunnel** (tube) around the regression line and try to get **as many points as possible inside**. Points inside the tunnel cost nothing; only points *outside* are penalised.

Definition: ε-insensitive loss

$$L_\varepsilon(y, f(\boldsymbol{x})) = \max\left(0,\ |y - f(\boldsymbol{x})| - \varepsilon\right).$$

Errors smaller than ε are ignored (loss 0); larger errors cost the amount *by which they exceed* ε.

Definition: Support vector regression (SVR)

SVR fits a function $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$ (or its kernelised version) that is as *flat* as possible while keeping most training points within a tube of half-width ε:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi},\boldsymbol{\xi}^*} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i(\xi_i + \xi_i^*) \quad \text{s.t.} \quad \begin{cases} y_i - f(\boldsymbol{x}_i) \le \varepsilon + \xi_i \\ f(\boldsymbol{x}_i) - y_i \le \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \ge 0. \end{cases}$$

The slack variables ξ_i, ξ_i^* measure how far a point lies above or below the tube. The **support vectors** are the points on or outside the tube; points strictly inside have no influence. With a kernel the prediction is $f(\boldsymbol{x}) = \sum_i(\alpha_i - \alpha_i^*)K(\boldsymbol{x}_i, \boldsymbol{x}) + b$.

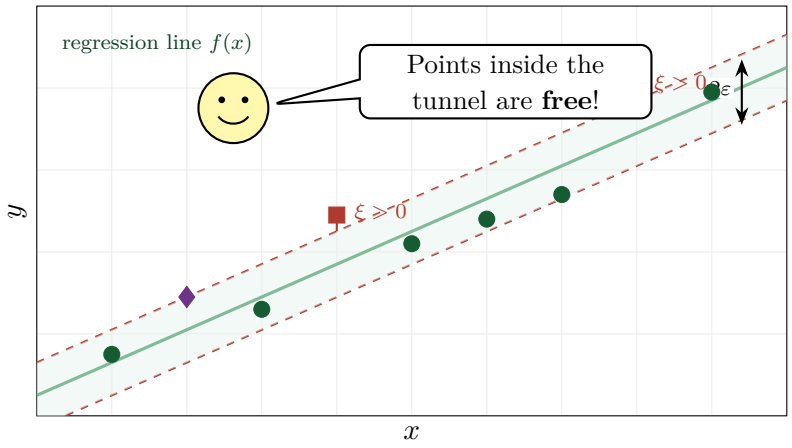


Figure 6.7. Support vector regression: a tube of half-width ε around the fitted line.

Reading the Figure

The **green line** is the fitted regression function and the **red dashed lines** are the tube edges, a distance ε above and below the line (total width 2ε).

- The **green dots** lie *inside* the tunnel. Their errors are smaller than ε, so they cost nothing and *do not affect* the line at all (they are not support vectors).
- The **red squares** lie *outside* the tunnel. They are penalised by the distance ξ to the nearest tube edge (the short red segments). They are support vectors.
- The **purple diamond** sits exactly on the tube edge and is also a support vector.

A wider tunnel (large ε) includes more points, gives a simpler model and fewer support vectors; a narrow tunnel follows the data more closely.

Example: The ε-insensitive loss

Let $\varepsilon = 0.5$. Three predictions:

	Point 1	Point 2	Point 3	Total
True y	3.0	4.1	6.5	
Predicted f	3.3	4.0	5.2	
Error $y - f$	-0.3	0.1	1.3	
$L_\varepsilon = \max(0, \lvert e\rvert - 0.5)$	0	0	0.8	**0.8**
Squared error e^2	0.09	0.01	1.69	1.79

Points 1 and 2 lie inside the tube and are *ignored*; only point 3 is charged ($1.3 - 0.5 = 0.8$). Ordinary squared error would have charged all three points.

6.3.2 Real Results

We fitted noisy data from $y = \sin x$ (60 points: 42 train, 18 test).

Model	**Train R^2**	**Test R^2**	**Support vectors**
Linear SVR	0.539	0.278	39
RBF SVR ($C = 10, \gamma = 1, \varepsilon = 0.1$)	**0.971**	**0.924**	26

ε (RBF SVR)	**Support vectors**	**Test R^2**	**Comment**
0.05	35	0.914	narrow tube, many support vectors
0.2	17	**0.942**	good balance
0.5	4	0.841	tube too wide: underfits

Read the tables. A straight line cannot follow the sine curve ($R^2 = 0.28$ on test data), while the RBF kernel can (0.92). Widening the tube from 0.05 to 0.5 reduces the number of support vectors from 35 to 4, but when the tube is too wide the fit gets worse.

Try It Yourself

```
import numpy as np
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
```

```
x = np.sort(rng.uniform(0, 6, 60))
y = np.sin(x) + rng.normal(0, 0.15, 60)
xa, xb, ya, yb = train_test_split(x.reshape(-1, 1), y,
                                  test_size=0.3, random_state=0)

svr = SVR(kernel="rbf", C=10, gamma=1, epsilon=0.1).fit(xa, ya)
print(svr.score(xa, ya), svr.score(xb, yb), len(svr.support_))
```

Challenge: try `epsilon=0.3` and `gamma=10` and compare the train and test R^2.

Parameter	Meaning	Effect when increased
ε	Half-width of the tube	Fewer support vectors, simpler (flatter) model
C	Penalty for points outside the tube	Fits training data more closely (overfits)
γ (RBF)	Reach of each support vector	More wiggly curve (overfits)

SVM regression (SVR)

Advantages

- Ignores small errors, so robust to noise
- Sparse: only support vectors matter
- Nonlinear fits through kernels
- Good generalisation

Disadvantages

- Three parameters to tune (ε, C, γ)
- Slow for very large datasets
- Needs feature scaling
- No probabilistic prediction interval

Applications

- Stock and sales forecasting
- Electricity-load and weather prediction
- Estimating material or engineering properties
- Travel-time estimation

6.4 Naive Bayes Classifiers

In Chapter 3 we introduced Bayes' theorem and the Naive Bayes classifier. Here we study the three main *variants*, one for each type of feature, and solve a full text-classification example. First recall the principle.

Definition: Naive Bayes classifier

A **Naive Bayes classifier** predicts the class

$$\hat{y} = \arg\max_{c} \; P(c) \prod_{j=1}^{d} P(x_j \mid c),$$

where the "naive" assumption is that the features are *conditionally independent given the class*. It is a **generative** classifier: it models how each class generates the data, $P(\boldsymbol{x} \mid c)$, and then uses Bayes' theorem to get $P(c \mid \boldsymbol{x})$.

6.4.1 The Detective Who Multiplies Clues

Think of the classifier as a **detective**. Each feature is a *clue*. The detective does not worry about how the clues are connected (the naive assumption); each clue is allowed to "speak" separately, and the detective **multiplies the votes** of all clues together with the prior suspicion.

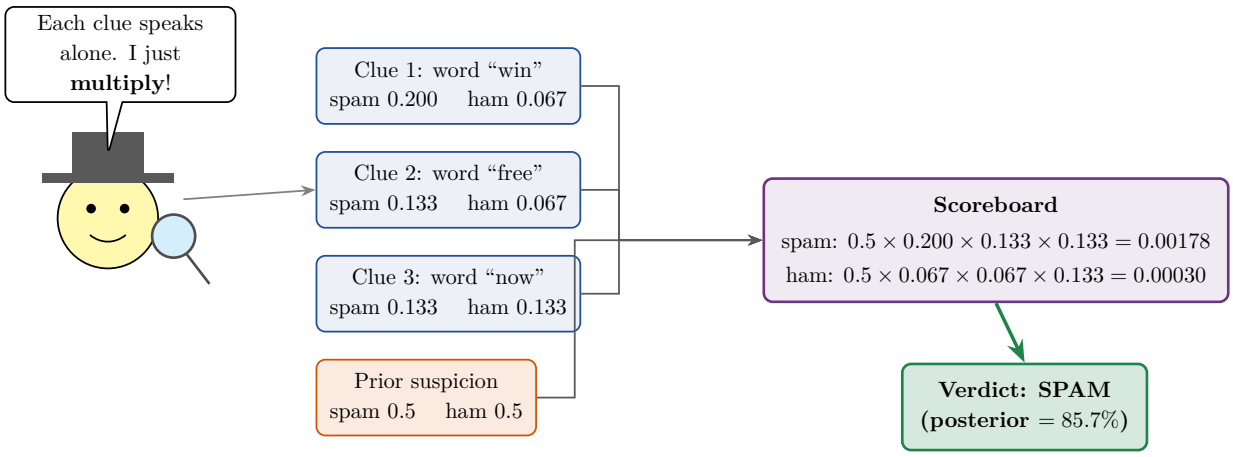


Figure 6.8. Naive Bayes as a detective: the evidence of each word is multiplied with the prior.

Reading the Figure

The detective is examining the message "**win free now**". The three blue cards are the clues, each giving how likely the word is under *spam* and under *ham* (probabilities estimated from a tiny training set, worked out in detail below). The orange card is the prior: before reading the message we think spam and ham are equally likely. The **purple scoreboard** multiplies the prior with each clue. Spam scores 0.00178 and ham only 0.00030, so the **verdict is spam**. The word "win" is the strongest clue because it is three times more likely in spam; the word "now" is no help at all (equal probability), so it simply cancels out.

6.4.2 Gaussian Naive Bayes (continuous features)

Definition: Gaussian Naive Bayes

For **continuous** features, $P(x_j \mid c)$ is modelled by a normal density with a *class-specific* mean μ_{jc} and variance σ_{jc}^2 estimated from the training data of class c:

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}} \exp\Big(- \frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \Big).$$

Real-life example: ripe or unripe mango. A fruit seller measures the *sweetness score* and the *colour* of mangoes. From past data: ripe mangoes have sweetness $\sim \mathcal{N}(6,1)$ and are yellow with probability 0.9; unripe mangoes have sweetness $\sim \mathcal{N}(3,1)$ and are yellow with probability 0.2. Both classes are equally common.

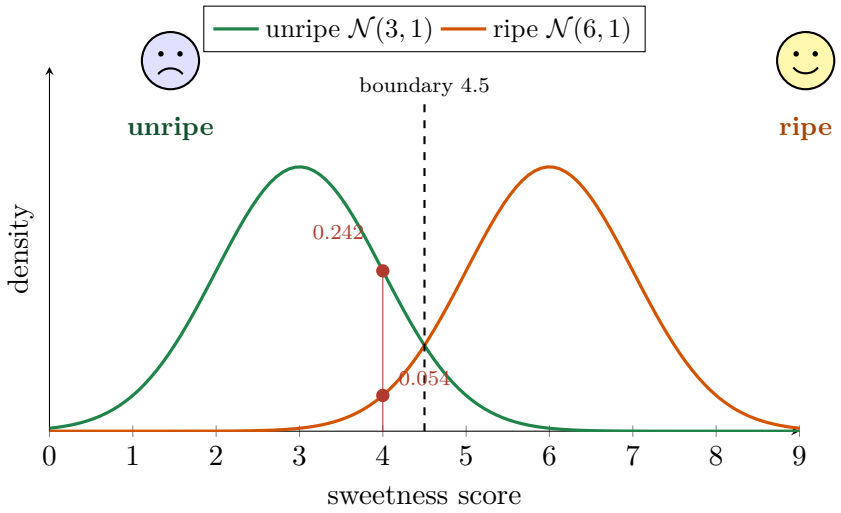


Figure 6.9. Class-conditional densities of sweetness for ripe and unripe mangoes (Gaussian Naive Bayes in one dimension).

Reading the Figure

Each bell curve shows how sweetness is distributed in one class: unripe mangoes cluster around 3 (green) and ripe ones around 6 (orange). The curves cross at 4.5, halfway between the means: with equal priors, a mango sweeter than 4.5 is classified ripe and a less sweet one unripe. For a mango of sweetness 4 (red dots) the unripe curve is higher (0.242) than the ripe curve (0.054), so the model leans towards *unripe*.

Example: Mango classification, one feature

A mango has sweetness $x = 4$. Equal priors 0.5 each.

$$P(4 \mid \text{ripe}) = \tfrac{1}{\sqrt{2\pi}}\, e^{-(4-6)^2/2} = 0.3989\, e^{-2} = 0.0540,$$

$$P(4 \mid \text{unripe}) = \tfrac{1}{\sqrt{2\pi}}\, e^{-(4-3)^2/2} = 0.3989\, e^{-0.5} = 0.2420.$$

$$P(\text{ripe} \mid x=4) = \frac{0.5(0.0540)}{0.5(0.0540)+0.5(0.2420)} = \frac{0.0540}{0.2960} = \mathbf{0.182} \;\Rightarrow\; \text{unripe}.$$

Effect of the prior. If ripe mangoes were four times as common (priors 0.8 ripe, 0.2 unripe): $P(\text{ripe} \mid x=4) = \frac{0.8(0.0540)}{0.8(0.0540)+0.2(0.2420)} = \frac{0.0432}{0.0916} = 0.472$, much closer to 0.5. A strong prior can move the decision.

Example: Mango classification, two features (sweetness and colour)

A mango has sweetness 4.8 and is **yellow**. Equal priors.

Quantity	**Ripe**	**Unripe**
Prior $P(c)$	0.5	0.5
$P(\text{sweetness} = 4.8 \mid c)$ (Gaussian)	$\varphi(-1.2) = 0.1942$	$\varphi(1.8) = 0.0790$
$P(\text{yellow} \mid c)$	0.9	0.2
Score $= P(c)\prod P(x_j \mid c)$	$0.5(0.1942)(0.9) = 0.0874$	$0.5(0.0790)(0.2) = 0.0079$

$$P(\text{ripe} \mid \text{data}) = \frac{0.0874}{0.0874 + 0.0079} = \mathbf{0.917} \;\Rightarrow\; \text{ripe}.$$

Teamwork of the clues: sweetness 4.8 alone is a moderately ripe-looking value ($P = 0.1942/(0.1942 + 0.0790) = 0.71$), and the yellow colour makes it much more convincing (0.917).

6.4.3 Multinomial Naive Bayes (word counts)

Definition: Multinomial Naive Bayes

For **count data** such as word counts in a document, a class c is described by a probability $P(w \mid c)$ for each word w in the vocabulary of size V. With Laplace smoothing α,

$$P(w \mid c) = \frac{N_{wc} + \alpha}{N_c + \alpha V},$$

where N_{wc} is the number of times word w appears in training documents of class c and N_c is the total number of word occurrences in class c. A document with words $w_1, \ldots, w_m$ is scored by $P(c)\prod_{k=1}^{m} P(w_k \mid c)$.

Real-life example: spam or ham SMS. A tiny training set of four messages:

Message	**Class**	**Words**
“win cash now”	spam	win, cash, now
“win free prize”	spam	win, free, prize
“meet me now”	ham	meet, me, now
“lunch with me”	ham	lunch, with, me

Example: Multinomial Naive Bayes, end to end

Step 1: priors. 2 spam and 2 ham messages: $P(\text{spam}) = P(\text{ham}) = 0.5$.

Step 2: counts. The vocabulary has $V = 9$ words: cash, free, lunch, me, meet, now, prize, win, with. Each class has $N_c = 6$ word occurrences.

Step 3: smoothed probabilities with $\alpha = 1$, so every denominator is $6 + 9 = 15$:

Word	win	cash	now	free	prize	meet	me	lunch	with
Count in spam	2	1	1	1	1	0	0	0	0
$P(w \mid \text{spam})$	$\frac{3}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$
Count in ham	0	0	1	0	0	1	2	1	1
$P(w \mid \text{ham})$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{2}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{2}{15}$	$\frac{3}{15}$	$\frac{2}{15}$	$\frac{2}{15}$

Step 4: classify "win free now".

$$\text{score}(\text{spam}) = 0.5 \cdot \tfrac{3}{15} \cdot \tfrac{2}{15} \cdot \tfrac{2}{15} = 0.5 \cdot \tfrac{12}{3375} = 0.001778,$$
$$\text{score}(\text{ham}) = 0.5 \cdot \tfrac{1}{15} \cdot \tfrac{1}{15} \cdot \tfrac{2}{15} = 0.5 \cdot \tfrac{2}{3375} = 0.000296.$$

$$P(\text{spam} \mid \text{message}) = \frac{12}{12+2} = \frac{6}{7} = \mathbf{0.857} \;\Rightarrow\; \textbf{spam}.$$

Step 5: classify "meet me now". spam: $0.5 \cdot \frac{1}{15} \cdot \frac{1}{15} \cdot \frac{2}{15} \to 2$ units; ham: $0.5 \cdot \frac{2}{15} \cdot \frac{3}{15} \cdot \frac{2}{15} \to 12$ units. So $P(\text{ham}) = \frac{12}{14} = 0.857$: **ham**.

Why smoothing matters. Without smoothing, $P(\text{win} \mid \text{ham}) = \frac{0}{6} = 0$, and any message containing "win" would get a ham score of exactly 0, whatever its other words. With $\alpha = 1$ it is $\frac{1}{15}$: small but not zero.

6.4.4 Bernoulli Naive Bayes (word present or absent)

Definition: Bernoulli Naive Bayes

For **binary** features $x_j \in \{0, 1\}$ (word present or absent), each feature has a probability $p_{jc} = P(x_j = 1 \mid c)$ and

$$P(x_j \mid c) = p_{jc}^{x_j} (1 - p_{jc})^{1 - x_j}, \qquad p_{jc} = \frac{n_{jc} + \alpha}{N_c^{\text{docs}} + 2\alpha},$$

where n_{jc} is the number of documents of class c containing feature j. Unlike the multinomial model, it *explicitly uses the absence of a word* as evidence.

Example: Bernoulli Naive Bayes on the same messages

Each class has 2 documents, so $p_{jc} = \frac{n_{jc}+1}{4}$.

Word	win	cash	now	free	prize	meet	me	lunch	with
$P(\text{present} \mid \text{spam})$	0.75	0.5	0.5	0.5	0.5	0.25	0.25	0.25	0.25
$P(\text{present} \mid \text{ham})$	0.25	0.25	0.5	0.25	0.25	0.5	0.75	0.5	0.5

Classify "win free now": present words win, free, now; **absent** words cash, prize, meet, me, lunch, with.

$$\text{spam: } 0.5 \times \underbrace{0.75 \cdot 0.5 \cdot 0.5}_{\text{present}} \times \underbrace{0.5 \cdot 0.5 \cdot 0.75 \cdot 0.75 \cdot 0.75 \cdot 0.75}_{\text{absent}} = 0.01483,$$

$$\text{ham: } 0.5 \times \underbrace{0.25 \cdot 0.25 \cdot 0.5}_{\text{present}} \times \underbrace{0.75 \cdot 0.75 \cdot 0.5 \cdot 0.25 \cdot 0.5 \cdot 0.5}_{\text{absent}} = 0.00055.$$

$P(\text{spam}) = \dfrac{0.01483}{0.01483 + 0.00055} = \mathbf{0.964}$: spam, with an even higher confidence than the multinomial model, because the *absence* of ham-typical words (me, meet, lunch, with) is also evidence.

Variant	**Feature type**	**Models $P(x_j \mid c)$ by**	**Typical use**
Gaussian	Continuous measurements	Normal density per class	Medical measurements, sensors
Multinomial	Counts (word counts, frequencies)	Word probabilities with smoothing	Text classification, spam
Bernoulli	Binary (present/absent)	Presence probability per feature	Short texts, binary indicators

Try It Yourself

Confirm the hand calculations with `scikit-learn`:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB

docs = ["win cash now", "win free prize", "meet me now",
        "lunch with me"]
labels = [1, 1, 0, 0]                          # 1 = spam, 0 = ham

cv = CountVectorizer(binary=False)
mnb = MultinomialNB(alpha=1).fit(cv.fit_transform(docs), labels)
print(mnb.predict_proba(cv.transform(["win free now"])))
```

```
# -> [[0.143 0.857]]   P(ham), P(spam)

cvb = CountVectorizer(binary=True)
bnb = BernoulliNB(alpha=1).fit(cvb.fit_transform(docs), labels)
print(bnb.predict_proba(cvb.transform(["win free now"])))
# -> [[0.036 0.964]]
```

The outputs, 0.857 (multinomial) and 0.964 (Bernoulli) for "spam", agree with the hand calculation.

6.4.5 Generative vs. Discriminative Classifiers

	Naive Bayes (generative)	**Logistic regression / SVM (discriminative)**
What is learned	$P(\boldsymbol{x} \mid c)$ and $P(c)$	$P(c \mid \boldsymbol{x})$ or the boundary directly
Training	Counting; very fast	Optimisation; slower
Data needed	Works with little data	Needs more data, but usually more accurate with plenty
Feature dependence	Assumed independent	Can use correlated features
Probabilities	Often over-confident	Logistic: better calibrated

Common Mistake

Naive Bayes often classifies well even when its probabilities are *wrong*. If the posterior says 0.999, do not trust that number literally: the independence assumption multiplies correlated evidence several times and makes the model over-confident. Use it to *rank* or *decide*, not to quote exact probabilities.

Naive Bayes classifiers

Advantages

- Extremely fast to train and predict
- Works with little training data and many features
- Handles multiple classes naturally
- Excellent baseline for text

Disadvantages

- Independence assumption is rarely true
- Probabilities over-confident
- Zero-frequency problem (needs smoothing)
- Cannot learn feature interactions

Applications

- Spam filtering and sentiment analysis
- News and document categorisation
- Medical diagnosis (Gaussian)
- Real-time prediction

6.5 Comparing the Methods of Unit III

	Kernel SVM	Naive Bayes	Random Forest / Boosting
Training speed	Slow for large n	**Very fast**	Moderate
Prediction speed	Fast (few support vectors)	**Very fast**	Moderate
Nonlinear?	Yes (kernel)	Mild	**Yes**
Needs scaling?	**Yes**	No	No
Probabilities?	Not directly	Yes (over-confident)	Yes
Works with little data	Yes	**Yes**	Less so
Interpretability	Low	Moderate	Low–moderate

Chapter Summary

- **Linear SVM:** the maximum-margin hyperplane; margin $2/\|\boldsymbol{w}\|$; support vectors lie on the road edge and alone determine the model.
- **Soft margin:** slack ξ_i and parameter C; large C = strict (narrow margin, overfit), small C = relaxed (wide margin, underfit).
- **Dual form:** $f(\boldsymbol{x}) = \sum \alpha_i y_i (\boldsymbol{x}_i \cdot \boldsymbol{x}) + b$; only support vectors have $\alpha_i > 0$; data appear only through dot products.
- **Nonlinear SVM:** lift the data with φ; the **kernel trick** replaces dot products by $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. Polynomial: $(\gamma \boldsymbol{a} \cdot \boldsymbol{b} + r)^d$; RBF: $e^{-\gamma \|\boldsymbol{a}-\boldsymbol{b}\|^2}$. Large γ or C overfits.
- **SVR:** ε-tube; loss $\max(0, |y - f| - \varepsilon)$; points inside the tube are free; support vectors are on or outside.
- **Naive Bayes:** $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$; Gaussian (continuous), multinomial (counts), Bernoulli (binary); use Laplace smoothing $\frac{N_{wc}+\alpha}{N_c+\alpha V}$.
- Worked results: SVM lifted boundary at $x = \pm 1.90$; RBF kernel $e^{-0.5 \cdot 5} = 0.082$; SMS "win free now" spam with 0.857 (multinomial) and 0.964 (Bernoulli); mango ripe with 0.917.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define a support vector machine. **[2 marks]**
2. What are support vectors? What is the margin? **[2 marks]**

3. What is the role of the parameter C in a soft-margin SVM? **[2 marks]**
4. Write the decision function of an SVM in dual form. **[2 marks]**
5. What is the kernel trick? **[2 marks]**
6. Write the formulas of the polynomial and RBF kernels. **[2 marks]**
7. What does a large γ do in the RBF kernel? **[2 marks]**
8. Define the ε-insensitive loss. **[2 marks]**
9. What is the effect of increasing ε in SVR? **[2 marks]**
10. State the independence assumption of Naive Bayes. **[2 marks]**
11. What is Laplace smoothing? **[2 marks]**
12. Name the three types of Naive Bayes and their feature types. **[2 marks]**

Model Answer: Q5, Q7 and Q9

Q5. The kernel trick is the replacement of every dot product $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$ in the SVM by a kernel $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_j)$. The SVM then works in a high-dimensional feature space without ever computing the mapping φ, which makes nonlinear classification possible at low cost.

Q7. A large γ makes the similarity fall off very quickly with distance: every training point influences only a tiny neighbourhood, producing a very wiggly decision boundary. It reduces bias but increases variance, so it *overfits*.

Q9. A larger ε gives a wider tube; more points fall inside, fewer are support vectors, and the model becomes simpler (flatter). If ε is too large the model underfits.

B. Short Answer Questions (5 marks each)

1. Explain the maximum-margin classifier and support vectors with a diagram. **[5 marks]**
2. Explain the soft-margin SVM. Discuss the effect of C. **[5 marks]**
3. Explain the dual form of the SVM and why it is important for kernels. **[5 marks]**
4. Why do we need nonlinear SVMs? Explain the lifting idea with an example. **[5 marks]**
5. Explain the kernel trick. Describe the linear, polynomial and RBF kernels. **[5 marks]**
6. Explain the effect of C and γ on an RBF SVM. **[5 marks]**
7. Explain SVM regression with the ε-tube. **[5 marks]**
8. Explain the Gaussian, multinomial and Bernoulli Naive Bayes classifiers. **[5 marks]**
9. Explain why Laplace smoothing is needed in Naive Bayes with an example. **[5 marks]**
10. Compare generative and discriminative classifiers. **[5 marks]**

Model Answer: B6 (outline for 5 marks)

Parameter C (penalty for margin violations): large $C \to$ few violations, narrow margin, complex boundary, *low bias, high variance* (overfit); small $C \to$ wide margin, simple boundary, *higher bias, low variance* (underfit).
Parameter γ (reach of each support vector in $e^{-\gamma\|a-b\|^2}$): large $\gamma \to$ narrow influence, wiggly boundary, overfit; small $\gamma \to$ wide influence, smooth boundary, underfit.
Example: on the moons data ($\gamma = 0.1, 1, 100$) test accuracy was 0.867, 0.992, 0.850 while training accuracy rose to 1.0 at $\gamma = 100$. Both parameters are chosen by cross-validation on scaled features.

C. Long Answer Questions (10 marks each)

1. Explain linear SVM classification: the margin, the optimisation problem, support vectors, soft margin and the dual form. Solve a small numerical example. **[10 marks]**
2. Explain nonlinear SVM classification. Describe the kernel trick with the polynomial and RBF kernels, including a numerical verification that $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. **[10 marks]**
3. Explain SVM regression: the ε-insensitive loss, the optimisation problem, support vectors and the roles of ε, C and γ. **[10 marks]**
4. Explain the Naive Bayes classifier. Classify a message using the multinomial model with Laplace smoothing for a given training set. **[10 marks]**
5. Describe the Gaussian, multinomial and Bernoulli Naive Bayes classifiers with formulas, examples, advantages and disadvantages. **[10 marks]**
6. Compare SVM, Naive Bayes and ensemble methods (Random Forest / boosting) for classification. **[10 marks]**

Model Answer: C2 (numerical verification)

Take $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (3, 1)$ and $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b})^2$.
Via the kernel: $\boldsymbol{a} \cdot \boldsymbol{b} = 1(3) + 2(1) = 5$, $K = 25$.
Via the feature map $\varphi(\boldsymbol{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$: $\varphi(\boldsymbol{a}) = (1, 2\sqrt{2}, 4)$, $\varphi(\boldsymbol{b}) = (9, 3\sqrt{2}, 1)$; $\varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b}) = 9 + 2\sqrt{2} \cdot 3\sqrt{2} + 4 = 9 + 12 + 4 = 25$.
Both give 25, so the kernel equals the dot product in the 3-D feature space while needing only one 2-D dot product.

D. Numerical Problems

1. An SVM has $\boldsymbol{w} = (2, 1)$, $b = -4$. Classify $(1, 1)$ and $(3, 2)$ and find the margin. *(Answer:* $f(1, 1) = -1 \Rightarrow -1$ *(on the margin);* $f(3, 2) = 4 \Rightarrow +1$*; margin* $= 2/\sqrt{5} = 0.894$*.)*

2. In 1-D the support vectors are $x = 1$ ($y = -1$) and $x = 3$ ($y = +1$). Find w, b, the margin and the dual values α_i. *(Answer:* $w + b = -1$, $3w + b = 1 \Rightarrow w = 1$, $b = -2$*; margin* $= 2$*;* $w = \sum \alpha_i y_i x_i = 3\alpha - \alpha = 2\alpha = 1 \Rightarrow \alpha = 0.5$ *each; check* $\sum \alpha_i y_i = 0$*.)*

3. Using the dual weights $\alpha = 0.5$ at $x = 1$ ($y = -1$) and $x = 3$ ($y = +1$) with $b = -2$, compute $f(2.5)$. *(Answer:* $0.5(-1)(1 \cdot 2.5) + 0.5(1)(3 \cdot 2.5) - 2 = -1.25 + 3.75 - 2 = 0.5 > 0$*.)*

4. Compute the polynomial kernel $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b} + 1)^2$ for $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (2, 1)$. *(Answer:* $\boldsymbol{a} \cdot \boldsymbol{b} = 4$*;* $K = 25$*.)*

5. Compute the RBF kernel for $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (2, 1)$ with $\gamma = 0.5$. *(Answer:* $\|\boldsymbol{a} - \boldsymbol{b}\|^2 = 2$*;* $K = e^{-1} = 0.368$*.)*

6. With $\varphi(x) = (x, x^2)$ compute $\varphi(2) \cdot \varphi(3)$ and the matching kernel $K(2, 3) = 2 \cdot 3 + (2 \cdot 3)^2$. *(Answer:* $\varphi(2) \cdot \varphi(3) = (2, 4) \cdot (3, 9) = 6 + 36 = 42$*;* $K = 6 + 36 = 42$*.)*

7. In SVR with $\varepsilon = 0.3$: $y = (2.0, 3.0, 5.0)$, $f = (2.2, 3.5, 4.1)$. Find the total ε-insensitive loss. *(Answer: errors* $-0.2, -0.5, 0.9$*; losses* $0, 0.2, 0.6$*; total* 0.8*.)*

8. Gaussian NB: ripe $\sim \mathcal{N}(6, 1)$, unripe $\sim \mathcal{N}(3, 1)$, equal priors. Find $P(\text{ripe} \mid x = 4)$. *(Answer: densities* 0.0540 *and* 0.2420*; posterior* 0.182*.)*

9. Using the SMS example, classify "win now" with multinomial NB. *(Answer: spam* $0.5 \cdot \frac{3}{15} \cdot \frac{2}{15} = 0.01333$*; ham* $0.5 \cdot \frac{1}{15} \cdot \frac{2}{15} = 0.00444$*;* $P(\text{spam}) = 0.75$*.)*

10. A word never occurs in the 200 word-tokens of a class; the vocabulary has 1000 words. Find its Laplace-smoothed probability ($\alpha = 1$). *(Answer:* $\frac{0+1}{200+1000} = 0.00083$*.)*

E. Multiple-Choice Questions

1. The support vectors of an SVM are:
   (a) all training points (b) points on or inside the margin (c) the outliers only (d) the test points

2. The margin of a linear SVM equals:
   (a) $\|\boldsymbol{w}\|$ (b) $1/\|\boldsymbol{w}\|$ (c) $2/\|\boldsymbol{w}\|$ (d) $\|\boldsymbol{w}\|^2$

3. A very large C in a soft-margin SVM tends to:
   (a) widen the margin (b) narrow the margin and overfit (c) remove all support vectors (d) make the model linear

4. In the dual form the data appear only through:
   (a) squares (b) dot products (c) means (d) labels

5. The RBF kernel is:
   (a) $\boldsymbol{a} \cdot \boldsymbol{b}$ (b) $(\boldsymbol{a} \cdot \boldsymbol{b} + 1)^2$ (c) $e^{-\gamma\|\boldsymbol{a}-\boldsymbol{b}\|^2}$ (d) $\tanh(\boldsymbol{a} \cdot \boldsymbol{b})$

6. A very large γ in an RBF SVM leads to:
   (a) underfitting (b) overfitting (c) a linear boundary (d) fewer features

7. In SVR, points strictly inside the ε-tube:
   (a) are support vectors (b) have no effect on the model (c) have the largest loss (d) are

removed from the data

8. Increasing ε in SVR typically:
   (a) increases support vectors (b) decreases support vectors (c) changes nothing (d) adds features

9. Naive Bayes assumes features are:
   (a) identical (b) conditionally independent given the class (c) all Gaussian (d) uncorrelated with the class

10. Which variant suits word counts?
    (a) Gaussian (b) Multinomial (c) Linear (d) Sigmoid

11. Laplace smoothing prevents:
    (a) overfitting of trees (b) zero probabilities (c) scaling problems (d) ties

12. Which classifier is *generative*?
    (a) SVM (b) logistic regression (c) Naive Bayes (d) k-NN

Answers: 1-b, 2-c, 3-b, 4-b, 5-c, 6-b, 7-b, 8-b, 9-b, 10-b, 11-b, 12-c.

F. Practice Problems

1. Using the 1-D lifted example, classify $x = -2$ and $x = 0.5$ with $f(x) = 0.381x^2 - 1.381$. *(Answer:* $f(-2) = 0.143 \Rightarrow$ *orange;* $f(0.5) = -1.286 \Rightarrow$ *blue.)*

2. Draw the margin and support vectors for the data $(1,1), (2,2)$ in class -1 and $(4,4), (5,5)$ in class $+1$. Find $\boldsymbol{w}$, b and the margin. *(Hint: closest pair* $(2,2)$ *and* $(4,4)$*.)*

3. **Coding:** train `SVC` with linear, polynomial and RBF kernels on `make_circles` and report the accuracies.

4. **Coding:** use `GridSearchCV` to tune C and γ of an RBF SVM on the *breast cancer* dataset (after scaling).

5. **Coding:** write your own spam examples and train `MultinomialNB` with `alpha=1` and `alpha=0.01`; compare predicted probabilities.

Quick Revision

Topic	One-line recall
Linear SVM	Max-margin hyperplane; margin $2/\|w\|$; min $\frac{1}{2}\|w\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$.
Support vectors	Points on / inside the margin; only they determine w, b.
Soft margin	Slack ξ_i, penalty C: large C strict (overfit), small C relaxed (underfit).
Dual form	$f(x) = \sum \alpha_i y_i (x_i \cdot x) + b$; $\alpha_i > 0$ only for support vectors.
Kernel trick	Replace dot product by $K(a, b) = \varphi(a) \cdot \varphi(b)$.
Kernels	Linear $a \cdot b$; poly $(\gamma a \cdot b + r)^d$; RBF $e^{-\gamma \|a-b\|^2}$.
C, γ	Large C / large γ: overfit; small: underfit.
SVR	ε-tube; loss $\max(0, \|y - f\| - \varepsilon)$; points inside are free.
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$; generative; conditional independence.
Variants	Gaussian (continuous), multinomial (counts), Bernoulli (binary).
Smoothing	$\frac{N_{wc}+\alpha}{N_c+\alpha V}$ avoids zero probabilities.